

Università degli Studi di Trento

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Reti

Lezioni, Lab, e Simulazioni d'esame

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la versione Dark Mode.

novembre 2026

Indice

1 Capitolo 0 – Organizzazione del corso **4**
1.1 Informazioni essenziali . . . 4
1.2 Modalità d'esame . . . 4

2 Capitolo 1 – Introduzione alle reti **4**
2.1 Obiettivi e struttura del capitolo . . . 4
2.2 1.1 – Che cos'è Internet? . . . 5
2.2.1 Vista "hardware": i componenti di base . . . 5
2.2.2 Vista "a protocolli" . . . 5
2.2.3 Vista "a servizi" . . . 6
2.3 1.2 – Ai confini della rete . . . 6
2.3.1 Sistemi terminali e architetture applicative . . . 6
2.3.2 Reti di accesso e mezzi fisici . . . 6
2.3.3 Mezzi trasmissivi . . . 7
2.4 1.3 – Il nucleo della rete . . . 9
2.4.1 Commutazione di circuito . . . 9
2.4.2 Commutazione di pacchetto . . . 10
2.4.3 Struttura di Internet: la rete delle reti . . . 11
2.5 1.4 – Ritardo, perdita e throughput . . . 12
2.5.1 Le quattro componenti del ritardo di nodo . . . 12
2.5.2 Ritardo di accodamento e intensità di traffico . . . 13
2.5.3 traceroute e i ritardi reali di Internet . . . 14
2.5.4 Perdita di pacchetti . . . 14
2.5.5 Throughput . . . 15
2.5.6 Esercizio riassuntivo: bandwidth-delay product . . . 16
2.6 1.5 – Livelli di protocollo e modelli di servizio . . . 16
2.6.1 Perché la stratificazione? . . . 16
2.6.2 Formalismo: layer, entità, SAP, protocollo . . . 17
2.6.3 Lo stack Internet e il modello di riferimento OSI . . . 17
2.6.4 Data unit e incapsulamento . . . 19
2.7 1.6 – Sicurezza di rete (cenni – non materia d'esame) . . . 19
2.8 1.7 – Cenni storici (non materia d'esame) . . . 20
2.9 Riepilogo del capitolo 1 . . . 20

3 Capitolo 2 – Il livello applicazione **21**
3.1 Obiettivi e argomenti del capitolo . . . 21
3.2 2.1 – Principi delle applicazioni di rete . . . 21
3.2.1 Creare un'applicazione di rete . . . 21
3.2.2 Architetture delle applicazioni di rete . . . 21
3.2.3 Processi in comunicazione . . . 22
3.2.4 Indirizzamento . . . 22
3.2.5 Protocolli a livello applicazione . . . 23
3.2.6 Quale servizio di trasporto serve a un'applicazione? . . . 23
3.3 2.2 – Web e HTTP . . . 24
3.3.1 Terminologia . . . 24
3.3.2 Panoramica su HTTP . . . 24
3.3.3 Connessioni non persistenti e persistenti . . . 25

3.3.4 Formato dei messaggi HTTP . . . 25
3.3.5 Cookie . . . 27
3.3.6 Cache Web (server proxy) e GET condizionale . . . 27
3.3.7 Da HTTP/1.0/1.1 a HTTP/2 . . . 28
3.3.8 HTTPS e TLS . . . 29
3.4 2.3 – FTP . . . 29
3.5 2.4 – Posta elettronica . . . 30
3.5.1 SMTP [RFC 2821] . . . 30
3.5.2 Formato dei messaggi e MIME . . . 31
3.5.3 Protocolli di accesso alla posta: POP3 e IMAP . . . 31
3.6 2.5 – DNS (Domain Name System) . . . 32
3.6.1 Gerarchia dei server DNS . . . 33
3.6.2 Caching DNS . . . 34
3.6.3 Resource Record (RR) e messaggi DNS . . . 34
3.7 2.6 – Applicazioni P2P: condivisione di file . . . 35
3.7.1 Client-server vs P2P: analisi delle prestazioni . . . 35
3.7.2 BitTorrent . . . 36
3.7.3 Ricerca di informazioni in un sistema P2P . . . 37
3.7.4 Caso di studio: Skype . . . 37
3.8 2.7 – Cloud computing . . . 38
3.8.1 Data center . . . 38
3.8.2 Content Delivery Network (CDN) . . . 38
3.9 2.8 – Programmazione delle socket (cenni) . . . 38
3.10 Riepilogo del capitolo 2 . . . 39

4 Capitolo 3 – Il livello di trasporto 39
4.1 3.1 – Servizi a livello di trasporto . . . 39
4.2 3.2 – Multiplexing e demultiplexing . . . 40
4.2.1 Demultiplexing senza connessione (UDP) . . . 40
4.2.2 Demultiplexing orientato alla connessione (TCP) . . . 40
4.3 3.3 – Trasporto senza connessione: UDP [RFC 768] . . . 40
4.3.1 Checksum UDP . . . 41
4.4 3.4 – Principi del trasferimento dati affidabile . . . 42
4.4.1 ARQ e Stop-and-Wait . . . 42
4.4.2 Pipelining: finestre di trasmissione e ricezione . . . 42
4.5 3.5 – Trasporto orientato alla connessione: TCP [RFC 793, 1122, 1323, 2018, 2581] . . . 44
4.5.1 Struttura del segmento TCP . . . 44
4.5.2 Numeri di sequenza e ACK . . . 44
4.5.3 Setup della connessione: three-way handshake . . . 45
4.5.4 Maximum Segment Size (MSS) . . . 45
4.5.5 Chiusura della connessione . . . 45
4.5.6 Stima di RTT e Retransmission TimeOut (RTO) . . . 46
4.5.7 Controllo di flusso . . . 46
4.6 3.6 – Principi del controllo di congestione . . . 46
4.7 3.7 – Controllo di congestione TCP . . . 47
4.7.1 AIMD – Additive Increase, Multiplicative Decrease . . . 47
4.7.2 Le tre finestre di TCP . . . 48

4.7.3 Slow start e Congestion avoidance . . . 48
4.7.4 Fast Retransmit e Fast Recovery . . . 48
4.7.5 Formula approssimata del throughput TCP . . . 49
4.7.6 Equità (fairness) di TCP . . . 50
4.7.7 Oltre TCP loss-based: CUBIC, BBR, QUIC . . . 50
4.8 Riepilogo del capitolo 3 . . . 51

5 Capitolo 4 – Livello di rete (Parte 1: piano dati e protocollo IPv4) 51
5.1 Visione d'insieme . . . 51
5.1.1 Le due funzioni del livello di rete . . . 51
5.1.2 Data plane e control plane . . . 51
5.2 Come è fatto un router . . . 52
5.2.1 Porte di ingresso . . . 52
5.2.2 Sistemi di commutazione . . . 52
5.2.3 Porte di uscita e scheduling . . . 52
5.3 Il protocollo IP . . . 53
5.3.1 Formato del datagramma IPv4 . . . 53
5.3.2 Frammentazione . . . 54
5.3.3 Indirizzamento IP . . . 54
5.3.4 NAT – Network Address Translation . . . 55
5.3.5 Indirizzi IP speciali . . . 56
5.3.6 ARP – Address Resolution Protocol . . . 57
5.3.7 ICMP – Internet Control Message Protocol . . . 57
5.3.8 DHCP – Dynamic Host Configuration Protocol . . . 58
5.3.9 Il viaggio di un pacchetto . . . 59
5.3.10 IPv6 . . . 59
5.4 Riepilogo (Parte 1) . . . 60
5.5 Protocolli di instradamento (Parte 2: come si costruiscono le tabelle) . . . 60
5.5.1 Il problema come grafo . . . 60
5.5.2 Algoritmo di Dijkstra (link state) . . . 61
5.5.3 Sistemi Autonomi e OSPF . . . 62
5.5.4 Algoritmo di Bellman-Ford (distance vector) . . . 62
5.5.5 RIP – Routing Information Protocol . . . 63
5.5.6 Link state vs. distance vector: confronto . . . 64
5.5.7 BGP – Border Gateway Protocol . . . 64
5.5.8 Network neutrality . . . 66
5.6 Riepilogo completo del Capitolo 4-5 (livello di rete) . . . 66

Appendice A – Banca domande d'esame 66

Appendice B – Esercizi d'esame svolti 71

Appendice C – Cosa viene chiesto all'orale 77

6 Appendice D: Esempio di Prova d'Esame Completa con Soluzioni (AA 2018/19) 78

7 Appendice E: Esercizio "Company" — Configurazione di Rete (traccia d'esame, soluzione proposta) 82

1 Capitolo 0 – Organizzazione del corso

1.1 Informazioni essenziali

- **Testo di riferimento**: *Computer Networking: A Top-Down Approach*, 8th edition, J. Kurose & K. Ross, Pearson. Si consiglia la versione in inglese.
- **Materiale**: slide su DidatticaOnLine/Moodle; integrazioni anche da altri corsi di "Reti" UniTN (Granelli, Lo Cigno, Segata).
- **Software di laboratorio**:
  - **Wireshark** – per ispezionare gli scambi di messaggi in rete (https://www.wireshark.org/)
  - **Kathará** – per configurare e testare protocolli di rete (https://www.kathara.org/)

1.2 Modalità d'esame

Struttura della valutazione

- **Esame scritto** a domande aperte – peso 60% (1-2 domande che richiedono la soluzione di un problema)
- **Esame di laboratorio** – peso 40% (prova pratica con Wireshark e Kathará)
- **Prova orale** – ± 3 punti circa; accesso solo con media pesata scritto+lab ≥ 18; 10-15 minuti a testa
- **Vincolo importante**: tutte le prove vanno superate nello stesso appello
- 5 appelli l'anno, come da regolamento; la lode è a discrezione del docente

Nota: verifica sempre su Moodle/DidatticaOnLine eventuali aggiornamenti a questa modalità, poiché il regolamento può cambiare tra un anno accademico e l'altro.

2 Capitolo 1 – Introduzione alle reti

2.1 Obiettivi e struttura del capitolo

Questo capitolo introduce la terminologia e i concetti di base delle reti di telecomunicazioni. L'approccio del corso è **top-down**: si parte dalla vista d'insieme (Internet, i suoi servizi) per poi scendere via via nei livelli più bassi nei capitoli successivi. Gli argomenti trattati sono:

1.1 Cos'è Internet?

1.2 Ai confini della rete: sistemi terminali, reti di accesso, collegamenti

1.3 Il nucleo della rete: commutazione di circuito e di pacchetto, struttura di Internet

1.4 Ritardi, perdite e throughput nelle reti a commutazione di pacchetto

1.5 Livelli di protocollo e loro modelli di servizio

1.6 Reti sotto attacco: la sicurezza

1.7 Storia del computer networking e di Internet

2.2 1.1 – Che cos'è Internet?

Ci sono tre modi complementari di rispondere a questa domanda: guardando ai **componenti fisici**, ai **protocolli** che li governano, oppure ai **servizi** che Internet offre.

2.2.1 Vista "hardware": i componenti di base

- **Milioni di dispositivi collegati**, chiamati **host** o **sistemi terminali** (*end system*): PC, server, portatili, smartphone, ma anche oggetti IoT.
- Gli host eseguono le **applicazioni di rete**.
- I **collegamenti** (link) trasportano i dati tra i dispositivi: possono essere via rame, fibra ottica o onde elettromagnetiche. La loro capacità di trasmissione si misura in **ampiezza di banda** (*bandwidth*, bit/s).
- I **router** instradano i pacchetti verso la loro destinazione finale, seguendo un percorso attraverso una rete di collegamenti.
- L'**ISP** (*Internet Service Provider*) è l'ente che fornisce l'accesso a Internet: reti domestiche, aziendali e mobili si collegano a Internet tramite ISP locali, che a loro volta si connettono a ISP di livello superiore (si veda la Sezione 1.3.3).

2.2.2 Vista "a protocolli"

Definizione: protocollo

Un **protocollo** definisce il **formato** e l'**ordine** dei messaggi scambiati tra due o più entità in comunicazione, così come le **azioni intraprese** in fase di trasmissione e/o ricezione di un messaggio o di un altro evento.

Esattamente come esistono *protocolli umani* (es. "che ore sono?", una presentazione, il galateo di una telefonata), esistono **protocolli di rete**: qui non sono persone a scambiarsi messaggi, ma **dispositivi hardware e software**. Tutta l'attività di comunicazione in Internet è governata da protocolli (es. TCP, IP, HTTP, Ethernet).

L'analogia è diretta: così come un protocollo umano prevede uno scambio del tipo "Ciao" / "Ciao" / "Sai l'ora?" / "17:00", un protocollo di rete prevede scambi come richiesta di connessione TCP / risposta di connessione TCP / `GET http://...` / `<file>`. In entrambi i casi, **si mandano messaggi specifici e si agisce diversamente a seconda delle risposte ricevute**.

Internet può essere definita come la "*rete delle reti*": ha una **struttura gerarchica**, comprende sia la Internet pubblica sia le intranet private, ed è governata da **standard** definiti tramite **RFC** (*Request for Comments*) dalla **IETF** (*Internet Engineering Task Force*).

2.2.3 Vista "a servizi"

Internet è anche un'**infrastruttura di comunicazione** che mette a disposizione delle applicazioni distribuite due tipi di servizio:

- servizio **affidabile** dalla sorgente alla destinazione;
- servizio **"best effort"** (non affidabile), senza connessione.

Possibile domanda orale

"Conoscete altri protocolli umani?" – Esempi: il galateo delle presentazioni, il protocollo di una stretta di mano, le regole di precedenza al semaforo/incrocio (anche i protocolli umani hanno un formato e un ordine dei "messaggi"!).

2.3 1.2 – Ai confini della rete

Conviene immaginare la rete come divisa in due parti concettuali: l'**edge** (i confini, dove stanno host e applicazioni) e il **core** (il nucleo, fatto di router interconnessi – si veda la Sezione 1.3).

2.3.1 Sistemi terminali e architetture applicative

I **sistemi terminali (host)** fanno girare i programmi applicativi (es. Web, e-mail) e si trovano all'estremità di Internet. Esistono due architetture applicative principali:

- **Client/server**: l'host **client** richiede e riceve un servizio da un programma **server** sempre attivo su un altro host (es. browser/server Web, client/server e-mail). I server sono tipicamente ospitati in data center.
- **Peer-to-peer (P2P)**: uso limitato o **inesistente di server dedicati**; ogni host (*peer*) comunica direttamente con altri host, agendo sia da client che da server (es. Skype, BitTorrent).

Possibile domanda orale

"Dove si trovano i server, in un'architettura P2P?" – Non ci sono server dedicati in senso stretto: ogni peer mette a disposizione le proprie risorse (banda, storage) agli altri peer, quindi i "server" sono distribuiti sugli host degli stessi utenti, non centralizzati in un data center.

2.3.2 Reti di accesso e mezzi fisici

La domanda chiave è: **come si collegano sistemi terminali e router esterni?** Tre grandi categorie:

Accesso residenziale.

- **Modem dial-up**: fino a 56 kbit/s; non è possibile navigare e telefonare contemporaneamente sulla stessa linea.
- **DSL** (*Digital Subscriber Line*): installata tipicamente dalla società telefonica; fino a 1-5 Mbit/s in upstream e 10-50 Mbit/s in downstream; linea dedicata (non condivisa con altri utenti).
- **FTTH** (*Fiber-to-the-Home*): fibra ottica fino all'abitazione, tramite ONT (*Optical Network Terminal*) e splitter.

Accesso aziendale: le LAN. Una **LAN** (*Local Area Network*) collega i sistemi terminali di aziende e università all'edge router. Lo standard più diffuso è **Ethernet**, con velocità storiche di 10 Mbit/s, 100 Mbit/s, 1 Gbit/s, 10 Gbit/s. Nella configurazione moderna, i sistemi terminali sono collegati tramite uno **switch** Ethernet (approfondito nel capitolo sul livello di collegamento).

Accesso wireless. Una rete condivisa d'accesso wireless collega i sistemi terminali al router attraverso una **stazione base** (*access point*).

- **LAN wireless** (WiFi, 802.11 a/b/g/n/ac): 11, 54, 100+ Mbit/s.
- **Rete d'accesso wireless geografica**: gestita da un provider di telecomunicazioni; dalle prime reti 3G (UMTS, HSDPA: qualche Mbit/s – 14,4 Mbit/s) a LTE ($\sim$100 Mbit/s), fino a 4G/5G.

Reti domestiche tipiche. Componenti: modem via cavo/DSL, router/firewall/NAT, switch Ethernet, punto d'accesso wireless – tutti spesso integrati in un unico dispositivo.

2.3.3 Mezzi trasmissivi

Definizioni

- **Bit**: unità di informazione base (0/1), viaggia da un sistema terminale a un altro passando per coppie trasmittente-ricevente.
- **Mezzo fisico**: strumento che permette il passaggio di bit tra trasmittente e ricevente.
- **Mezzi guidati**: i segnali si propagano in un mezzo fisico solido (fibra ottica, rame, cavo coassiale).
- **Mezzi a onda libera**: i segnali si propagano nell'atmosfera o nello spazio (radio, satellite).

Doppino intrecciato (twisted pair, TP): due fili di rame avvolti, il cavo standard per Ethernet. Si distingue per il tipo di **schermatura**:

- nessuna (*unshielded*);
- schermatura del singolo doppino (*shielded*);

- lamina o maglia metallica che avvolge l'intero cavo (*foiled*);
- entrambe (*screened*).

La denominazione sintetica è del tipo **X/YTP**, dove **X** indica la schermatura dell'intero cavo (U=unshielded, F=foiled cioè lamina di alluminio, S=maglia metallica intrecciata, SF=entrambe) e **Y** la schermatura di ogni singolo doppino (U=unshielded, F=shielded). Esempi: U/UTP, F/UTP, S/FTP.

Tabella 1: Principali standard Ethernet su doppino intrecciato

Nome	**Standard**	**Mbit/s**	**Coppie**	**Distanza**	**Cavo**
10BASE-T	802.3i-1990	10	2	100 m	Cat.3
100BASE-T1	802.3bw-2015	100	1	15 m	Cat.5e
100BASE-TX	802.3u-1995	100	2	100 m	Cat 5e
1000BASE-T	802.3ab-1999	1000	3	100 m	Cat 5e
1000BASE-T1	802.3bp-2016	1000	1	40 m	Cat 6A
2.5GBASE-T	802.3bz-2016	2500	4	100 m	Cat 5e
5GBASE-T	802.3bz-2016	5000	4	100 m	Cat 6
10GBASE-T	802.3an-2016	10000	4	100 m	Cat 6A
25GBASE-T	802.3bq-2016	25000	4	30 m	Cat 8
40GBASE-T	802.3bq-2016	40000	4	30 m	Cat 8

Cavo coassiale: due conduttori in rame concentrici, bidirezionale.

- **Banda base**: un singolo canale sul cavo (Ethernet legacy).
- **Banda larga**: più canali sullo stesso cavo (tecnologia HFC).

Fibra ottica: mezzo sottile e flessibile che conduce impulsi di luce (ciascun impulso rappresenta un bit).

- Alta frequenza trasmissiva: 10-100 Gbit/s su collegamenti punto-punto.
- Basso tasso di errore, ripetitori distanziati, immune all'interferenza elettromagnetica.
- Mezzo preferito per i collegamenti *long-haul* (dorsali intercontinentali).

Canali radio: trasportano segnali nello spettro elettromagnetico, non richiedono cavi, sono bidirezionali, ma risentono di riflessione, ostruzione da ostacoli e interferenza.

- **Microonde terrestri**: fino a 45 Mbit/s.
- **LAN** (WiFi): 11, 54, 100+ Mbit/s.
- **Wide-area** (cellulari): 3G ∼1 Mbit/s, HSDPA ∼14,4 Mbit/s, LTE 100 Mbit/s.
- **Satellitari**: canali fino a 45 Mbit/s (o sottomultipli); ritardo punto-punto ∼280 ms per i satelliti **geostazionari (GEO)** a 36.000 km; ritardo molto minore per i satelliti **low-earth orbit (LEO)**, a bassa quota.

Esercizio pratico 1.1

Misura la velocità della tua connessione a Internet (ad es. con http://www.speedtest.net/). È coerente con quella dichiarata dal tuo ISP? Se no, prova a ipotizzare perché (condivisione della banda con altri utenti, distanza dalla centrale DSL, congestione WiFi domestica, ecc.).

2.4 1.3 – Il nucleo della rete

Il **nucleo (core)** della rete è una rete magliata di router che interconnette i sistemi terminali. Il **quesito fondamentale** è: **come vengono trasferiti i dati attraverso la rete?** Le due risposte possibili sono la commutazione di circuito e la commutazione di pacchetto.

2.4.1 Commutazione di circuito

Nella **commutazione di circuito**, le risorse di rete lungo il percorso punto-punto (ampiezza di banda, capacità del commutatore) vengono **riservate** per l'intera durata della sessione (esempio tipico: la rete telefonica).

- Le risorse sono **dedicate**: non c'è condivisione.
- Le **prestazioni sono garantite**.
- È necessaria una fase di **impostazione della connessione** prima di iniziare a trasmettere.
- Le risorse restano **inattive** (sprecate) se non utilizzate, perché non c'è condivisione.

La larghezza di banda viene suddivisa in "porzioni" con due tecniche classiche di **multiplexing**:

- **FDM** (*Frequency Division Multiplexing*): la banda disponibile viene divisa in sotto-bande di frequenza, una per utente, usate per tutta la durata della chiamata.
- **TDM** (*Time Division Multiplexing*): il tempo viene diviso in slot ciclici (*frame*); a ogni utente è assegnato uno slot fisso e periodico.

Esercizio 1.2 – Tempo di trasmissione a commutazione di circuito

Quanto tempo occorre per inviare un file di 640.000 bit dall'host A all'host B su una rete a commutazione di circuito, sapendo che:

- tutti i collegamenti hanno un bit rate di 1,536 Mbit/s;
- ciascun collegamento usa TDM con 24 slot/secondo;
- si impiegano 500 ms per stabilire il circuito punto-punto?

Soluzione:

$$\text{bit rate per singolo utente} = \frac{1{,}536 \text{ Mbit/s}}{24} = 64 \text{ kbit/s}$$

$$\text{tempo di trasmissione dati} = \frac{640.000 \text{ bit}}{64.000 \text{ bit/s}} = 10 \text{ s}$$

$$\text{tempo totale} = \underbrace{0{,}5 \text{ s}}_{\text{setup}} + \underbrace{10 \text{ s}}_{\text{trasmissione}} = \mathbf{10{,}5} \text{ s}$$

Nota: 24 slot da 64 kbit/s corrispondono esattamente allo standard storico T1 (24 canali DS0), non a caso $24 \times 64 = 1536$.

2.4.2 Commutazione di pacchetto

Nella **commutazione di pacchetto**, il flusso di dati punto-punto viene suddiviso in **pacchetti**. I pacchetti degli utenti **condividono** le risorse di rete: ciascun pacchetto utilizza **per intero** le risorse fisiche disponibili in quel momento, e le risorse vengono ripartite in modalità **best effort** e *on demand* (a seconda delle necessità) – **non** c'è allocazione dedicata né riserva di risorse.

- **Contesa per le risorse**: la richiesta di risorse può eccedere il quantitativo disponibile → **congestione**: i pacchetti si accodano e attendono l'uso del collegamento.
- **Store-and-forward**: chi inoltra un pacchetto deve **riceverlo per intero** prima di iniziare a trasmetterlo sul collegamento in uscita.
- **Multiplexing statistico**: la sequenza di pacchetti di utenti diversi in coda su un collegamento in uscita **non segue uno schema prefissato** (a differenza del TDM, dove ogni host ha uno slot dedicato).

Esempio 1.3 – Ritardo di trasmissione (store-and-forward)

Dato $L = 12$ kbit (lunghezza del pacchetto) e $R = 1{,}5$ Mbit/s (frequenza di trasmissione del collegamento), il tempo di trasmissione è:

$$\frac{L}{R} = \frac{12.000 \text{ bit}}{1.500.000 \text{ bit/s}} = 8 \text{ ms}$$

Per un percorso di più collegamenti in serie (supponendo ritardo di propagazione nullo), il tempo totale end-to-end è $3L/R$ per 2 router intermedi, ecc. – si veda la Sezione 1.4 per il ritardo di propagazione.

Confronto pacchetto vs. circuito. Con 1 collegamento da 1 Mbit/s e utenti che richiedono 100 kbit/s quando sono "attivi" (attivi il 10% del tempo):

- con **commutazione di circuito**, il collegamento supporta al massimo **10 utenti** contemporaneamente (garantiti);
- con **commutazione di pacchetto**, fino a 10 utenti attivi possono trasmettere tutti insieme; oltre questa soglia i pacchetti si accodano, ma con **35 utenti totali**, la probabilità che più di 10 siano attivi simultaneamente è **inferiore a 0,0004**.

Possibile domanda orale – Come si ottiene il valore 0,0004?

Il numero di utenti attivi X su $N = 35$ utenti indipendenti, ciascuno attivo con probabilità $p = 0{,}1$, segue una **distribuzione binomiale** $X \sim \text{Bin}(35; 0{,}1)$. Si cerca $P(X > 10)$:

$$P(X > 10) = 1 - \sum_{k=0}^{10} \binom{35}{k} p^k (1-p)^{35-k} \approx 0{,}0004$$

Il valore medio è $\mathbb{E}[X] = Np = 3{,}5$, con deviazione standard $\sigma = \sqrt{Np(1-p)} \approx 1{,}77$: la soglia di 10 utenti attivi è a quasi 4 deviazioni standard dalla media, quindi un evento molto raro.

Conclusione: la commutazione di pacchetto è ottima per il traffico **a raffica** ("burst"), è più semplice (non serve scambiare messaggi per riservare risorse), ma in caso di congestione introduce ritardo e possibile perdita di pacchetti – da cui la necessità di protocolli per il trasferimento affidabile e per il controllo della congestione (Capitolo 3).

2.4.3 Struttura di Internet: la rete delle reti

Internet ha una struttura **fondamentalmente gerarchica**:

- **ISP di livello 1** (*Tier 1*): al centro della gerarchia (es. Sprint, AT&T, Deutsche Telekom, Telecom Italia Sparkle); copertura nazionale/internazionale; comunicano tra loro alla pari tramite **peering**.
- **ISP di livello 2** (*Tier 2*): più piccoli (nazionali o distrettuali); si connettono solo ad alcuni ISP di livello 1 (di cui sono **clienti** pagando per la connettività), e possibilmente ad altri ISP di livello 2 (a volte anch'essi in peering diretto).
- **ISP di livello 3 e ISP locali**: reti di "ultimo salto" (*last hop network*), le più vicine ai sistemi terminali; sono clienti degli ISP di livello superiore.

Un pacchetto attraversa quindi **molte reti diverse** lungo il suo percorso, e **il percorso di andata e quello di ritorno tra due host non sono necessariamente gli stessi**. Nella pratica reale la struttura è ancora più complessa, con **IXP** (*Internet eXchange Point*), **PoP** (*Point of Presence*) e **content provider network** (es. Google, Akamai) che si interconnettono direttamente con i Tier 2 per ridurre latenza e costi di transito.

2.5 1.4 – Ritardo, perdita e throughput

I pacchetti si **accodano** nei buffer (memorie) dei router: questo succede quando il tasso di arrivo dei pacchetti su un collegamento eccede la capacità del router/collegamento di smaltirli. Se non ci sono buffer liberi, i pacchetti in arrivo vengono **scartati** (perdita).

2.5.1 Le quattro componenti del ritardo di nodo

Formula del ritardo di nodo

$$d_{\text{node}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

1. $d_{\textbf{proc}}$ – **ritardo di elaborazione** (*processing delay*): controllo errori sui bit, determinazione della porta/canale di uscita. In genere pochi microsecondi o meno.

2. $d_{\textbf{queue}}$ – **ritardo di accodamento** (*queuing delay*): attesa di trasmissione; dipende dal livello di congestione del router.

3. $d_{\textbf{trans}}$ – **ritardo di trasmissione** (*transmission delay*): $d_{\text{trans}} = L/R$, dove L è la lunghezza del pacchetto (bit) e R la frequenza di trasmissione del collegamento (bit/s). Significativo sui collegamenti a bassa velocità.

4. $d_{\textbf{prop}}$ – **ritardo di propagazione** (*propagation delay*): $d_{\text{prop}} = d/s$, dove d è la lunghezza fisica del collegamento e s la velocità di propagazione ($\sim 2 \times 10^8$ m/s nei mezzi guidati). Va da pochi microsecondi a centinaia di millisecondi.

Attenzione: s (velocità di propagazione) e R (frequenza di trasmissione) sono due quantità *molto* diverse concettualmente, anche se entrambe legate alla "velocità" del collegamento.

Esempio 1.4 – L'analogia del casello autostradale

Dieci auto viaggiano in colonna a 100 km/h; il casello impiega 12 secondi per far passare ("servire") un'auto; la distanza tra due caselli è 100 km.
Analogia: auto ∼ bit, colonna di auto ∼ pacchetto, tempo di servizio al casello ∼ ritardo di trasmissione, tempo di percorrenza tra i caselli ∼ ritardo di propagazione.
Domanda: quanto tempo occorre perché tutte le 10 auto si trovino di fronte al secondo casello?

- tempo per il casello per far passare l'intera colonna: $12 \times 10 = 120$ s $= 2$ min (ritardo di "trasmissione");
- tempo di percorrenza tra i due caselli: 100 km/100 km/h $= 1$ h $= 60$ min (ritardo di "propagazione");
- **totale: 62 minuti**.

Variante: ora le auto viaggiano a 1000 km/h, ma il casello impiega 1 minuto per auto. *Domanda*: la prima auto arriva al secondo casello prima che l'ultima abbia lasciato il primo?
Sì! Dopo 7 minuti la prima auto è al secondo casello (1 min di "trasmissione" + 6 min di "propagazione" a 1000 km/h), mentre al primo casello **ci sono ancora 3 auto in coda** (il casello ne ha servite solo 7 in 7 minuti). Questo dimostra un punto concettualmente importante: **il primo bit di un pacchetto può arrivare al router successivo prima che il pacchetto sia stato interamente trasmesso dal router precedente** – è il fenomeno alla base del *pipelining* nella trasmissione a pacchetto.

2.5.2 Ritardo di accodamento e intensità di traffico

Definiamo:

- R = frequenza di trasmissione (bit/s)
- L = lunghezza del pacchetto (bit)
- A = tasso medio di arrivo dei pacchetti

$$\frac{L \cdot A}{R} = \textbf{intensità di traffico}$$

- $\frac{LA}{R} \approx 0$: poco ritardo di accodamento.
- $\frac{LA}{R} \to 1$: il ritardo di accodamento diventa consistente (cresce in modo non lineare, quasi asintotico).
- $\frac{LA}{R} > 1$: più "lavoro" arriva di quanto il collegamento possa smaltire → **ritardo medio tendenzialmente infinito** (la coda cresce senza limite).

2.5.3 traceroute e i ritardi reali di Internet

Il comando `traceroute` è un programma diagnostico che misura il ritardo dalla sorgente a ciascun router lungo il percorso verso una destinazione:

1. invia **tre pacchetti** che raggiungeranno il router i-esimo sul percorso (sfruttando il campo TTL);
2. il router i restituisce i pacchetti al mittente;
3. il mittente calcola l'intervallo tra trasmissione e risposta, ripetendo per $i = 1, 2, 3, \ldots$

Esempio classico: traceroute da gaia.cs.umass.edu a www.eurecom.fr

```
1  cs-gw (128.119.240.254)                    1 ms   1 ms   2 ms
2  border1-rt-fa5-1-0.gw.umass.edu             1 ms   1 ms   2 ms
3  cht-vbns.gw.umass.edu                       6 ms   5 ms   5 ms
4  jn1-at1-0-0-19.wor.vbns.net                16 ms  11 ms  13 ms
5  jn1-so7-0-0-0.wae.vbns.net                 21 ms  18 ms  18 ms
6  abilene-vbns.abilene.ucaid.edu             22 ms  18 ms  22 ms
7  nycm-wash.abilene.ucaid.edu                22 ms  22 ms  22 ms
8  62.40.103.253      <- collegamento transoceanico     104 ms 109 ms 106 ms
9  de2-1.de1.de.geant.net                    109 ms 102 ms 104 ms
...
16 194.214.211.25                            126 ms 128 ms 126 ms
17 * * *              <- nessuna risposta (router non risponde)
18 * * *
19 fantasia.eurecom.fr (193.55.113.142)      132 ms 128 ms 136 ms
```

Si nota bene il **salto di ritardo** tra i router 7 e 8 (da 22 ms a 104 ms): è il collegamento **transoceanico**. Le righe con `* * *` indicano un router che non risponde (risposta persa o filtrata), non necessariamente un guasto.

Esercizio pratico 1.5

Prova tu stesso `traceroute` (disponibile su Linux/macOS come comando, oppure via servizi online tipo https://ping.eu/traceroute) verso un sito a tua scelta. Quanti salti restituisce? Dove individui eventuali salti transoceanici o particolarmente lenti?

2.5.4 Perdita di pacchetti

Una coda (**buffer**) ha **capacità finita**. Quando un pacchetto trova la coda piena, viene **scartato** (va perso). Un pacchetto perso può essere:

- ritrasmesso dal nodo precedente;
- ritrasmesso dal sistema terminale che lo ha generato (tramite un protocollo di trasporto affidabile, es. TCP);
- non ritrasmesso affatto (es. traffico multimediale in tempo reale, dove un dato in ritardo è inutile).

2.5.5 Throughput

Il **throughput** è la frequenza (dati/unità di tempo) alla quale una certa quantità di dati viene trasferita tra mittente e ricevente. Si distingue tra **throughput istantaneo** (in un dato istante) e **throughput medio** (su un periodo più lungo).

Immaginando i dati come un fluido che scorre in un tubo con due tratti di capacità diverse (R_s e R_c):

- se $R_s < R_c$: il throughput medio end-to-end è limitato da R_s (il tratto più lento);
- se $R_s > R_c$: il throughput medio end-to-end è limitato da R_c.

Il collegamento più lento lungo il percorso si chiama **collo di bottiglia** (*bottleneck*): è il collegamento su un percorso punto-punto che vincola il throughput end-to-end.

Nello scenario più realistico, con N collegamenti che condividono equamente un link centrale di capacità R:

$$\text{throughput end-to-end} = \min\left(R_c,\ R_s,\ \frac{R}{10}\right)$$

In pratica, spesso sono proprio R_c o R_s (i collegamenti d'accesso, non il core della rete) a essere il collo di bottiglia.

2.5.6 Esercizio riassuntivo: bandwidth-delay product

Esercizio 1.6 (P.25, Cap. 1 Kurose-Ross)

Due host A e B sono separati da $m = 20.000$ km e connessi da un link a velocità $R = 2$ Mbit/s. La velocità di propagazione è $c = 2{,}5 \times 10^8$ m/s.

1. Qual è il prodotto banda-ritardo (*bandwidth-delay product*)?
2. Volete mandare un messaggio di 800.000 bit da A a B, in un'unica lunga trasmissione continua. Quanti bit ci sono sul collegamento in un istante qualsiasi?
3. La risposta precedente è legata alla risposta 1?
4. Qual è la lunghezza, in metri, di un singolo bit sul collegamento?

Soluzione:
1) Bandwidth-delay product. Il ritardo di propagazione è:

$$d_{\text{prop}} = \frac{m}{c} = \frac{20.000.000 \text{ m}}{2{,}5 \times 10^8 \text{ m/s}} = 0{,}08 \text{ s} = 80 \text{ ms}$$

Il prodotto banda-ritardo è:

$$R \times d_{\text{prop}} = 2 \times 10^6 \text{ bit/s} \times 0{,}08 \text{ s} = \mathbf{160.000} \text{ bit}$$

2) Bit "in volo" sul collegamento. Il prodotto banda-ritardo rappresenta esattamente la **capacità del collegamento in bit**, cioè quanti bit possono trovarsi contemporaneamente "in transito" sul filo in un dato istante, se il mittente trasmette senza interruzioni: quindi **160.000 bit**, indipendentemente dal fatto che il messaggio totale sia più lungo (800.000 bit).
3) Legame con la risposta 1. Sì: sono **esattamente la stessa quantità**. Il bandwidth-delay product *è*, per definizione, il numero massimo di bit che "riempiono" il collegamento in un istante.
4) Lunghezza di un bit sul collegamento. Un bit occupa una porzione del collegamento pari a:

$$\text{lunghezza di un bit} = c \times \frac{1}{R} = 2{,}5 \times 10^8 \text{ m/s} \times \frac{1}{2 \times 10^6 \text{ bit/s}} = \mathbf{125} \text{ m}$$

(equivalentemente: $m/160.000$ bit $= 20.000.000/160.000 = 125$ m – stesso risultato, coerente con il fatto che le risposte 1 e 2 sono la stessa grandezza vista da due prospettive).

2.6 1.5 – Livelli di protocollo e modelli di servizio

2.6.1 Perché la stratificazione?

Le reti sono sistemi **complessi**: molti host, router, mezzi trasmissivi, applicazioni e protocolli diversi. Una struttura **esplicita a strati (layer)** consente di:

- identificare i vari componenti del sistema e le loro inter-relazioni;
- facilitare manutenzione e aggiornamento: modifiche implementative a un livello risultano **trasparenti** al resto del sistema.

Analogia: l'organizzazione di un viaggio aereo

Un viaggio aereo si scompone in una serie di passi successivi, ciascuno gestito da un "livello" diverso della compagnia aerea: **biglietto** (acquisto/reclami), **bagaglio** (check-in/ritiro), **gate** (imbarco/uscita), **pista** (decollo/atterraggio), **rotta aerea** (instradamento, gestito dai centri di controllo del traffico intermedi). Ogni livello realizza un servizio, agendo autonomamente al proprio interno e usando i servizi del livello immediatamente inferiore.

Spunto per l'orale

"Il modello a strati può essere considerato dannoso?" È una domanda aperta e discussa in letteratura: la stratificazione rigida introduce **overhead** (ogni livello aggiunge la propria intestazione) e può impedire ottimizzazioni "cross-layer" (es. un livello applicativo che vorrebbe conoscere le condizioni del livello fisico/radio per adattarsi meglio). È un compromesso tra **modularità** e **efficienza**.

2.6.2 Formalismo: layer, entità, SAP, protocollo

- Ogni sistema è composto da **sottosistemi**; ogni sottosistema è composto da diverse **entità**, ciascuna delle quali implementa le funzionalità di un layer (una "**(N)-entità**" per il layer N).
- Il layer $N+1$ sa **solo** che i layer inferiori offrono il "servizio N": tutto ciò che sta al di sotto è una **black box**.
- Un servizio del layer N è offerto all'entità di layer N+1 tramite un'interfaccia di programmazione chiamata **Service Access Point (SAP)**.
- Lo scambio di informazioni **tra entità dello stesso layer**, su sistemi diversi, è regolato da un **protocollo** (es. il protocollo di layer N regola la comunicazione tra la (N)-entità del sistema A e la (N)-entità del sistema B).

2.6.3 Lo stack Internet e il modello di riferimento OSI

Tabella 2: Confronto tra lo stack Internet (5 livelli) e il modello ISO/OSI (7 livelli)

Stack Internet	Modello OSI
	7. Applicazione
5. Applicazione	6. Presentazione
	5. Sessione
4. Trasporto	4. Trasporto
3. Rete	3. Rete
2. Collegamento (link)	2. Collegamento
1. Fisico	1. Fisico

Lo stack Internet **non ha** i livelli Presentazione e Sessione: le loro funzioni, quando servono, vengono implementate direttamente nelle applicazioni (es. TLS gestisce cifratura e sessione a livello applicativo).

Dettaglio dei 7 livelli OSI (funzioni e PDU):

Layer 7 – Applicazione Fornisce alle applicazioni i mezzi per scambiarsi dati. Esempi: Web (HTTP), trasferimento file (FTP), e-mail (POP3/IMAP/SMTP), file sharing P2P, terminale virtuale remoto (SSH). *Data unit:* ***messaggi***.

Layer 6 – Presentazione Rappresentazione e codifica/decodifica dei dati; conversione tra formato host (*little endian*) e formato di rete (*big endian*); criptatura. Spesso integrato con l'applicazione.

Layer 5 – Sessione Stabilisce e mantiene una sessione di comunicazione tra due entità (una sessione può comprendere più connessioni con qualità differenti, es. una videochiamata). Permette di interrompere, riprendere e terminare lo scambio dati, mascherando eventuali disconnessioni al livello 4. Spesso integrato con l'applicazione.

Layer 4 – Trasporto Risolve i problemi di qualità del servizio del livello 3: segmentazione/ricomposizione dei dati, multiplexing/demultiplexing tra applicazioni. Funzioni principali (non obbligatorie): controllo di flusso, controllo di errore, riordino dei pacchetti. *Data unit:* ***segmenti*** (il termine "datagrammi" è usato più raramente qui, ed è meglio evitarlo per non confonderlo col livello 3).

Layer 3 – Rete Responsabile della consegna delle data unit da un host all'altro. Due modelli di servizio: **connectionless** (reti orientate al datagramma, ogni pacchetto instradato indipendentemente) oppure **connection-oriented** (reti a circuito virtuale, che stabiliscono una rotta fissa mantenuta per tutta la sessione). Funzioni: instradamento e inoltro dei pacchetti, controllo di congestione (non obbligatorio). *Data unit:* ***pacchetti*** *o* ***datagrammi***.

Layer 2 – Collegamento (link) Trasferimento di data unit tra entità di livello 2 e correzione degli errori del livello 1; comunicazione punto-punto ("single-hop"). Funzioni: multiplexing/demultiplexing di protocolli di livello 3 (sublayer LLC), controllo/correzione di errori di trasmissione, **medium access control** (sublayer MAC), affidabilità. *Data unit:* ***frame***.

Layer 1 – Fisico Trasmette bit sul mezzo di comunicazione (elettrico, elettromagnetico, luminoso, sonoro); crea, mantiene e distrugge connessioni fisiche; definisce codifica, livelli di tensione, modulazione. *Data unit:* ***bit*** *o "simboli"* (gruppi di più bit).

2.6.4 Data unit e incapsulamento

SDU, PCI, PDU

In un sistema a M livelli, i dati da trasmettere a livello M costituiscono una **M-SDU** (*Service Data Unit*) – pensala come il testo di una lettera. A questa il livello aggancia le proprie informazioni di controllo, la **M-PCI** (*Protocol Control Information*) – l'equivalente di mittente e destinatario sulla busta. Il risultato è una **M-PDU** (*Protocol Data Unit*), cioè "lettera + busta".
Scendendo di un livello: la N-PDU diventa la $(N-1)$-SDU per il livello sottostante, che le antepone la propria $(N-1)$-PCI, ottenendo la $(N-1)$-PDU. **Ogni livello tratta la PDU del livello superiore come una "busta chiusa"**, di cui non guarda il contenuto. Al ricevitore il processo si inverte: ogni livello rimuove la propria PCI.

Scendendo lungo lo stack, la PDU diventa quindi un "sandwich" di intestazioni:

$$\underbrace{\texttt{1-PCI}}_{\text{Header}} \cdots \underbrace{\texttt{(N-1)-PCI}}_{\text{Header}} \underbrace{\texttt{N-PCI}}_{\text{Header}} \underbrace{\texttt{User data}}_{\text{Payload}} \text{ (Trailer)}$$

Nello stack Internet, questo processo si chiama **incapsulamento**: partendo da un **messaggio** applicativo, il livello di trasporto aggiunge l'header H_t ottenendo un **segmento**; il livello di rete aggiunge H_n ottenendo un **datagramma**; il livello di collegamento aggiunge H_l ottenendo un **frame**. Un router intermedio legge solo fino all'header di rete (H_n) per instradare il pacchetto, mentre uno switch legge solo fino all'header di collegamento (H_l) – nessuno dei due due apre il "sandwich" oltre il proprio livello di competenza.

Le data unit possono inoltre essere:

- **assemblate**: si attende che arrivino più "pagine" per riempire una singola unità, evitando inefficienze;
- **segmentate**: un dato troppo lungo viene spezzato in più unità più piccole (es. un testo lungo diviso in più pagine, ciascuna in una busta separata);
- **ri-assemblate**: processo inverso della segmentazione, al ricevitore.

2.7 1.6 – Sicurezza di rete (cenni – non materia d'esame)

Internet non fu inizialmente progettata pensando alla sicurezza: la visione originaria era quella di "un gruppo di utenti che si fidavano l'uno dell'altro, collegati a una rete trasparente". I progettisti dei protocolli Internet hanno dovuto in seguito "recuperare", introducendo attenzione alla sicurezza a tutti i livelli. Principali categorie di minaccia:

- **Malware** (virus, worm, cavalli di Troia): un **virus** richiede interazione dell'utente per essere eseguito (es. allegato e-mail) ed è auto-replicante; un **worm** si auto-esegue senza richiedere interazione, propagandosi da solo; un **cavallo di Troia** è nascosto dentro software apparentemente utile. Gli host infetti possono essere "arruolati" in una **botnet** per spamming e attacchi DDoS.

- **Denial of Service (DoS/DDoS)**: gli attaccanti saturano le risorse (banda, server) di una vittima con traffico artefatto, spesso **distribuito** da molti host compromessi (*Distributed* DoS), in tre fasi: selezione dell'obiettivo, irruzione negli host, invio coordinato di traffico verso l'obiettivo.
- **Packet sniffing**: su un mezzo broadcast (Ethernet condivisa, WiFi), un'interfaccia di rete può leggere/registrare tutti i pacchetti che la attraversano, incluse eventuali password in chiaro. **Wireshark è un packet sniffer** (lo stesso usato nei laboratori del corso).
- **IP spoofing**: invio di pacchetti con un indirizzo sorgente falsificato.
- **Record-and-playback**: si "sniffano" dati sensibili (es. una password) per poi riutilizzarli in un secondo momento.

2.8 1.7 – Cenni storici (non materia d'esame)

Tabella 3: Tappe principali della storia di Internet

Periodo	Eventi principali
1961–1972	Kleinrock: teoria delle code per la commutazione di pacchetto (1961); Baran: commutazione di pacchetto in reti militari (1964); nasce il progetto ARPAnet (1967), primo nodo operativo (1969); dimostrazione pubblica e primo protocollo NCP (1972, 15 nodi)
1972–1980	ALOHAnet via satellite (1970); Cerf e Kahn definiscono l'architettura di interconnessione delle reti (1974); Ethernet allo Xerox PARC (1976); architetture proprietarie (DECnet, SNA); ARPAnet a 200 nodi (1979)
1980–1990	Rilascio di TCP/IP (1983); SMTP (1982), DNS (1983), FTP (1985); controllo di congestione TCP (1988); nuove reti nazionali (Csnet, BITnet, NSFnet, Minitel); 100.000 host
1990–2000	Dismissione di ARPAnet; fine restrizioni commerciali su NSFnet (1991); nascita del Web (Berners-Lee, HTML/HTTP); Mosaic poi Netscape (1994); commercializzazione del Web
2000–2007	"Killer application": messaggistica istantanea, file sharing P2P; sicurezza di rete; 50 milioni di host; dorsali a velocità dell'ordine di Gbit/s
2008–oggi	$\sim$500 milioni di host (2008) $\rightarrow$ 4,39 miliardi di utenti (2020); voice/video over IP, P2P (BitTorrent, Skype), cloud computing, smartphone e IoT, 5G, edge/fog computing; crescente attenzione a sicurezza e privacy

I principi guida definiti da Cerf e Kahn (**minimalismo/autonomia** – collegare reti diverse senza richiedere modifiche interne, **modello di servizio best-effort**, **router stateless**, **controllo decentralizzato**) sono ancora oggi alla base dell'architettura di Internet.

2.9 Riepilogo del capitolo 1

Al termine di questo capitolo dovresti essere in grado di:

- dare una panoramica di Internet dal punto di vista hardware, dei protocolli e dei servizi;
- distinguere edge e core della rete, con le rispettive reti di accesso e mezzi trasmissivi;
- confrontare commutazione di circuito e di pacchetto, e calcolare tempi di trasmissione in entrambi i casi;
- scomporre il ritardo end-to-end nelle sue quattro componenti e calcolare il bandwidth-delay product;
- spiegare la stratificazione protocollare, il ruolo di SAP e protocolli, e il processo di incapsulamento.

3 Capitolo 2 – Il livello applicazione

3.1 Obiettivi e argomenti del capitolo

- Fornire i concetti base e gli aspetti implementativi dei protocolli delle applicazioni di rete (modelli di servizio del livello di trasporto, paradigma client-server, paradigma peer-to-peer).
- Studiare i protocolli delle applicazioni più diffuse: HTTP (1.0, 1.1, 2.0), FTP, SMTP/POP3/IMAP, DNS.
- Programmare le applicazioni di rete (Socket API, cenni).

Argomenti: 2.1 Principi delle applicazioni di rete; 2.2 Web e HTTP; 2.3 FTP; 2.4 Posta elettronica; 2.5 DNS; 2.6 Applicazioni P2P; 2.7 Cloud computing; 2.8 Programmazione delle socket (cenni).

3.2 2.1 – Principi delle applicazioni di rete

3.2.1 Creare un'applicazione di rete

Creare un'applicazione di rete significa scrivere programmi che **girano su sistemi terminali diversi** e **comunicano attraverso la rete** (es. il software di un server Web comunica con il software di un browser). Punto fondamentale: **non occorre predisporre programmi per i dispositivi del nucleo della rete**, quali router o commutatori Ethernet – lo sviluppo applicativo riguarda solo gli *end system*. Questo è coerente con quanto visto nel Capitolo 1 sulla stratificazione: la complessità resta ai margini della rete.

3.2.2 Architetture delle applicazioni di rete

- **Client-server**: il **server** è un host sempre attivo, con indirizzo permanente (problema aperto: *come scalare?*); il **client** comunica con il server, può disconnettersi temporaneamente, può avere un indirizzo dinamico, e **non comunica direttamente con altri client**.
- **Peer-to-peer (P2P) pura**: non c'è un server sempre attivo; coppie arbitrarie di host (*peer*) comunicano direttamente tra loro; i peer non devono essere sempre attivi e possono cambiare indirizzo IP. Facilmente scalabile, ma difficile da gestire.

- **Ibride** (client-server + P2P): es. **Skype** (VoIP P2P, ma con un server centralizzato per la ricerca degli indirizzi della parte remota; la connessione vera e propria è diretta client-client); **messaggistica istantanea** (la chat è P2P diretta, ma l'individuazione di presenza/posizione è centralizzata: l'utente registra il proprio indirizzo IP sul server quando è online, e lo consulta per conoscere gli indirizzi IP dei propri contatti).
- **Cloud computing**: insieme di tecnologie che permettono di memorizzare, archiviare e/o elaborare dati (con CPU o software) tramite risorse distribuite e virtualizzate in rete. Il backup è automatico e l'operatività si trasferisce online; i dati risiedono in *server farm* tipicamente localizzate nei paesi d'origine del provider. (Da qui la battuta ricorrente: *"There is no cloud, it's just someone else's computer"*.)

3.2.3 Processi in comunicazione

- **Processo**: programma in esecuzione su un host. Due processi sullo stesso host comunicano tramite schemi **interprocesso** (definiti dal SO); processi su host diversi comunicano scambiandosi **messaggi**.
- **Processo client**: dà inizio alla comunicazione. **Processo server**: attende di essere contattato.
- *Nota*: le applicazioni con architettura P2P hanno processi che agiscono **sia** da client sia da server.

Socket

Un processo invia/riceve messaggi verso/dalla sua **socket**: è analoga a una "porta". Un processo che vuole inviare un messaggio lo fa uscire dalla propria socket, presupponendo l'esistenza di un'infrastruttura esterna che lo trasporterà attraverso la rete fino alla socket del processo destinatario. La socket è **controllata dallo sviluppatore dell'applicazione** sul lato "sopra" l'interfaccia, mentre il trasporto (es. buffer TCP) è **controllato dal sistema operativo**. L'API di una socket permette di: (1) scegliere il protocollo di trasporto; (2) determinare alcuni parametri di configurazione.

3.2.4 Indirizzamento

Affinché un processo su un host invii un messaggio a un processo su un altro host, il mittente deve identificare il **processo destinatario**: non basta l'**indirizzo IP** (32 bit) dell'host, perché sullo stesso host possono girare più processi contemporaneamente. Serve quindi la coppia **(indirizzo IP, numero di porta)**.

- Esempi di porte standard: **HTTP server: 80**; **Mail server: 25**.
- Esempio: per contattare il server HTTP su `gaia.cs.umass.edu`: indirizzo IP `128.119.245.12`, porta `80`.

3.2.5 Protocolli a livello applicazione

Un protocollo a livello applicazione definisce: i **tipi di messaggi** scambiati (richiesta/risposta), la **sintassi** dei messaggi (quali campi, come sono descritti), la **semantica** dei campi (il significato delle informazioni), e le **regole** su quando/come un processo invia e risponde ai messaggi.

- **Protocolli di pubblico dominio**: definiti nelle RFC dell'IETF, garantiscono interoperabilità (es. HTTP, SMTP).
- **Protocolli proprietari**: es. Skype.

3.2.6 Quale servizio di trasporto serve a un'applicazione?

Quattro dimensioni: **perdita di dati** (alcune app, es. audio, tollerano perdita; altre, es. trasferimento file, richiedono affidabilità al 100%), **temporizzazione** (telefonia Internet, giochi interattivi richiedono bassa latenza), **throughput** (app "efficaci" hanno un minimo richiesto; app "elastiche" usano quello disponibile), **sicurezza** (cifratura, integrità dei dati).

Tabella 4: Requisiti del servizio di trasporto di alcune applicazioni comuni

Applicazione	**Tolleranza perdita**	**Throughput**	**Sensibilità al tempo**
Trasferimento file	No	Variabile	No
Posta elettronica	No	Variabile	No
Documenti Web	No	Variabile	No
Audio/video in tempo reale	Sì	5 kbit/s – 5 Mbit/s	Sì, centinaia di ms
Audio/video memorizzati	Sì	Come sopra	Sì, pochi secondi
Giochi interattivi	Sì	Fino a pochi kbit/s	Sì, centinaia di ms
Messaggistica istantanea	No	Variabile	Sì e no

TCP vs UDP.

- **TCP**: *orientato alla connessione* (setup tra client e server), *trasporto affidabile*, **controllo di flusso** (il mittente non vuole sovraccaricare il destinatario), **controllo della congestione** ("strozza" il mittente quando la rete è sovraccarica). **Non** offre: temporizzazione, garanzie sull'ampiezza di banda minima, sicurezza.
- **UDP**: trasferimento *inaffidabile* di dati tra processi. **Non** offre: setup della connessione, affidabilità, controllo di flusso, controllo di congestione, temporizzazione, ampiezza di banda minima, sicurezza.

Spunto per l'orale – perché esiste UDP?

Se UDP non offre nessuna garanzia, perché usarlo? Perché **elimina l'overhead** di TCP (niente handshake, niente ritrasmissioni, niente controllo di flusso/congestione): è quindi più **veloce e leggero**, ideale per applicazioni che tollerano qualche perdita ma non tollerano ritardo (streaming, VoIP, giochi online) o per protocolli che gestiscono l'affidabilità da soli a un livello superiore (es. DNS, dove i timeout/retry sono gestiti dall'applicazione stessa).

Tabella 5: Applicazioni Internet e relativi protocolli

Applicazione	**Protocollo applicativo**	**Trasporto**
Posta elettronica	SMTP [RFC 2821]	TCP
Accesso a terminali remoti	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
Trasferimento file	FTP [RFC 959]	TCP
Multimedia in streaming	HTTP (es. YouTube), RTP [RFC 1889]	TCP o UDP
Telefonia Internet	SIP, RTP, proprietario (es. Skype)	Tipicamente UDP

3.3 2.2 – Web e HTTP

3.3.1 Terminologia

- Una **pagina web** è costituita da **oggetti** (un file HTML, un'immagine JPEG, un applet Java, un file audio, ...).
- È formata da un **file base** scritto in **HTML** (*HyperText Markup Language*), che include diversi oggetti referenziati.
- Ogni oggetto è referenziato da un **URL** (*Uniform Resource Locator*), es. `www.someschool.edu/so` dove la prima parte è il **nome dell'host** e la seconda il **nome del percorso**.

Curiosità: l'inventore del World Wide Web è **Tim Berners-Lee** (ACM Turing Award per "*aver inventato il World Wide Web, il primo browser, e i protocolli e algoritmi fondamentali che hanno permesso al web di scalare*").

3.3.2 Panoramica su HTTP

HTTP (*HyperText Transfer Protocol*) è il protocollo a livello applicazione del Web, con modello **client/server**: il **client** (browser) richiede, riceve e visualizza gli oggetti; il **server** li invia in risposta.

- Usa **TCP**: il client inizializza una connessione (socket) verso il server, **porta 80**; il server accetta la connessione; si scambiano messaggi HTTP; la connessione TCP viene chiusa.
- HTTP è un protocollo **"senza stato" (stateless)**: il server non mantiene alcuna informazione sulle richieste passate del client.

Nota – perché evitare lo stato?

I protocolli *con stato* sono complessi: la storia passata deve essere memorizzata, e se client e/o server si bloccano, le rispettive viste dello stato potrebbero risultare disallineate e vanno riconciliate. Progettare HTTP senza stato semplifica enormemente l'implementazione dei server.

3.3.3 Connessioni non persistenti e persistenti

- **Connessioni non persistenti**: almeno un oggetto viene trasmesso su una connessione TCP (nella versione più restrittiva, un oggetto per connessione).
- **Connessioni persistenti**: più oggetti possono essere trasmessi su un'unica connessione TCP tra client e server.

Esempio – connessione non persistente. Utente immette l'URL `www.someSchool.edu/someDepar` (pagina con testo e 10 immagini JPEG):

1. Il client HTTP inizializza una connessione TCP con il server sulla porta 80; il server accetta e avvisa il client.
2. Il client trasmette un **messaggio di richiesta** (con l'URL) nella socket; il server riceve, forma il **messaggio di risposta** con l'oggetto richiesto e lo invia.
3. Il server **chiude** la connessione TCP.
4. Il client riceve la risposta, visualizza l'HTML, e vi trova riferimenti a 10 oggetti JPEG.
5. I passi 1–4 vengono **ripetuti per ciascuno** dei 10 oggetti.

Round-Trip Time (RTT) e tempo di risposta

RTT: tempo di propagazione di andata e ritorno tra due host (es. tempo impiegato da un piccolo pacchetto per andare dal client al server e tornare).
Tempo di risposta per un oggetto (connessione non persistente):

- 1 RTT per inizializzare la connessione TCP;
- 1 RTT perché la richiesta HTTP e i primi byte della risposta tornino al client;
- + tempo di trasmissione del file.

$$\textbf{tempo totale} = 2\,\mathrm{RTT} + \text{tempo di trasmissione}$$

Svantaggi delle connessioni non persistenti: richiedono 2 RTT *per ogni oggetto*; overhead del sistema operativo per ogni nuova connessione TCP; i browser spesso aprono più connessioni TCP parallele per mitigare il problema.

Connessioni persistenti: il server lascia la connessione TCP aperta dopo aver inviato una risposta; i messaggi successivi tra lo stesso client/server viaggiano sulla connessione già aperta; il client invia le richieste non appena incontra un oggetto referenziato → **un solo RTT per tutti gli oggetti** referenziati (anziché uno a oggetto).

3.3.4 Formato dei messaggi HTTP

Messaggio di richiesta. Formato ASCII, leggibile dall'utente:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
```

```
Connection: close
Accept-language: fr
                                        <- riga vuota (CR LF extra: fine messaggio)
```

Struttura generale: una **riga di richiesta** (`metodo URL versione`), seguita da una o più **righe di intestazione** (`campo: valore`), una riga vuota, e infine l'eventuale **corpo dell'entità**.

Metodi.

- **HTTP/1.0**: `GET`, `POST`, `HEAD` (chiede al server di escludere l'oggetto richiesto dalla risposta, utile per debug).
- **HTTP/1.1**: aggiunge `PUT` (invia un file nel corpo dell'entità al percorso specificato nell'URL) e `DELETE` (cancella il file specificato nell'URL).
- **Metodo POST**: l'input di un form arriva al server nel **corpo dell'entità**.
- **Metodo URL** (con GET): l'input arriva nel campo URL stesso, es. `www.somesite.com/animalse`

Messaggio di risposta.

```
HTTP/1.1 200 OK
Connection close
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 ...
Content-Length: 6821
Content-Type: text/html

dati dati dati dati dati ...
```

Struttura: una **riga di stato** (protocollo + codice di stato + espressione), righe di intestazione, riga vuota, dati (es. il file HTML richiesto).

Principali codici di stato.

`200 OK` La richiesta ha avuto successo; l'oggetto richiesto è nella risposta.

`301 Moved Permanently` L'oggetto è stato trasferito; la nuova posizione è nell'intestazione `Location:`.

`400 Bad Request` Il messaggio di richiesta non è stato compreso dal server.

`404 Not Found` Il documento richiesto non si trova su questo server.

`505 HTTP Version Not Supported` Il server non supporta la versione HTTP richiesta.

Esercizio pratico 2.1 – Dialogare a mano con un server HTTP

1. Collegati via Telnet (o `netcat`) alla porta 80 di un server web: `telnet www.unitn.it 80` oppure `rlwrap nc -C www.unitn.it 80` (o via PuTTY su Windows). Si apre una connessione TCP alla porta 80 di `www.unitn.it`: tutto ciò che digiti viene trasmesso a quella porta.
2. Digita una richiesta GET minima ma completa (premi Invio due volte):

   ```
   GET / HTTP/1.1
   Host: www.unitn.it
   ```

3. Osserva il messaggio di risposta restituito dal server.
4. Prova a inviare quanto segue verso `gaia.cs.umass.edu`: cosa cambia rispetto a prima?

   ```
   GET /kurose_ross/interactive/index.php HTTP/1.1
   Host: gaia.cs.umass.edu
   ```

3.3.5 Cookie

Molti siti importanti usano i cookie per mantenere uno pseudo-"stato" sopra un protocollo stateless. Quattro componenti:

1. una riga di intestazione nel messaggio di *risposta* HTTP (`Set-cookie: ...`);
2. una riga di intestazione nel messaggio di *richiesta* HTTP (`cookie: ...`);
3. un file cookie mantenuto sul terminale dell'utente, gestito dal browser;
4. un database sul sito.

Esempio: alla prima visita di Susan a un sito di e-commerce, il sito crea un ID univoco (es. `1678`) e una *entry* nel proprio database; il server risponde con `Set-cookie: 1678`; ad ogni visita successiva il browser di Susan invia `cookie: 1678`, permettendo al sito di riconoscerla.

I cookie possono contenere: **autorizzazione**, dati per acquisti (carrello), raccomandazioni personalizzate, stato della sessione (es. email). *Nota su privacy*: i cookie permettono ai siti di imparare molto sugli utenti (abitudini di navigazione), e l'utente può fornire al sito anche nome e indirizzo email.

3.3.6 Cache Web (server proxy) e GET condizionale

Obiettivo della **cache**: soddisfare la richiesta del client **senza coinvolgere il server d'origine**. L'utente configura il browser per accedere al Web tramite la cache: se l'oggetto è in cache, viene fornito direttamente; altrimenti la cache lo richiede al server d'origine e lo inoltra al client (**la cache opera sia da client che da server**). Tipicamente installata da un ISP. **Perché il caching**: riduce i tempi di risposta, riduce il traffico sul collegamento d'accesso a Internet.

Esercizio 2.2 – Dimensionare l'accesso a Internet con e senza cache

Ipotesi: dimensione media oggetto = 100.000 bit; frequenza media di richieste = 15/s; ritardo del router istituzionale verso un server d'origine (andata e ritorno) = 2 s; LAN istituzionale a 10 Mbit/s; collegamento d'accesso a 1,5 Mbit/s.
Situazione iniziale.

$$\text{intensità di traffico sulla LAN} = \frac{15 \cdot 0{,}1 \text{ Mbit}}{10 \text{ Mbit/s}} = 0{,}15 \quad (15\%)$$

$$\text{intensità di traffico sul link d'accesso} = \frac{15 \cdot 0{,}1 \text{ Mbit}}{1{,}5 \text{ Mbit/s}} = \mathbf{1} \quad (100\%)$$

Con intensità di traffico = 1 sul collegamento d'accesso, il ritardo di accodamento tende all'infinito (si veda la Sezione 1.4): ritardo totale $\approx$ 2 s+**minuti**+qualche ms. **Inaccettabile.**
Soluzione A – aumentare la banda d'accesso a 10 Mbit/s. Intensità sul link d'accesso scende a 0,15: ritardo totale $\approx$ 2 s + qualche ms. Funziona, ma cambiare infrastruttura è spesso **costoso**.
Soluzione B – installare una cache istituzionale, con *hit rate* del 40%:

$$\text{intensità sul link d'accesso} = \frac{0{,}6 \cdot 15 \cdot 0{,}1 \text{ Mbit}}{1{,}5 \text{ Mbit/s}} = 0{,}6 < 1 \quad \text{(molto meglio!)}$$

$$\text{ritardo se hit (40\% dei casi)} \approx 0{,}01 \text{ s}$$

$$\text{ritardo se miss (60\% dei casi)} \approx 2 \text{ s} + \text{pochi ms} \approx 2{,}01 \text{ s}$$

$$\text{ritardo medio} = 0{,}6 \times 2{,}01 \text{ s} + 0{,}4 \times 0{,}01 \text{ s} = \mathbf{1{,}21} \text{ s}$$

Una soluzione economica (cache) può battere nettamente una soluzione costosa (upgrade di banda) se il tasso di successo della cache è ragionevole.

GET condizionale: obiettivo, non inviare un oggetto se la cache ne ha già una copia aggiornata.

- La cache specifica la data della propria copia nella richiesta: `If-modified-since: <data>`.
- Se la copia in cache è ancora valida, il server risponde `HTTP/1.0 304 Not Modified` **senza corpo**; altrimenti risponde `HTTP/1.0 200 OK` con i dati aggiornati.

3.3.7 Da HTTP/1.0/1.1 a HTTP/2

HTTP/2 mantiene metodi, codici di stato e semantica di HTTP/1.1, ma si concentra sulle **prestazioni** (latenza percepita, uso delle risorse), con l'obiettivo di usare un'**unica connessione TCP** per sito.

- Si basa su **SPDY**, introducendo: **multiplexing di flussi** (numero illimitato di *stream* su un'unica connessione TCP), **priorità delle richieste** (il client assegna un peso 1–256 e dipendenze tra stream, costruendo un "albero di priorità"), **compressione dell'header** HTTP.
- **Framing binario**: nuovo (sub)livello che gestisce l'incapsulamento e il trasferimento dei messaggi. Ogni **messaggio** HTTP logico (richiesta o risposta) è composto

da uno o più **frame** (unità minima di comunicazione, es. `HEADERS frame`, `DATA frame`), ciascuno codificato in **formato binario** e taggato con l'identificativo del proprio *stream*. **Un client HTTP/1.x non comprende un server HTTP/2**, e viceversa: serve un accordo esplicito tra le due parti.

- **Multiplexing**: in HTTP/1.x, richieste parallele richiedevano **più connessioni TCP**; in HTTP/2 i frame di stream diversi vengono **interposti** sulla stessa connessione e riassemblati a destinazione tramite l'identificatore di stream – tutte le connessioni HTTP/2 sono **persistenti**, e ne basta **una sola per origine**.
- **Server push**: il server può inviare al client risorse aggiuntive (es. `script.js`, `style.css`) **senza che il client le richieda esplicitamente**, anticipando ciò che servirà per renderizzare la pagina, eliminando un giro di richiesta/risposta.

3.3.8 HTTPS e TLS

TLS (*Transport Layer Security*) è un protocollo crittografico che garantisce **autenticazione**, **integrità dei dati** e **confidenzialità**, in tre fasi: negoziazione dell'algoritmo tra le parti, scambio delle chiavi e autenticazione, cifratura simmetrica e autenticazione dei messaggi.

HTTPS estende HTTP inviando le richieste/risposte attraverso una comunicazione cifrata con TLS, sulla **porta 443**. Il browser riconosce automaticamente lo schema `https://` e gestisce cifratura/decifratura in modo trasparente. **Nota**: TLS cifra solo il payload HTTP, **non** gli header dei protocolli inferiori – porte TCP e indirizzi IP restano in chiaro. Oggi oltre il 50–70% dei siti web usa HTTPS.

Limite: la "S" aggiunge **ritardo di connessione** (2 RTT per l'handshake TLS, oltre a 1 RTT per la connessione TCP sottostante) → protocolli più recenti come **HTTP/3** (basato su **QUIC**, si veda il Capitolo 3) cercano di ridurre questo overhead.

3.4 2.3 – FTP

FTP (*File Transfer Protocol*, RFC 959) trasferisce file da/verso un host remoto, con modello client/server: **client** (chi avvia il trasferimento), **server** (host remoto, in ascolto sulla **porta 21**).

Due connessioni TCP separate

- **Connessione di controllo** (porta **21**): il client si autentica e invia comandi (es. cambio directory); resta aperta per tutta la sessione – si dice "fuori banda" (*out-of-band*).
- **Connessione dati** (porta **20**): il server ne apre una **nuova per ogni file** trasferito, e la chiude al termine del trasferimento.

Il server FTP mantiene quindi uno **stato**: directory corrente e autenticazione dell'utente (a differenza di HTTP!).

Comandi comuni: `USER` *`username`*, `PASS` *`password`*, `LIST` (elenca i file), `RETR` *`filename`* (scarica/*get*), `STOR` *`filename`* (carica/*put*). **Codici di ritorno**: `331 Username OK, password required`; `125 data connection already open; transfer starting`; `425 Can't open data connection`; `452 Error writing file`.

3.5 2.4 – Posta elettronica

Tre componenti principali: **agente utente** ("mail reader", es. Outlook, Thunderbird – composizione, editing e lettura dei messaggi), **server di posta** (contiene la *mailbox* con i messaggi in arrivo e la coda di quelli da trasmettere), **SMTP** (*Simple Mail Transfer Protocol*).

3.5.1 SMTP [RFC 2821]

Usa **TCP** per un trasferimento affidabile, sulla **porta 25**, con trasferimento **diretto** dal server SMTP trasmittente al server SMTP ricevente, in tre fasi: **handshaking** (saluto), **trasferimento** dei messaggi, **chiusura**. Interazione comando/risposta in **ASCII a 7 bit**: comandi in testo ASCII, risposte con codice di stato ed espressione.

Scenario: Alice invia un messaggio a Bob.

1. Alice compone il messaggio con il suo agente utente, indirizzato a `bob@someschool.edu`.
2. L'agente utente di Alice lo invia al server di posta di Alice; il messaggio finisce nella coda.
3. Il lato client di SMTP (sul server di Alice) apre una connessione TCP col server di posta di Bob.
4. Il client SMTP trasmette il messaggio sulla connessione TCP.
5. Il server di posta di Bob deposita il messaggio nella mailbox di Bob.
6. Bob invoca il suo agente utente per leggere il messaggio.

Esempio classico di interazione SMTP

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Note finali su SMTP: usa connessioni **persistenti**; richiede che intestazione e corpo siano in **ASCII a 7 bit**; usa `<CR><LF>.<CR><LF>` per segnalare la fine del messaggio.

Confronto SMTP vs HTTP

Entrambi hanno interazione comando/risposta in ASCII con codici di stato. Differenze: **HTTP è "pull"** (il client tira i dati a sé), **SMTP è "push"** (il mittente spinge i dati verso il ricevente); in HTTP ogni oggetto è incapsulato nel proprio messaggio di risposta, in SMTP più oggetti (allegati) possono viaggiare in un **unico** messaggio.

Esercizio pratico 2.3 – Dialogare a mano con un server SMTP

Apri una connessione al server (`telnet servername 25` o `rlwrap nc -C servername 25`, oppure PuTTY su Windows): riceverai `220` come primo codice. Digita in sequenza `HELO`, `MAIL FROM`, `RCPT TO`, `DATA` (poi le intestazioni `From:`/`To:`/`Subject:`, una riga vuota, il corpo, e un `.` da solo su una riga), infine `QUIT`. Così puoi inviare un'email anche senza un client di posta. Per test locali si può usare un server SMTP finto via Docker: `sudo docker run -name fake-smtp -p 1080:1080 -p 1025:1025 reachfive/fake-smtp-server`.

3.5.2 Formato dei messaggi e MIME

RFC 822: standard per il formato dei messaggi di testo – righe di intestazione (`From:`, `To:`, `Subject:` – **diverse dai comandi SMTP!**), una riga vuota, poi il corpo (solo caratteri ASCII).

Per contenuti multimediali si usa **MIME** (*Multipurpose Internet Mail Extensions*, RFC 2045/2056), che aggiunge intestazioni come:

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
```

La **codifica base-64** converte i dati binari in soli caratteri ASCII stampabili: ogni gruppo di **6 bit** viene codificato usando **8 bit** (da cui l'inefficienza "per definizione", ∼33% di dati in più) – eventuale *padding* con `=`.

Esercizio pratico 2.4

Prendi il corpo di un'email con un allegato codificato in base64, copialo in un file di testo, e prova a riconvertirlo in un file immagine (es. `redball.png`) usando un tool di decodifica base64.

3.5.3 Protocolli di accesso alla posta: POP3 e IMAP

SMTP consegna e memorizza la posta sul server del **destinatario**, ma non basta: serve un **protocollo di accesso** per prelevarla.

POP3 (RFC 1939) – fase di autorizzazione, poi transazione.

```
S: +OK POP3 server ready
C: user rob
S: +OK
C: pass hungry
S: +OK user successfully logged on
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 2 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

Comandi: `user`/`pass` (autorizzazione), `list` (numeri messaggi), `retr` (scarica), `dele` (cancella), `quit`.

L'esempio sopra usa la modalità **"scarica e cancella"**: se Bob cambia client, **non può rileggere** le email già scaricate. Esiste anche la modalità **"scarica e mantieni"** (copia i messaggi su più client). **POP3 è un protocollo senza stato tra sessioni successive**.

IMAP (RFC 1730) – più evoluto. Mantiene tutti i messaggi **in un unico posto** (il server); consente di organizzarli in **cartelle**; conserva lo **stato dell'utente tra sessioni** (nomi delle cartelle, associazione tra ID dei messaggi e cartelle). Un terzo approccio è l'accesso via **HTTP** (webmail: Gmail, Hotmail, Yahoo! Mail).

STARTTLS. Evoluzione che permette di cifrare la connessione anche sulle **porte originali** (110 POP3, 143 IMAP, 25 SMTP): il client chiede al server di instaurare una connessione cifrata, la sessione parte "in chiaro" e diventa cifrata **prima** che passi qualunque informazione sensibile. Usato spesso tra MTA (*Mail Transfer Agent*) durante il trasporto dell'email da un provider all'altro. In alternativa si usano **porte dedicate** già cifrate: SMTP su **465**, POP3 su **995**, IMAP su **993** (tutte con SSL/TLS).

3.6 2.5 – DNS (Domain Name System)

Le persone usano **nomi** (es. `www.yahoo.com`), mentre host e router usano **indirizzi IP** a 32 bit (es. `121.7.106.83`, 4 campi da 0 a 255). Serve un modo per **tradurre** i nomi in indirizzi: il **DNS**.

Cos'è il DNS

- Un **database distribuito**, implementato come gerarchia di server DNS.
- Un **protocollo a livello applicazione** che consente a host, router e server DNS di comunicare per *risolvere* nomi/indirizzi.

Nota concettuale: una funzione così critica per Internet è implementata "semplicemente" come protocollo applicativo – ancora un esempio del principio secondo cui **la complessità sta ai margini della rete**.

Altri servizi DNS: **host aliasing** (un host può avere più nomi, es. `relay1.west.enterprise.com` → `www.enterprise.com`), **mail server aliasing**, **distribuzione del carico** (più indirizzi IP di server replicati raggruppati sotto lo stesso nome canonico).

Perché non centralizzare il DNS? Un unico server centralizzato avrebbe: **singolo punto di guasto**, **volume di traffico** eccessivo, problemi di **distanza** (database lontano) e di **manutenzione** → **non scalabile**.

3.6.1 Gerarchia dei server DNS

- **Server DNS radice** (*root*): contattato da un server locale che non sa risolvere un nome; contatta un server autorevole (o TLD) se non conosce la mappatura, ottiene la risposta e la restituisce. Esistono 13 server radice "logici" nel mondo, ciascuno replicato in molte sedi fisiche tramite *anycast*.
- **Server TLD** (*top-level domain*): gestiscono i domini `com`, `org`, `net`, `edu`, e i domini nazionali (`it`, `uk`, `fr`, ...). Es. Network Solutions gestisce i TLD `com`, Educause gestisce `edu`.
- **Server di competenza** (*authoritative*): ogni organizzazione con host pubblicamente accessibili deve fornire i record DNS pubblici che li mappano; può gestirli da sé o tramite il proprio service provider.
- **Server DNS locale**: non fa parte in senso stretto della gerarchia; ogni ISP (università, azienda, provider residenziale) ne ha uno ("*default name server*"), che riceve le query degli host e le **inoltra** nella gerarchia agendo da proxy.

Query iterativa vs ricorsiva. Esempio: `cis.poly.edu` vuole l'indirizzo IP di `gaia.cs.umass.edu`.

- **Iterativa**: ogni server contattato, se non conosce la risposta, indica **il nome del prossimo server** da contattare ("non conosco questo nome, ma prova a chiedere qui"); il server locale fa quindi query successive verso radice → TLD → competenza (8 messaggi in totale nell'esempio classico).
- **Ricorsiva**: il server locale "scarica" il compito della traduzione sul server contattato, che si occupa lui stesso di interrogare il resto della gerarchia e restituisce la risposta finale.

Spunto per l'orale – iterativa o ricorsiva, quale conviene?

La modalità **ricorsiva** sposta il carico di lavoro (e di stato) sui server intermedi, che devono gestire l'intera catena di risoluzione per conto di altri: per questo, in pratica, i server **radice e TLD** preferiscono rispondere in modalità **iterativa** (per non sovraccaricarsi), mentre il server **locale** tipicamente serve i propri host in modalità **ricorsiva** (l'host fa una sola richiesta e riceve già la risposta finale).

3.6.2 Caching DNS

Un server DNS che apprende una mappatura la memorizza in **cache**, da cui viene rimossa dopo un certo **TTL**. Tipicamente un server locale mette in cache gli indirizzi IP dei server TLD, per cui i server radice vengono contattati raramente. I meccanismi di aggiornamento/notifica sono definiti da IETF (RFC 2136).

Spunto per l'orale – problemi del caching DNS

Un TTL troppo **lungo** rischia di restituire indirizzi **non più validi** se il record cambia (es. un sito che migra server); un TTL troppo **corto** vanifica i benefici del caching, aumentando il traffico verso i server autorevoli. È un classico compromesso **staleness vs carico**, identico nello spirito a quello del caching Web (GET condizionale, Sezione 2.2.6).

E se i server radice si guastassero? **Molto improbabile**: infrastruttura estremamente ridondata, e gli indirizzi IP dei TLD sono comunque cached quasi ovunque (l'attacco DDoS di ottobre 2002 contro i root server, per quanto massiccio, fu del tutto inefficace). Attacchi più **mirati** sono più pericolosi: un attacco al DNS **locale** impedisce l'accesso al DNS per quell'ISP; un attacco al DNS **autoritativo** di un dominio ne impedisce l'accesso globale.

3.6.3 Resource Record (RR) e messaggi DNS

Il DNS memorizza **Resource Record**, nel formato (`name`, `value`, `type`, `ttl`):

Tabella 6: Principali tipi di Resource Record DNS

Type	name	value	Significato
A	nome dell'host	indirizzo IP	associazione host → IP
NS	dominio (es. `foo.com`)	nome del server di competenza	delega di un dominio
CNAME	nome alias	nome canonico ("vero")	host aliasing
MX	dominio	nome del server di posta	mail server aliasing

Esistono molti altri tipi (AAAA per IPv6, TXT, SOA, PTR, SRV, DNSKEY/DS per DNSSEC, ...), ma i quattro sopra sono quelli fondamentali del corso. *Esempio CNAME*: `www.ibm.com` è in realtà `servereast.backup2.ibm.com` (il primo è l'alias, il secondo il nome canonico).

I **messaggi DNS** (sia query che risposta) condividono lo stesso formato: un'intestazione di 12 byte con **identificazione** (16 bit, la risposta riusa lo stesso numero della

domanda) e **flag** (domanda/risposta, richiesta di ricorsione, ricorsione disponibile, risposta di competenza), seguita da quattro sezioni a lunghezza variabile: **domande**, **risposte** (RR), **competenza** (RR dei server autorevoli), **informazioni aggiuntive**.

Esempio pratico 2.5 – Registrare un dominio e risolverlo

Supponiamo di avviare la società "Network Utopia".

1. Registriamo `networkutopia.com` presso un **registrar** (es. Network Solutions), che verifica l'unicità del nome e lo inserisce nel database DNS (a pagamento).
2. Forniamo al registrar nomi e IP dei nostri server di competenza (primario e secondario); il registrar inserisce due RR nel server TLD di `com`:

   ```
   (networkutopia.com, dns1.networkutopia.com, NS)
   (dns1.networkutopia.com, 212.212.212.1, A)
   ```

3. Sul nostro server di competenza inseriamo un record **A** per `www.networkutopia.com` e un record **MX** per `networkutopia.com`.

Come un utente in Australia raggiunge `www.networkutopia.com`:

1. Query al proprio server DNS locale.
2. Il DNS locale contatta il server TLD di `com` (eventualmente passando dal root, se il TLD non è in cache).
3. Il server TLD trova il record NS inserito dal registrar e lo restituisce al DNS locale.
4. Il DNS locale interroga `212.212.212.1` chiedendo il record **A** per `www.networkutopia.com`.
5. Il DNS locale risponde all'utente, diciamo con `212.212.71.4`.
6. L'utente apre finalmente una connessione HTTP verso `212.212.71.4` e scarica la pagina.

3.7 2.6 – Applicazioni P2P: condivisione di file

Come visto, in un'architettura **P2P pura** non c'è un server sempre attivo: coppie arbitrarie di peer comunicano direttamente, entrano ed escono liberamente, cambiano indirizzo IP. Tre temi chiave: **distribuzione di file**, **ricerca di informazioni**, e il caso di studio **Skype**.

3.7.1 Client-server vs P2P: analisi delle prestazioni

Domanda: quanto tempo serve per distribuire un file di dimensione F da un server a N peer? Notazione: u_s = tasso di upload del server; u_i = tasso di upload del peer i; d_i = tasso di download del peer i.

Modello client-server

Il server deve inviare N copie in sequenza (tempo NF/u_s); il client più lento impiega $F/\min_i(d_i)$ per scaricare:

$$d_{cs} = \max\left\{\frac{NF}{u_s},\ \frac{F}{\min_i(d_i)}\right\}$$

Il termine NF/u_s **aumenta linearmente con** N: la capacità del server è **fissa**, quindi più peer vuol dire tempi via via più lunghi, **senza limite**.

Modello P2P

Il server deve solo inviare **una** copia iniziale (tempo F/u_s); il peer più lento a scaricare impiega $F/\min_i(d_i)$; ma soprattutto, i **peer si scambiano pezzi tra loro**, quindi il tasso di upload aggregato è $u_s + \sum_i u_i$ e servono NF bit in totale:

$$d_{P2P} = \max\left\{\frac{F}{u_s},\ \frac{F}{\min_i(d_i)},\ \frac{NF}{u_s + \sum_i u_i}\right\}$$

Perché il P2P scala meglio – l'intuizione chiave

Nel termine $NF/(u_s+\sum_i u_i)$, **sia il numeratore che il denominatore crescono con** N: ogni nuovo peer porta lavoro da fare (F bit in più da distribuire) ma **anche** nuova capacità di upload da offrire agli altri. Nel modello client-server, invece, solo il numeratore cresce: la capacità del server resta fissa. Per questo, nel grafico del tempo di distribuzione minimo in funzione di N, la curva **client-server cresce linearmente e senza limite**, mentre la curva **P2P cresce molto più lentamente** e tende ad appiattirsi: è la ragione fondamentale per cui BitTorrent & co. reggono carichi che manderebbero in crisi un'architettura client-server con le stesse risorse del server d'origine.

3.7.2 BitTorrent

- Il file è diviso in **chunk** (tipicamente 256 kByte). Un **torrent** è il gruppo di peer che si scambia le parti di un dato file; il **tracker** tiene traccia di quali peer partecipano al torrent.
- Un peer che entra nel torrent non possiede nessun chunk, ma li accumula nel tempo; si registra presso il tracker per ottenere la lista dei peer, e si collega a un sottoinsieme di **vicini** (*neighbors*).
- Mentre scarica, un peer **carica** contemporaneamente le parti già ottenute verso altri peer.
- Un peer può lasciare il torrent appena completato il file (**egoisticamente**, si dice *leech*) oppure restare connesso per continuare a fornire dati (**altruisticamente**, *seeder*).

- **Selezione dei chunk**: Alice chiede periodicamente ai vicini quali chunk possiedono, e richiede prioritariamente le parti **più rare** nel proprio vicinato (*rarest first*) – questo massimizza la disponibilità complessiva dei chunk nella rete.

"Tit for tat" – occhio per occhio

Alice invia le proprie parti ai **quattro** vicini che al momento le stanno inviando dati alla frequenza più alta (peer "**unchoked**"); questi 4 favoriti vengono rivalutati ogni 10 secondi. Ogni 30 secondi, Alice sceglie inoltre **a caso** un peer aggiuntivo a cui inviare dati ("**optimistically unchoked**"): se il nuovo arrivato ricambia con una buona frequenza, può entrare stabilmente nei top 4. Tutti gli altri vicini restano "**choked**" (soffocati): non ricevono nulla. Meccanismo: (1) Alice sceglie casualmente Bob; (2) Alice diventa uno dei 4 fornitori preferiti di Bob, che ricambia; (3) Bob diventa uno dei 4 fornitori preferiti di Alice. **Chi carica di più, tendenzialmente scarica più in fretta**: è un incentivo alla cooperazione.

3.7.3 Ricerca di informazioni in un sistema P2P

Serve un **indice** che faccia corrispondere le informazioni alla loro posizione tra gli host (spesso una *Distributed Hash Table*, DHT).

- **Directory centralizzata** (progetto originale **Napster**): quando un peer si collega, informa un server centrale di indirizzo IP e contenuto condiviso; Alice cerca un brano, il server le dice che Bob lo possiede, e Alice scarica direttamente da Bob. **Problemi**: singolo punto di guasto, collo di bottiglia, e – storicamente – violazione del diritto d'autore (il trasferimento è distribuito, ma la **localizzazione** è fortemente centralizzata).

- **Query flooding** (usato da **Gnutella**): completamente distribuito, nessun server centrale. I peer attivi e le connessioni TCP tra loro formano una **rete di copertura** (*overlay network*, un grafo virtuale non legato alla topologia fisica); un peer è tipicamente connesso a meno di 10 vicini. Una richiesta viene inoltrata di vicino in vicino; un messaggio di successo torna indietro lungo il percorso inverso, e il trasferimento vero e proprio avviene via HTTP diretto tra i due peer. Per scalabilità, il flooding ha un **raggio limitato** (TTL).

- **Copertura gerarchica**: combina i pregi di indice centralizzato e query flooding. Alcuni peer diventano **leader di gruppo**; gli altri si connettono al proprio leader; i leader si connettono tra loro; ogni leader tiene traccia del contenuto dei propri "figli".

3.7.4 Caso di studio: Skype

Sistema **ibrido**: intrinsecamente P2P (le coppie di utenti comunicano direttamente), con protocollo **proprietario** (noto solo tramite *reverse engineering*), copertura gerarchica basata su **supernodi** (SN), e un indice che fa corrispondere nomi utente a indirizzi IP passando per un login server centrale.

3.8 2.7 – Cloud computing

Il cloud computing prevede uno o più **server reali**, organizzati in un'architettura ad alta affidabilità, fisicamente collocati nel data center del fornitore. Il fornitore espone interfacce per elencare/gestire i servizi; un **cliente amministratore** le usa per selezionare (es. server virtuale completo o solo storage) e amministrare il servizio (configurazione, attivazione, disattivazione); un **cliente finale** utilizza il servizio così configurato. Le caratteristiche fisiche dell'implementazione (quale server reale, dove si trova) sono **irrilevanti** per l'utente finale.

Criticità: sicurezza e privacy, problemi internazionali di natura economica/politica (localizzazione dei dati), continuità del servizio, difficoltà di migrazione dei dati.

3.8.1 Data center

Architettura tipica a tre livelli: **rete di accesso** (server di calcolo) → **rete di aggregazione** → **rete core**, con switch L2/L3 e collegamenti tipicamente a 1 o 10 Gigabit Ethernet.

I data center di Google (dati 2016, per ordine di grandezza): 14 *mega data-center* nel mondo (con ∼100.000 server ciascuno, per contenuti dinamici/personalizzati come ricerche e Gmail); ∼50 cluster "*bring home*" negli IXP (100–500 server, contenuti statici incluso YouTube); centinaia di cluster "*enter-deep*" nelle reti di accesso degli ISP (un rack, poche decine di server, contenuti statici e *TCP splitting*).

3.8.2 Content Delivery Network (CDN)

Una **CDN** costruisce una rete *overlay* per la distribuzione di contenuti (social media, video, ecc.), con l'obiettivo di memorizzare i dati il più vicino possibile ai consumatori, per: ottimizzare le prestazioni, ridurre la latenza, evitare colli di bottiglia. Due filosofie:

- **Enter deep** (es. **Akamai**): molti server, installati nelle reti degli ISP in tutto il mondo – migliora ritardo/throughput percepiti dall'utente.
- **Bring home** (es. **Limelight**): meno server, ma installati negli **IXP**, per servire più ISP contemporaneamente con la stessa infrastruttura.

Esercizio pratico 2.6 – Trovare il nodo CDN che ti serve

Molti grandi siti hanno una propria CDN (es. Facebook: `fbcdn.net`, visibile esaminando l'HTML della pagina). Per scoprire quale nodo geografico ti serve i contenuti: esegui `ping www.fbcdn.net` per ottenerne l'indirizzo IP, poi localizza geograficamente quell'indirizzo (es. cercando "where is IP xx.xx.xx.xx").

3.9 2.8 – Programmazione delle socket (cenni)

Le slide del corso specificano esplicitamente che **l'esempio di implementazione in linguaggio C non fa parte dei contenuti d'esame**. Qui sotto ne riporto solo i **concetti generali** e le funzioni dell'API, senza il codice completo riga per riga.

Obiettivo: costruire un'applicazione client/server che comunica tramite socket. La **Socket API** fu introdotta in BSD4.1 UNIX nel 1981.

- **Socket TCP** (*stream socket*, `SOCK_STREAM`): trasferimento **affidabile** di un flusso di byte. Il client crea una socket TCP specificando indirizzo IP e porta del server: questo, di per sé, fa sì che il client TCP stabilisca già la connessione con il server. Quando un server viene contattato, **crea una nuova socket dedicata** a quel client (distinta dalla socket "in ascolto"), permettendogli di gestire più client contemporaneamente.
- **Socket UDP** (*datagram socket*, `SOCK_DGRAM`): non c'è handshake né connessione; il mittente allega esplicitamente indirizzo IP e porta di destinazione a **ogni** pacchetto; il server estrae mittente/porta dal pacchetto ricevuto; trasferimento di **gruppi di byte ("datagrammi") inaffidabile**, che possono arrivare fuori ordine o perdersi.

Sequenza tipica lato server (TCP): `socket()` (crea), `bind()` (associa a una porta), `listen()` (si mette in ascolto, con una coda di connessioni in attesa lunga `backlog`), `accept()` (accetta una connessione in coda, restituendo un nuovo file descriptor dedicato al client). **Lato client**: `socket()`, poi `connect()` verso IP/porta del server. Su entrambi i lati, una volta stabilita la connessione: `send()`/`recv()` per scambiare dati, infine `close()`.

Indirizzamento in C: la struct `sockaddr_in` rappresenta un indirizzo di socket (`sin_family` sempre `AF_INET`, `sin_port`, `sin_addr`).

3.10 Riepilogo del capitolo 2

- **Architetture** delle applicazioni: client-server, P2P, ibride, cloud.
- **Requisiti** delle applicazioni (affidabilità, banda, ritardo) e **modello di trasporto** Internet (TCP orientato alla connessione/affidabile, UDP a datagrammi/inaffidabile).
- Protocolli specifici: **HTTP** (stateless, connessioni persistenti/non persistenti, HTTP/2, HTTPS), **FTP** (connessione di controllo + dati separate), **SMTP/POP/IMAP** (posta elettronica), **DNS** (database gerarchico distribuito), **P2P** (BitTorrent, Skype).
- Cenni di **programmazione delle socket** (non d'esame nel dettaglio implementativo).
- **Idea guida da ricordare**: gran parte della complessità di Internet vive **ai margini della rete** (negli host, nelle applicazioni), non nel suo nucleo – un filo conduttore che lega tutto il capitolo (e il Capitolo 1).

4 Capitolo 3 – Il livello di trasporto

4.1 3.1 – Servizi a livello di trasporto

Il livello di trasporto fornisce **comunicazione logica tra processi** applicativi su host diversi (mentre il livello di rete fornisce comunicazione logica tra *host*). I protocolli di trasporto girano solo nei **sistemi terminali**: lato invio, scindono i messaggi applicativi

in **segmenti** e li passano al livello di rete; lato ricezione, riassemblano i segmenti in messaggi e li passano al livello applicazione. Internet mette a disposizione due protocolli di trasporto: **TCP** e **UDP**.

Analogia con la posta ordinaria

12 studenti a Torino inviano lettere a 12 studenti a Enna. **Processi** = studenti; **messaggi delle applicazioni** = lettere nelle buste; **host** = abitazioni; **protocollo di trasporto** = il fratello/sorella maggiore di ciascuna casa che raccoglie/distribuisce la posta ai fratelli minori; **protocollo del livello di rete** = il servizio postale. Il servizio postale (rete) porta le buste da casa a casa; il fratello maggiore (trasporto) le smista poi tra i destinatari *giusti dentro casa*.

TCP: affidabile, consegne nell'ordine originario – offre controllo di congestione, controllo di flusso, setup della connessione. **UDP**: inaffidabile, consegne senza ordine – estensione "senza fronzoli" del servizio di consegna *best effort* del livello di rete. **Servizi che nessuno dei due offre**: garanzie sui ritardi, garanzie sull'ampiezza di banda.

4.2 3.2 – Multiplexing e demultiplexing

Multiplexing (al trasmettitore): raccoglie dati da diverse socket, aggiungendo un header con le informazioni di controllo (PCI) del livello di trasporto, necessarie poi per il demultiplexing. **Demultiplexing** (al ricevitore): usa le informazioni nell'header per consegnare i dati ricevuti alla socket corretta. Ogni segmento ha, tra i primi campi dell'header, un numero di **porta sorgente** e un numero di **porta destinazione** (32 bit combinati).

Porte: l'interfaccia tra applicazione e trasporto; identificate da un intero a **16 bit**. Porte **statiche/well-known** (< 1024, es. 80=HTTP, 25=SMTP, 53=DNS) per servizi standard; porte **dinamiche/ephemeral** (fino a 65535), assegnate automaticamente dal SO per client e connessioni non standard.

4.2.1 Demultiplexing senza connessione (UDP)

Una socket UDP è identificata da **soli 2 parametri**: (indirizzo IP di destinazione, numero di porta di destinazione). Quando l'host riceve un segmento UDP, controlla solo la porta di destinazione e lo consegna a quella socket – datagrammi con IP/porta **sorgente** diversi possono finire nella **stessa** socket, che a sua volta si occuperà di distinguerli se necessario.

4.2.2 Demultiplexing orientato alla connessione (TCP)

Una socket TCP è identificata da **4 parametri**: (IP sorgente, porta sorgente, IP destinazione, porta destinazione). Un server può quindi avere **più socket sulla stessa porta locale** (es. 80), distinte dalla combinazione IP/porta del client: con HTTP non persistente, ogni richiesta genera una socket TCP diversa sul server.

4.3 3.3 – Trasporto senza connessione: UDP [RFC 768]

Protocollo di trasporto "senza fronzoli": consegna **best effort** (i segmenti possono essere persi o consegnati fuori sequenza); **senza connessione** (nessun handshaking, ogni segmento gestito indipendentemente).

Perché esiste UDP?

Non richiede setup della connessione (niente ritardo aggiuntivo); **semplice** (nessuno stato di connessione da mantenere); header **corti**; **senza controllo di congestione**, quindi può inviare dati a raffica senza essere rallentato dalla rete. Impieghi tipici: applicazioni multimediali (tollerano piccole perdite, sensibili alla frequenza più che all'affidabilità), DNS, SNMP.

Struttura del segmento UDP (header di soli 8 byte): num. porta sorgente | num. porta destinazione; lunghezza (in byte, header incluso) | checksum; poi i dati applicativi.

4.3.1 Checksum UDP

Obiettivo: rilevare errori (bit alterati) nel segmento. **Mittente**: tratta il segmento come una sequenza di interi a 16 bit, li somma, e pone nel campo checksum il **complemento a 1** della somma (se c'è un riporto dal bit più significativo, va sommato al risultato). **Ricevente**: somma tutte le parole di 16 bit *incluso* il checksum – se il risultato è una parola di soli 1, nessun errore rilevato (ma non è una garanzia assoluta); altrimenti, errore rilevato.

Esempio numerico completo

Due parole da sommare:

```
Parola 1:  1110 0110 0110 0110
Parola 2:  1101 0101 0101 0101
```

Sommandole bit a bit (con riporto) si ottiene un riporto finale di 1 e una somma parziale `1011 1011 1011 1011`; sommando il riporto (*end-around carry*) si ha:

$$\texttt{Somma} = 1011\,1011\,1011\,1100$$

Il checksum è il complemento a 1 (si inverte ogni bit):

$$\texttt{Checksum} = 0100\,0100\,0100\,0011$$

Verifica al ricevitore: Somma + Checksum = `1111 1111 1111 1111` (tutti 1) $\Rightarrow$ nessun errore rilevato. Questo è vero *per costruzione* quando checksum è il complemento a 1 della somma: ogni bit b sommato al suo complemento $\overline{b}$ dà sempre 1, senza generare ulteriori riporti.

Cenni su rilevamento/correzione d'errore: un **bit di parità** rileva un numero **dispari** di bit errati; un **codice a ripetizione** (invio ripetuto della stessa informazione) permette un voto di maggioranza e la correzione di qualche errore – tecniche più elementari del checksum, utili come introduzione concettuale.

4.4 3.4 – Principi del trasferimento dati affidabile

4.4.1 ARQ e Stop-and-Wait

ARQ (*Automatic Repeat reQuest*): classe di protocolli che recupera le perdite tramite pacchetti di riscontro (**ACK**). Esempi: Stop-and-Wait, Go-Back-N, Selective Repeat, TCP, il MAC del WiFi.

Stop-and-Wait: il trasmettitore invia una PDU (mantenendone una copia), imposta un timeout, e attende l'ACK: se scade il timeout senza ACK, ritrasmette; se riceve l'ACK, verifica checksum e numero di sequenza, poi invia la PDU successiva. Il ricevitore controlla checksum e sequenza: se tutto ok, invia l'ACK e passa i dati al livello superiore; altrimenti scarta (*drop*) la PDU.

Esercizio 3.1 – Efficienza di Stop-and-Wait

Dati: $R = 1$ Gbit/s, ritardo di propagazione $= 15$ ms (quindi $RTT = 30$ ms), $L = 8000$ bit.

$$T_{trans} = \frac{L}{R} = \frac{8000}{10^9} = 8\ \mu s = 0{,}008 \text{ ms}$$

$$\text{throughput applicativo} = \frac{L}{T_{trans} + RTT} = \frac{8000 \text{ bit}}{30{,}008 \text{ ms}} \approx \mathbf{33\ kByte/s}$$

$$\text{efficienza} = \frac{T_{trans}}{T_{trans} + RTT} = \frac{0{,}008}{30{,}008} \approx \mathbf{0{,}027\%}$$

Nonostante un link da 1 Gbit/s, si sfrutta solo lo 0,027% della sua capacità! **I protocolli di trasporto (qui, l'attesa sincrona dell'ACK) limitano l'uso delle risorse fisiche** – da qui la necessità del pipelining.

4.4.2 Pipelining: finestre di trasmissione e ricezione

Col **pipelining**, il trasmettitore accetta di avere più pacchetti "in volo" (senza ancora ACK), tenendo traccia dei loro numeri di sequenza; sia mittente sia ricevitore devono bufferizzare più segmenti.

- Con N pacchetti inviati per RTT (sempre che il tempo di trasmissione di N pacchetti sia $< RTT$): Throughput $= N \times L/(RTT + T_{trans})$. Nell'esempio sopra, con $N = 3$: throughput ≈ 100 kByte/s (triplo rispetto a Stop-and-Wait).
- N è la **dimensione della finestra**.
- **Finestra di trasmissione** W_T: PDU trasmissibili senza ancora avere il rispettivo ACK (limitata dalla memoria del trasmettitore). Puntatori: W_{LOW} (primo pacchetto della finestra) e W_{UP} (ultimo pacchetto **già** trasmesso).
- **Finestra di ricezione** W_R: PDU che il ricevitore può accettare/bufferizzare (limitata dalla sua memoria).

Tipi di ACK: **individuale** (ACK(n) = "ho ricevuto il pacchetto n"); **cumulativo** (ACK(n) = "ho ricevuto tutto fino al pacchetto n *escluso*"); **negativo/NACK** (NACK(n) = "rinviami il pacchetto n"); **piggybacking** (un ACK viaggia dentro un pacchetto dati).

Tabella 7: Go-Back-N vs Selective Repeat

Go-Back-N	**Selective Repeat**
Fino a N pacchetti senza ACK in pipeline	Fino a N pacchetti senza ACK in pipeline
Il ricevente invia **solo ACK cumulativi**: niente ACK se c'è un *gap*	Il ricevente dà ACK ai **singoli** pacchetti
Un solo timer, per il pacchetto più vecchio senza ACK	Un timer **per ciascun** pacchetto senza ACK
Timeout $\Rightarrow$ ritrasmette **tutti** i pacchetti senza ACK	Timeout $\Rightarrow$ ritrasmette **solo** quel pacchetto

Esempio – Go-Back-N in azione ($|W_T| = 4$, $|W_R| = 1$)

Il trasmettitore invia `pkt0,1,2,3` (finestra piena, attende). Il ricevitore riscontra `pkt0` (ack1) e `pkt1` (ack2); **pkt2 si perde**; `pkt3` arriva fuori sequenza e viene **scartato** (il ricevitore, con $|W_R| = 1$, accetta solo l'esatto prossimo atteso), reinviando l'ACK cumulativo già dato (*ack2 duplicato*). Il trasmettitore, ricevuti ack1 e ack2, invia `pkt4` e `pkt5`; ignora l'ACK duplicato; allo scadere del timer sul pacchetto più vecchio non riscontrato, **ritrasmette tutta la finestra**: `pkt2,3,4,5`. Il ricevitore li accetta ora in ordine, riscontrandoli progressivamente. **Nota**: con GBN, i pacchetti vengono consegnati all'applicazione solo **in ordine stretto** – pkt0 e pkt1 subito, ma pkt2-5 tutti insieme solo quando finalmente arriva (di nuovo) pkt2.

Esempio – Selective Repeat in azione ($|W_T| = |W_R| = 4$)

Come sopra, ma il ricevitore riscontra **individualmente** ogni pacchetto corretto, anche se fuori sequenza: quando `pkt3` arriva (con pkt2 ancora mancante), viene **bufferizzato** e riscontrato con `ack3` (non scartato come in GBN!). Quando `pkt2` viene infine ritrasmesso (per timeout **specifico** di pkt2) e arriva, il ricevitore può finalmente **consegnare in ordine** pkt2,3,4,5 tutti insieme all'applicazione.

Vincolo tra dimensione delle finestre e spazio dei numeri di sequenza

Con numeri di sequenza a k bit (periodo 2^k), se si perdono **tutti gli ACK** ma tutti i segmenti arrivano correttamente, W_T e W_R devono restare **contigue e mai sovrapposte**: la regola generale è

$$|W_T| + |W_R| \leq 2^k$$

In **Selective Repeat**, se questo vincolo non è rispettato (es. $k = 3 \Rightarrow 2^k = 8$ ma $|W_T| = |W_R| = 3$ superano involontariamente il limite in scenari particolari), un pacchetto **vecchio ritrasmesso** può essere scambiato per uno **nuovo** a causa del *wraparound* dei numeri di sequenza. Con $|W_T| = |W_R|$, serve $|W_T| \leq 2^{k-1}$; sono anche validi, ad es., $|W_T| = 7, |W_R| = 1$ oppure $|W_T| = 3, |W_R| = 5$. **In Go-Back-N**, poiché $|W_R| = 1$, il vincolo si allenta a $|W_T| \leq 2^k - 1$ (meno restrittivo, perché il ricevitore accetta solo l'esatto pacchetto atteso, eliminando l'ambiguità).

4.5 3.5 – Trasporto orientato alla connessione: TCP [RFC 793, 1122, 1323, 2018, 2581]

Caratteristiche: punto-punto (un mittente, un destinatario); **flusso di byte** affidabile e ordinato (nessun "limite di messaggi"); con pipelining (finestre dinamiche, gestite da controllo di flusso e congestione, con **ACK cumulativi**); **full duplex** (dati in entrambe le direzioni sulla stessa connessione); **MSS** (*Maximum Segment Size*); orientato alla connessione (*handshake* iniziale); **flusso controllato** (il mittente non sommerge il ricevitore).

4.5.1 Struttura del segmento TCP

<table>
<tr><td colspan="2">Numero di porta sorgente — Numero di porta destinazione</td></tr>
<tr><td colspan="2">Numero di sequenza</td></tr>
<tr><td colspan="2">Numero di ACK</td></tr>
<tr><td>Lung. header, non usato, flag <code>U A P R S F</code></td><td>Finestra di ricezione (RWND)</td></tr>
<tr><td>Checksum</td><td>Puntatore a dati urgenti</td></tr>
<tr><td colspan="2">Opzioni (lunghezza variabile)</td></tr>
<tr><td colspan="2">Dati (lunghezza variabile)</td></tr>
</table>

Flag: `URG` (dati urgenti, raramente usato), `ACK` (numero di riscontro valido), `PSH` (invia i dati subito, raramente usato), `RST/SYN/FIN` (impostano/chiudono la connessione). Il **checksum** copre header TCP + dati + uno *pseudoheader* IP (IP sorgente/destinazione, lunghezza del segmento, tipo di protocollo). **Numero di sequenza e ACK contano i *byte*, non i segmenti**; **RWND** è il numero di byte che il destinatario è disposto ad accettare.

4.5.2 Numeri di sequenza e ACK

Il **numero di sequenza** di un segmento è il numero (nel flusso di byte) del **primo byte** del segmento. L'**ACK** è il numero di sequenza del **prossimo byte atteso** dall'altro lato (ACK cumulativo). *Nota*: la gestione dei segmenti fuori sequenza **non è specificata** dallo standard TCP – dipende dall'implementazione.

RWND e bandwidth-delay product

RWND è un campo di **16 bit**: valore massimo 2^{16} Byte = 64 kByte. Con RTT = 100 ms, il *bandwidth-delay product* (quanti dati possono essere "in volo") cresce con la velocità del link:

Banda	Bandwidth-delay product
T1 (1,5 Mbit/s)	18 kB
Ethernet (10 Mbit/s)	122 kB
T3 (45 Mbit/s)	549 kB
Fast Ethernet (100 Mbit/s)	1,2 MB
STS-48 (2,5 Gbit/s)	29,6 MB

Su link molto veloci, i 64 kByte massimi di RWND diventano un **collo di bottiglia** artificiale: si usa allora l'opzione di **window scaling** (il campo RWND non conta più byte singoli, ma multipli di byte).

4.5.3 Setup della connessione: three-way handshake

1. **A→B**: `SYN`(PortA, PortB, ISN_A=x) – flag SYN=1, numero di sequenza iniziale x.
2. **B→A**: `SYN|ACK`(PortB, PortA, ISN_B=y, ACK=x+1) – flag SYN=1 e ACK=1.
3. **A→B**: `ACK`(PortA, PortB, ACK=y+1) – flag ACK=1.

Possibile domanda orale – perché proprio 3 vie?

Servono **entrambe le direzioni** confermate: con solo 2 messaggi, B non avrebbe conferma che A abbia ricevuto il suo ISN (né A potrebbe comunicare il proprio ACK), e un vecchio SYN duplicato (rimasto in rete da una connessione precedente) potrebbe far credere a B di dover aprire una nuova connessione senza che A lo sappia. Il terzo messaggio garantisce che **entrambe le parti** abbiano scambiato e confermato i rispettivi numeri di sequenza iniziali prima di scambiare dati.

4.5.4 Maximum Segment Size (MSS)

TCP invia segmenti lunghi quanto un **MSS** quando possibile (si riferisce alla lunghezza del solo **payload**). Dipende dalla **MTU** (*Maximum Transfer Unit*) del livello di rete/collegamento lungo **tutto** il percorso – non essendoci un meccanismo diretto per comunicarlo, si procede spesso per *trial and error*. **Default**: MSS = 1460 Byte (MTU Ethernet 1500 − 40 Byte di header TCP+IP). **Minimo**: MSS = 536 Byte (l'IP richiede una MTU minima di 576 Byte).

4.5.5 Chiusura della connessione

Chiusura "gentile": le connessioni TCP sono bidirezionali, vanno chiuse in **entrambe** le direzioni. Un lato invia `FIN`=1; l'altro risponde con ACK, e la connessione entra in stato **semi-chiuso** (*half-closed*) – può ancora inviare dati nell'altra direzione. La chiusura è completa solo quando anche l'altra direzione ha scambiato FIN+ACK.

Chiusura "brusca": `RST`=1, usato per resettare connessioni non gestibili o in stato errato (es. un ACK per una connessione mai aperta); entrambi gli host liberano subito le risorse. È più veloce, ma se il segmento RST si perde l'altro lato non saprà che la connessione è stata abortita.

4.5.6 Stima di RTT e Retransmission TimeOut (RTO)

Il timeout deve essere **maggiore di RTT**, ma RTT varia nel tempo: troppo piccolo → *timeout prematuro* e ritrasmissioni inutili; troppo grande → reazione lenta alla perdita.

Formule di stima (RFC 6298)

SampleRTT: tempo dalla trasmissione di un segmento alla ricezione del suo ACK (ignorando le ritrasmissioni).

$$EstimatedRTT = (1-\alpha)\cdot EstimatedRTT + \alpha \cdot SampleRTT \qquad (\alpha = 0{,}125 \text{ tipico})$$
$$DevRTT = (1-\beta)\cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT| \qquad (\beta = 0{,}25 \text{ tipico})$$
$$RTO = EstimatedRTT + 4 \cdot DevRTT$$

Inizializzazione: $RTO = 1\text{s}$ (o più); alla prima misura, $EstimatedRTT = SampleRTT$ e $DevRTT = SampleRTT/2$. La media mobile esponenziale fa sì che l'influenza dei campioni vecchi diminuisca esponenzialmente nel tempo – una stima "livellata" molto meno rumorosa del singolo `SampleRTT`.

4.5.7 Controllo di flusso

Il ricevitore comunica, tramite il campo **RWND** di ogni segmento, quanto spazio libero ha nel proprio buffer di ricezione. Il mittente limita la quantità di dati "in volo" (non ancora riscontrati) al valore di RWND, garantendo che il buffer del ricevitore non vada mai in overflow (che lo costringerebbe a scartare segmenti altrimenti corretti).

4.6 3.6 – Principi del controllo di congestione

Congestione: informalmente, "troppi trasmettitori mandano troppi dati e la **rete** non riesce a gestirli" – **diverso dal controllo di flusso** (che riguarda il *destinatario*, non la rete)! Si manifesta con pacchetti persi (buffer overflow nei router) e ritardi lunghi (codo nei buffer). È uno dei problemi classici dell'informatica di rete.

Nota (non materia d'esame): la teoria delle code (es. modelli M/M/1, con tasso di arrivo λ e di servizio μ) permette di quantificare formalmente questi fenomeni (lunghezza media della coda, probabilità di N pacchetti in coda), ma non è richiesta per l'esame.

I tre scenari classici dei costi della congestione

Scenario 1 (2 mittenti, buffer **infinito**, no ritrasmissioni): il throughput massimo per connessione è $R/2$; ma il **ritardo** cresce senza limite quando $\lambda_{in} \to R/2$ (coda che si allunga indefinitamente).
Scenario 2 (buffer **finito**, con ritrasmissioni): il tasso di arrivo **a livello di trasporto** include le ritrasmissioni ($\lambda'_{in} > \lambda_{in}$). Caso ideale (mittente sa sempre se un pacchetto si è perso): throughput ottimale fino a $R/2$. Caso realistico (il mittente ritrasmette per timeout anche quando il pacchetto era solo in ritardo, non perso): si arrivano a consegnare **copie duplicate**, dimezzando il throughput utile (fino a $R/4$). Con un'idealizzazione intermedia (si ritrasmette solo se il pacchetto è sicuramente perso): throughput fino a $R/3$. **Costo**: risorse sprecate in ritrasmissioni non necessarie.
Scenario 3 (percorsi **multi-hop**, con perdite lungo il cammino): quando λ'_{in} cresce senza controllo, i pacchetti finiscono per essere scartati a un hop **successivo** a quello dove sono stati accettati – ogni pacchetto scartato ha **sprecato tutte le risorse** (banda, buffer) usate per portarlo fin lì. Il throughput λ_{out}, dopo un iniziale aumento, **collassa verso 0** all'aumentare del carico offerto: è il fenomeno del **congestion collapse**.

4.7 3.7 – Controllo di congestione TCP

Due filosofie generali: **end-to-end** (nessun aiuto dalla rete; si inferisce la congestione da perdite e ritardi – il metodo usato da TCP) e **assistito dalla rete** (i router segnalano esplicitamente la congestione, es. un bit ECN).

Attenzione

Le slide sottolineano: **non esiste un solo algoritmo di controllo di congestione in TCP** (Tahoe, Reno, New Reno, Vegas, Westwood, CUBIC, BBR, ... – dipende dal sistema operativo). Quanto segue descrive **una** versione di riferimento (quella del libro di testo), non del tutto completa né universale.

4.7.1 AIMD – Additive Increase, Multiplicative Decrease

Additive increase: aumenta la finestra di **1 MSS ogni RTT** finché non si osservano perdite. **Multiplicative decrease**: dimezza la finestra quando si rileva una perdita. Il grafico risultante della finestra di congestione nel tempo è il classico **"dente di sega"**.

Perché AIMD e non altre combinazioni?

Con K sessioni TCP che condividono un collegamento collo-di-bottiglia di banda R, l'equità vorrebbe che ciascuna ottenga R/K. Su un grafico con il throughput della connessione 1 in ascissa e quello della connessione 2 in ordinata: l'**additive increase** sposta il punto di funzionamento lungo una retta a **pendenza 1** (parallela alla bisettrice di equità); la **multiplicative decrease** lo riporta verso l'origine **proporzionalmente** (mantenendo la direzione dall'origine). Iterando aumento e diminuzione, la traiettoria **converge** verso la retta di equità perfetta – è precisamente questa combinazione (additiva in crescita, moltiplicativa in decrescita) a garantire la convergenza; altre combinazioni (es. moltiplicativa in crescita) non la garantirebbero.

4.7.2 Le tre finestre di TCP

RWND (finestra di ricezione): quanto il ricevitore può bufferizzare. **CWND** (finestra di congestione): quanto il mittente ritiene di poter inviare senza sovraccaricare la **rete** (si adatta dinamicamente). **Finestra di trasmissione effettiva**: $|W_T| = \min(CWND, RWND)$.

4.7.3 Slow start e Congestion avoidance

Algoritmo – Slow start

Inizializzazione: $CWND = 1$ MSS; $SSTHRESH = RWND$ (o $RWND/2$, a seconda dell'implementazione).

- Per ogni ACK valido ricevuto: $CWND$ += 1 MSS – crescita **esponenziale** (1,2,4,8,... MSS, un raddoppio per RTT). Se $CWND \geq SSTHRESH$, passa a Congestion Avoidance.
- Se scatta un timeout: $SSTHRESH = \max(CWND/2, 2)$; $RTO *= 2$; $CWND = 1$; ritrasmette il segmento in timeout.

Algoritmo – Congestion avoidance

- Per ogni ACK valido ricevuto: $CWND \mathrel{+}= MSS \cdot \frac{MSS}{CWND}$ – equivalente ad aumentare $CWND$ di **1 MSS per RTT** (crescita **lineare**, non più esponenziale).
- Se scatta un timeout: stesse azioni di Slow Start (torna in Slow Start con $CWND = 1$).

4.7.4 Fast Retransmit e Fast Recovery

L'idea: gli **ACK duplicati** indicano che segmenti *successivi* a quello mancante sono comunque arrivati – la rete sta ancora funzionando, quindi ha senso reagire più velocemente (e in modo meno drastico) di un timeout completo.

Algoritmo – Fast Retransmit e Fast Recovery

Alla ricezione del 3° ACK duplicato (*fast retransmit*): ritrasmette immediatamente il segmento mancante, senza aspettare il timeout; pone $SSTHRESH = CWND/2$; imposta $CWND = SSTHRESH + 3$ MSS (suppone che solo quel segmento si sia perso: gli altri 3 ACK duplicati indicano 3 segmenti successivi già arrivati); **non** sposta W_{LOW}; memorizza $RECOVER = W_{UP}$ (per sapere quando la fase termina).
Per ogni ulteriore ACK duplicato: $CWND$ += 1 MSS (continua a trasmettere se la finestra lo consente); non sposta W_{LOW}.
Quando arriva un ACK valido che copre RECOVER: $CWND = SSTHRESH$; passa a Congestion Avoidance; sposta W_{LOW}.

Esempio numerico

Congestion avoidance, $CWND = 5$, finestra $[10, \ldots, 14]$. Arriva ACK per 11 $\rightarrow$ finestra $[11, \ldots, 15]$, si trasmette 15. Il segmento 11 si perde; arrivano 3 ACK duplicati ("11"): **Fast Retransmit** $\rightarrow$ $SSTHRESH = 5/2 = 2$; $CWND = 2+3 = 5$; si ritrasmette 11. Arriva un 4° ACK duplicato: $CWND = 6$, si trasmette 16. Finalmente arriva l'ACK per 16 (conferma tutto il recupero): $CWND = SSTHRESH = 2$, si torna in Congestion Avoidance con finestra $[16, 17]$.

Il risultato è il classico grafico **CWND vs. tempo**: crescita esponenziale (slow start) fino a `SSTHRESH`, poi crescita lineare (congestion avoidance) fino a una perdita. Con **Fast Recovery**, una perdita rilevata da ACK duplicati causa solo un piccolo "taglio" (a $SSTHRESH$, non a 1) prima di riprendere la crescita lineare – un dente di sega molto più dolce di quello prodotto da un timeout completo (che riporta sempre $CWND = 1$ e passa da Slow Start).

4.7.5 Formula approssimata del throughput TCP

Formula di Mathis et al. (1997)

$$Thr(RTT, p) \text{ [Gbit/s]} < \frac{MSS}{RTT} \cdot \frac{1}{\sqrt{p}}$$

dove p = probabilità di perdere un pacchetto, MSS in Byte, RTT in secondi.
Esempio: $RTT = 220$ ms, $MSS = 1460$ Byte, $p = 10^{-11}$:

$$Thr < \frac{1460}{0{,}220} \cdot \frac{1}{\sqrt{10^{-11}}} \approx 2{,}1 \text{ Gbit/s}$$

Interpretazione: per sostenere throughput elevati su link ad alto RTT, TCP "loss-based" richiede una probabilità di perdita **estremamente bassa** – un vincolo sempre più difficile da soddisfare man mano che crescono le velocità dei collegamenti (da qui l'interesse per algoritmi alternativi come CUBIC e BBR, si veda oltre).

4.7.6 Equità (fairness) di TCP

TCP e UDP: le applicazioni multimediali spesso evitano TCP (non vogliono ridurre il tasso per il controllo di congestione) e usano UDP a tasso costante, tollerando le perdite.
TCP e connessioni parallele: un'applicazione può aprire più connessioni TCP verso lo stesso host (i browser lo fanno): su un link di banda R già condiviso da 9 connessioni TCP, una nuova applicazione che ne apre **1** ottiene $R/10$; una che ne apre **9** ottiene $R/2$ (controllando 9 dei 18 "flussi equi" totali) – aprire più connessioni è quindi un modo (scorretto) per ottenere più della propria giusta quota di banda.

4.7.7 Oltre TCP loss-based: CUBIC, BBR, QUIC

Il problema di fondo: un TCP "loss-based" rallenta solo **dopo** aver perso un pacchetto – quando questo accade, è già "tardi" per reagire in modo ottimale (si è già superato il punto di funzionamento ottimo, accumulando ritardo di accodamento inutile prima della perdita).

CUBIC. Fa crescere $CWND$ secondo una **funzione cubica del tempo** (non lineare come Reno):

$$CWND_{cubic}(t) = C(t-K)^3 + CWND_{max}, \qquad K = \sqrt[3]{\frac{CWND_{max}(1-\beta)}{C}}$$

con (RFC 8312) $C = 0{,}4$ e $\beta = 0{,}7$ (dopo una perdita, la finestra è scalata di β, non di 0,5 come Reno) – combina una fase **concava** (recupero rapido verso la dimensione pre-perdita) e una **convessa** (esplorazione cauta oltre il massimo precedente), migliorando scalabilità e stabilità su reti veloci a lunga distanza. È l'algoritmo predefinito su Linux da anni (Windows lo ha adottato solo nel 2017).

BBR (*Bottleneck Bandwidth and RTT*, Google 2016). Non si basa sulle perdite, ma **stima direttamente** banda del collo di bottiglia e RTT, per trasmettere a una velocità che **non dovrebbe** generare accodamento (*pacing*: i pacchetti vengono spaziati nel tempo secondo la velocità del nodo più lento, invece di essere inviati tutti insieme non appena la finestra lo permette). Cicla tra fasi `STARTUP` (ricerca esponenziale della banda), `DRAIN` (svuota la coda creata in startup), `PROBE_BW` (esplora se c'è più banda disponibile) e `PROBE_RTT` (ricalibra la stima di RTT minimo). Secondo dati Google, su un link Ethernet a 10 Gbit con RTT= 100ms e perdita dell'1%: CUBIC ottiene ∼3,3 Mbit/s, BBR ∼9100 Mbit/s. Su un link d'accesso da 10 Mbit/s con RTT= 40ms e buffer da 1000 pacchetti: CUBIC ha un RTT medio di 1090 ms (*bufferbloat*), BBR di soli 43 ms.

QUIC (futuro HTTP/3). Protocollo di trasporto, costruito **su UDP** (non TCP), con due obiettivi: eliminare l'**Head-of-Line blocking** tra stream diversi di una stessa connessione (in TCP/HTTP2, un segmento perso blocca **tutti** gli stream multiplexati anche se il dato mancante appartiene solo a uno di essi, perché TCP impone un ordine totale sul flusso di byte; QUIC gestisce gli stream in modo indipendente, così solo lo stream colpito dalla perdita resta bloccato); e ridurre la latenza di connessione, integrando l'handshake TLS in quello di trasporto (spesso **0-RTT** per connessioni già viste, contro $\geq$ 2–3 RTT di TCP+TLS separati). QUIC identifica una connessione con un **ID esplicito**

(non con la 4-tupla IP/porta come TCP): cambiando rete (es. WiFi→4G, con cambio di IP), la connessione TCP andrebbe persa e ristabilita, mentre QUIC può **sopravvivere** rinviando un pacchetto con lo stesso ID di connessione.

4.8 Riepilogo del capitolo 3

- **Principi** del livello di trasporto: multiplexing/demultiplexing, trasferimento dati affidabile (ARQ, pipelining, GBN/SR), controllo di flusso, controllo di congestione.
- **UDP**: semplice, inaffidabile, con checksum opzionale.
- **TCP**: connessione (three-way handshake), affidabilità (ACK cumulativi, stima di RTT/RTO), controllo di flusso (RWND), controllo di congestione (AIMD, slow start, congestion avoidance, fast retransmit/recovery).
- **Protocolli di trasporto moderni**: CUBIC (funzione cubica), BBR (basato su banda/RTT stimati, non su perdite), QUIC (su UDP, niente HOL blocking, 0-RTT, resiliente al cambio rete).
- **Prossimo argomento**: si lascia la "periferia" della rete (applicazione e trasporto) per entrare nel **"cuore"** – il livello di rete (Capitolo 4-5).

5 Capitolo 4 – Livello di rete (Parte 1: piano dati e protocollo IPv4)

5.1 Visione d'insieme

5.1.1 Le due funzioni del livello di rete

- **Inoltro** (*forwarding*): operazione **locale**, a scala locale – spostamento del pacchetto da una porta di ingresso a una di uscita **dello stesso router**.
- **Instradamento** (*routing*): operazione a scala **globale** – determinare l'intero percorso che un pacchetto deve seguire dalla sorgente alla destinazione, tramite **algoritmi di routing**.

Analogia: fare un viaggio

Forwarding ≈ spostarsi all'interno di uno snodo (cambiare treno, prendere uno svincolo autostradale). **Routing** ≈ pianificare tutte le tappe del viaggio dall'inizio alla fine.

5.1.2 Data plane e control plane

Data plane ("piano dati"): funzione **locale** a ogni router, che determina come inoltrare un datagramma da una porta di ingresso a una di uscita – è la funzione di *forwarding*. **Control plane** ("piano di controllo"): logica **globale** di rete, che determina come instradare un datagramma sull'intero percorso end-to-end dall'host mittente a quello destinatario – popola le **tabelle di inoltro** usate poi dal data plane di ciascun router (*header livello 3 in ingresso* → tabella → *uscita*).

5.2 Come è fatto un router

Architettura di alto livello: un **processore** (routing, gestione, control plane – **software**, operazioni su scala di **millisecondi**) collegato a **porte di ingresso e uscita** tramite un **sistema di commutazione ad alta velocità** (inoltro, data plane – **hardware**, operazioni su scala di **nanosecondi**).

5.2.1 Porte di ingresso

Pipeline: **terminazione di linea** (livello fisico, ricezione dei bit) → **protocollo di livello data link** (es. Ethernet) → **inoltro** (con buffer). Il sistema di commutazione è **decentralizzato**: usando i campi dell'header di livello 3, ogni porta trova la porta di uscita corretta tramite la tabella di inoltro locale (schema "*match plus action*"), con l'obiettivo di **non introdurre ritardi ulteriori** oltre a quelli fisiologici.

5.2.2 Sistemi di commutazione

Il **tasso di commutazione** è la frequenza con cui i pacchetti passano dagli ingressi alle uscite – idealmente N volte la velocità di una singola porta, con N ingressi. Tre architetture storiche:

- **A memoria**: i pacchetti sono copiati nella memoria del router (come un computer tradizionale, commutazione controllata dalla CPU) – servono **2 accessi al bus di sistema** per datagramma; velocità limitata dalla banda della memoria.
- **A bus**: un bus dati condiviso interno trasmette i datagrammi dagli ingressi alle uscite – limitato dalla **contesa del bus** (es. 32 Gbit/s su un Cisco 5600, adeguato per router di accesso/aziendali).
- **A matrice** (*crossbar*): matrice di punti di interconnessione tra ogni linea di ingresso e ogni linea di uscita (ispirata ai commutatori telefonici elettromeccanici) – supera i limiti di velocità del bus; design avanzato: frammentazione dei datagrammi in celle di lunghezza fissa che attraversano la matrice (es. Cisco 12000: 60 Gbit/s).

Head-of-Line (HOL) blocking

Se il commutatore è più lento della velocità aggregata delle porte di ingresso, si formano **code agli ingressi**: ritardi, ma anche perdite per overflow dei buffer. Un pacchetto in testa a una coda di ingresso, in attesa di una porta di uscita occupata, **blocca** tutti i pacchetti dietro di lui – anche quelli diretti verso porte di uscita **libere**! Questo è il **blocco head-of-line**.

5.2.3 Porte di uscita e scheduling

Se i datagrammi arrivano dal commutatore più in fretta di quanto la porta di uscita possa smaltirli, si accodano nel **buffer di uscita**; se il buffer si riempie, si scarta.

Quanta memoria serve nei buffer? (RFC 3439)

Regola empirica classica: memoria $\approx RTT \times C$ (un tipico RTT moltiplicato per la velocità del link). Con $RTT = 250$ ms e $C = 10$ Gbit/s, servirebbero 2,5 Gbit di buffer! Raccomandazioni più recenti, con N flussi attivi sul link, riducono il requisito a:

$$\text{memoria} \approx \frac{RTT \times C}{\sqrt{N}}$$

Scheduling: quale pacchetto trasmettere sul link. **FIFO** (*first-in-first-out*): invia in ordine di arrivo. **Politiche di scarto** quando la coda è piena: *tail drop* (scarta gli arrivi), *priority drop* (scarta/cancella in base a una priorità), *random drop* (scarto casuale) – ognuna con implicazioni diverse su equità e *network neutrality*.

5.3 Il protocollo IP

5.3.1 Formato del datagramma IPv4

<table>
<tr><td colspan="2">Ver Lungh. header Tipo di servizio</td><td colspan="2">Lunghezza totale</td></tr>
<tr><td colspan="2">ID del datagramma (16 bit)</td><td>Flag</td><td>Offset del frammento</td></tr>
<tr><td>Time-to-live</td><td>Protocollo superiore</td><td colspan="2">Checksum dell'header</td></tr>
<tr><td colspan="4">Indirizzo IP sorgente (32 bit)</td></tr>
<tr><td colspan="4">Indirizzo IP destinazione (32 bit)</td></tr>
<tr><td colspan="4">Opzioni IP (non obbligatorio) + padding</td></tr>
<tr><td colspan="4">DATI (lunghezza variabile, tipicamente un segmento TCP o UDP)</td></tr>
</table>

Ver (4 bit) Numero di versione IP (4 o 6).

Lunghezza header (4 bit) Numero di parole da 32 bit nell'header (= 5 senza opzioni).

Tipo di servizio / ToS (8 bit) Classe di servizio (potenzialmente DiffServ/ECN, raramente usato in pratica).

Lunghezza totale (16 bit) Byte totali nel datagramma (header + dati).

ID del datagramma (16 bit) Numero (di solito sequenziale) usato per riassemblare i frammenti di uno stesso datagramma.

Flag (3 bit) Indicano se il datagramma è un frammento, e se è l'ultimo.

Offset del frammento (13 bit) Posizione del frammento nel datagramma originale, in multipli di 8 Byte.

TTL (8 bit) Decrementato di 1 da ogni router attraversato; a 0, il router scarta il datagramma e notifica il mittente (via ICMP).

Protocollo superiore (8 bit) Protocollo incapsulato nei dati (es. 6=TCP, 17=UDP).

Checksum dell'header (16 bit) Complemento a 1 della somma (con riporto) delle parole a 16 bit dell'header – **stesso principio del checksum UDP**, ma copre solo l'header IP.

Indirizzi IP sorgente/destinazione (32 bit ciascuno) Mittente iniziale e destinatario finale.

Opzioni / Padding Raramente usate; padding a zero per allineare l'header a multipli di 32 bit.

5.3.2 Frammentazione

Ogni tecnologia di livello 2 impone una **MTU** (*Maximum Transmission Unit*) massima: Ethernet 1500 Byte, WiFi 2304, FDDI 4352, Token Ring 4464–17914, PPP variabile. Se un router deve inoltrare un datagramma su un collegamento con MTU inferiore alla dimensione del datagramma, lo **frammenta**: ogni frammento ha il formato di un datagramma IP normale, con i **flag** che indicano se è un frammento (e se è l'ultimo, `M`=*more fragments*) e l'**offset del frammento** che indica come riassemblarli. Il flag `D` (*do not fragment*) impone di scartare il pacchetto anziché frammentarlo.

Chi riassembla i frammenti?

Solo la destinazione finale, mai un router intermedio: un router che riceve frammenti già formati li inoltra semplicemente come pacchetti indipendenti (comportamento **stateless**). Vantaggi: i frammenti possono seguire percorsi diversi; un router intermedio non deve attendere di aver ricevuto tutti i frammenti prima di inoltrare quelli già arrivati.

Un datagramma frammentato non può essere ricomposto finché non arrivano tutti i frammenti; il ricevitore li bufferizza per un tempo massimo (oltre il quale scarta tutto). **In pratica**, la frammentazione IP è quasi disabilitata su Internet moderna (alcuni router non la implementano nemmeno; IPv6 non la consente più *in-network*): motivi principali sono di **sicurezza** – i firewall stateless faticano a ispezionare header TCP/UDP sparsi su più frammenti (attacco *overlapping fragments*), la frammentazione può abilitare attacchi di riempimento memoria, e implementazioni errate del riassemblaggio hanno causato in passato vulnerabilità gravi (crash, esecuzione di codice arbitrario).

5.3.3 Indirizzamento IP

Un indirizzo IP (32 bit) è associato a un'**interfaccia di rete**, non a un host: un router, avendo più interfacce, ha necessariamente più indirizzi IP. Notazione *dotted decimal*: quattro numeri 0–255 separati da punti (es. `223.1.1.1`).

Un indirizzo si divide gerarchicamente in **prefisso** (*NetID*, identifica la rete, coordinato globalmente) e **suffisso** (*HostID*, identifica l'host sulla rete, assegnabile localmente).

Indirizzamento classful (storico, obsoleto). Classe A (primo bit 0): 7 bit NetID / 24 bit HostID → 128 reti da 16.777.216 host. Classe B (10): 14/16 → 16.384 reti da 65.536 host. Classe C (110): 21/8 → 2.097.152 reti da 256 host. Classe D (1110): multicast. Classe E (1111): riservata. **Problema**: granularità pessima (A troppo grandi, C troppo piccole) → spreco enorme di indirizzi ed esaurimento rapido dello spazio disponibile.

CIDR – Classless Inter-Domain Routing. Il confine prefisso/suffisso diventa **arbitrario**, notato `ddd.ddd.ddd.ddd/m` dove m = bit di prefisso (**subnet mask**). La subnet

mask ha m bit a 1 seguiti da zeri; un router estrae il prefisso di un indirizzo D con un **AND bit-a-bit**: $D \,\&\, M$.

Esempio – assegnazione CIDR

Un ISP possiede `128.211.0.0/16` e ha 3 clienti che necessitano 12, 9 e 6 indirizzi. Può assegnare: Cliente 1 `128.211.0.16/28` (16 indirizzi), Cliente 2 `128.211.0.32/28` (16 indirizzi), Cliente 3 `128.211.0.48/29` (8 indirizzi, prefisso più lungo perché servono meno indirizzi). Con 4 clienti da 12 indirizzi ciascuno, si può usare sempre `/28` (16 indirizzi, il "numero magico" da sommare per ottenere i prefissi successivi): `.0/28`, `.16/28`, `.32/28`, `.48/28`.

Tabella delle maschere più comuni (ottava finale): 128=`/25`, 192=`/26`, 224=`/27`, 240=`/28`, 248=`/29`, 252=`/30`, 254=`/31` (2 indirizzi, usata su link punto-punto), 255=`/32` (un solo host, es. *host route*).

Longest prefix matching

Quando **più righe** della tabella di inoltro corrispondono a un indirizzo di destinazione (con maschere di lunghezza diversa), il router sceglie sempre la corrispondenza con il **prefisso più lungo**. Esempio: destinazione `200.23.19.7`, tabella con `200.23.16.0/20` (eth0) e `200.23.18.0/23` (eth1): entrambe corrispondono (200.23.19.7 & /20 = 200.23.16.0; 200.23.19.7 & /23 = 200.23.18.0), ma si sceglie `/23` (più specifico) → **eth1**. Se nessuna riga corrisponde, si usa una riga di **default** `0.0.0.0/0` (il *default gateway*).

Aggregazione dei percorsi (*route summarization*): più prefissi contigui e della stessa dimensione possono essere annunciati verso l'esterno come un unico prefisso più corto (es. otto reti `/23` consecutive aggregate in un solo `/20`), riducendo la dimensione delle tabelle di instradamento globali. Se una sotto-rete si stacca dal gruppo (es. cambia ISP), il router che la annuncia deve pubblicizzare anche un prefisso **più specifico** per quella sotto-rete, che vince per longest-prefix-matching.

Le tabelle di inoltro moderne includono anche il **next hop**: non solo l'interfaccia di uscita, ma l'indirizzo IP del router successivo a cui consegnare il pacchetto (il **data plane** legge la tabella; il **control plane** la popola).

5.3.4 NAT – Network Address Translation

Indirizzi IP privati (non instradabili su Internet pubblica, riservati per reti private): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. Riutilizzabili in ogni rete privata, assegnati tipicamente in modo dinamico.

Un **NAT** è un apparato allacciato sia alla rete privata sia a Internet, che ri-mappa (indirizzo IP privato, porta sorgente) → (indirizzo IP pubblico del NAT, nuova porta sorgente), mantenendo una **tabella di traduzione**, e viceversa per il traffico di ritorno.

Esempio completo di NAT

1. L'host `10.0.0.1` invia un datagramma a `128.119.40.186:80` (sorgente `10.0.0.1:3345`).
2. Il NAT sostituisce sorgente con `138.76.29.7:5001` (il proprio IP pubblico + nuova porta) e **memorizza** la corrispondenza $(10.0.0.1, 3345) \leftrightarrow (138.76.29.7, 5001)$ in tabella.
3. Arriva la risposta con destinazione `138.76.29.7:5001`.
4. Il NAT consulta la tabella e riscrive la destinazione come `10.0.0.1:3345`, inoltrando all'host originale.

Con 16 bit di porta, un solo IP pubblico supporta fino a ∼60.000 connessioni simultanee.

Controversia: il NAT viola l'architettura a livelli (un dispositivo di rete che modifica campi di trasporto); la scarsità di indirizzi *dovrebbe* essere risolta da IPv6, non "aggirata"; costringe gli sviluppatori a gestire il caso NAT esplicitamente (problematico per il P2P, dove entrambe le parti potrebbero essere dietro NAT) – soluzioni: *port forwarding*, *hole punching*, UPnP/STUN. **Un NAT può anche tornare utile**: se non controllate tutti i router lungo un percorso (es. un router upstream che non ha una rotta di ritorno verso la vostra rete privata), un NAT locale fa apparire il traffico come proveniente dalla propria interfaccia (che il router upstream conosce), risolvendo il problema.

5.3.5 Indirizzi IP speciali

- **Indirizzo di rete**: HostID tutto a 0 (es. `128.211.0.16/28`), non assegnabile a un host.
- **Broadcast diretto** (*directed broadcast*): HostID tutto a 1 – una copia viaggia fino alla rete di destinazione, poi viene consegnata a tutti gli host di quella rete; instradato normalmente dai router.
- **Broadcast limitato** (*limited broadcast*): `255.255.255.255`, mai inoltrato da un router – usato quando un host non conosce ancora la propria rete.
- **"Questo computer"**: `0.0.0.0`, usato come sorgente da un host che non ha ancora un IP valido.
- **Loopback**: prefisso `127.0.0.0/8` (tipicamente `127.0.0.1`) – i datagrammi non lasciano mai il computer, utile per testare applicazioni di rete in locale.
- **Multicast**: `224.0.0.0`–`239.255.255.255` (primi bit `1110`) – un pacchetto per un gruppo di destinatari, con i router che creano le copie necessarie (minimizzando la duplicazione rispetto a tanti unicast separati); supporto reale in Internet molto limitato.
- **Link-local**: `169.254.0.0/16` – scelto a caso da un host che non trova un server DHCP, permette la comunicazione sulla sola sottorete locale (si veda DHCP).

5.3.6 ARP – Address Resolution Protocol

Per consegnare un frame di livello 2, serve l'indirizzo **MAC** (48 bit, 12 cifre esadecimali) del destinatario – ma le decisioni di instradamento IP usano indirizzi IP. **ARP** traduce IP→MAC, ma **solo per host sulla stessa rete** (se il NetID di destinazione differisce dal proprio, si inoltra sempre al router, di cui va comunque risolto il MAC).

Meccanismo

Per risolvere l'IP C: il mittente invia in **broadcast** "mi serve il MAC che corrisponde all'IP C"; la richiesta raggiunge tutti gli host della rete; **solo** l'host con IP C risponde (in unicast) "sono io, ecco il mio MAC". I messaggi ARP sono incapsulati direttamente nel payload di un frame (es. Ethernet, campo `Type`=`0x806`=2054).

ARP caching: le risposte si memorizzano in cache per evitare una richiesta ARP per ogni datagramma; nuove risposte sovrascrivono le vecchie; le entry scadono periodicamente (∼30s); si consulta sempre la cache prima di inviare una richiesta.

Proxy ARP: un router (o altro apparato) può rispondere ai messaggi ARP **per conto di** un host su una rete diversa, incaricandosi poi di inoltrare i pacchetti – utile per evitare di dover creare una sottorete separata.

5.3.7 ICMP – Internet Control Message Protocol

Protocollo "partner" di IP, usato per notificare errori e per gestione/manutenzione della rete: IP ha bisogno di ICMP per segnalare errori, ICMP si serve di IP per trasportare i propri messaggi (interdipendenti). Header semplice: campo `Type` (+`Code`+`Checksum`) identifica il messaggio.

Tipo	**Significato**
Destination/Port unreachable	Il pacchetto non è consegnabile / nessuno ascolta su quella porta
Time exceeded	TTL sceso a 0 (possibile routing loop)
Parameter problem	Valore non valido in un campo dell'header IP
Source quench	Chiede al mittente di rallentare
Redirect	Indica al router un percorso migliore
Echo request/reply	Usati da `ping`
Router advertisement/solicitation	Per scoprire i router vicini
Fragmentation needed	Flag "non frammentare" attivo ma datagramma > MTU

Due classi: messaggi di **errore** (es. Time Exceeded, Destination Unreachable) e di **recupero informazioni** (Echo Request/Reply). **Regola importante**: un messaggio ICMP che genera a sua volta un errore **non produce** un ulteriore messaggio ICMP di errore (evita cascate/loop di notifiche).

Come funzionano davvero ping e traceroute

ping: invia un Echo Request, misura il tempo fino all'Echo Reply → verifica raggiungibilità e stima l'RTT. **traceroute**: invia datagrammi con **TTL crescente** (1, 2, 3, ...). Con TTL=1, il **primo** router lo decrementa a 0, scarta il pacchetto e invia un messaggio "Time Exceeded" col proprio indirizzo; con TTL=2, il secondo router fa lo stesso; e così via, finché il pacchetto non raggiunge la destinazione. Ogni messaggio Time Exceeded ricevuto rivela un hop del percorso – esattamente il meccanismo dietro la tabella di traceroute vista nel Capitolo 1.

5.3.8 DHCP – Dynamic Host Configuration Protocol

Assegna automaticamente un indirizzo IP (e altri parametri) a un host quando si accende, e lo libera quando non più utilizzato.

I quattro messaggi ("DORA")

1. **DHCP discover** [broadcast, opzionale]: "c'è un server DHCP su questa rete?"
2. **DHCP offer** [broadcast, opzionale]: "sono io! puoi usare questo indirizzo IP"
3. **DHCP request** [broadcast]: "ok, lo prendo!"
4. **DHCP ack** [broadcast]: "ok, è tutto tuo!"

L'indirizzo è un **prestito** (*lease*) valido per un tempo limitato; alla scadenza, il client può richiederne l'estensione (di norma concessa) o il server può riassegnare l'indirizzo ad altri (in tal caso il client **deve** smettere di usarlo). DHCP usa **UDP** (non affidabile per design: nessuna risposta → il client ritrasmette; risposta duplicata → ignorata).

Il messaggio DHCP trasporta anche: server e router di default (*gateway*), server DNS, subnet mask, e altre opzioni di configurazione. Essendo inviato in broadcast di rete locale, un messaggio DHCP normalmente **non attraversa i router** – a meno che siano esplicitamente configurati per inoltrarlo a un server DHCP noto (altrimenti servirebbe un server per ogni sottorete).

Se non c'è DHCP: si configura l'IP manualmente, oppure il sistema si auto-assegna un indirizzo **link-local** (`169.254.0.0/16`): sceglie un indirizzo a caso nel blocco, verifica con una richiesta ARP che nessun altro lo stia già usando, e se libero lo adotta – permette almeno la comunicazione sulla rete locale.

5.3.9 Il viaggio di un pacchetto

Riepilogo end-to-end: A → R → B

1. A crea il datagramma IP (sorgente A, destinazione B) e lo incapsula in un frame con MAC destinazione = MAC di R (assumendo ARP già risolto).
2. R riceve il frame, ne estrae il datagramma IP e lo passa al proprio livello IP.
3. R consulta la propria tabella di inoltro (longest prefix matching su B) e decide su quale interfaccia/next-hop inoltrare.
4. R incapsula lo **stesso** datagramma IP (sorgente/destinazione IP **invariate**) in un **nuovo** frame, con MAC destinazione = MAC di B.
5. B riceve il frame, estrae il datagramma e lo passa al proprio livello IP.

Punto chiave: gli indirizzi **IP** restano costanti lungo tutto il percorso (tranne con NAT); gli indirizzi **MAC** cambiano a **ogni singolo hop**, perché identificano solo il tratto di collegamento locale.

5.3.10 IPv6

Motivazioni. Inizialmente, ampliare lo spazio di indirizzi prima dell'esaurimento di IPv4. Motivi aggiuntivi: un header più semplice velocizza l'elaborazione; facilita la gestione della qualità del servizio.

Formato del datagramma (header fisso a 40 Byte). `vers.`|`priority`|`flow label`; `payload length`|`next header`|`hop limit`; **indirizzo sorgente** (128 bit); **indirizzo destinazione** (128 bit); `data`.

- **Flow label**: etichetta per i datagrammi dello stesso "flusso" (concetto non rigorosamente definito).
- **Priority**: priorità relativa dei datagrammi di uno stesso flusso.
- **Next header**: sostituisce "protocollo superiore"; identifica anche eventuali opzioni concatenate fuori dall'header principale.

Altri cambiamenti rispetto a IPv4. Checksum rimosso (delegato ai livelli superiori/inferiori, per velocità). **Frammentazione vietata** in rete: se un datagramma eccede l'MTU di un link, viene scartato (col relativo messaggio ICMPv6 "*Packet Too Big*"), spostando l'onere sul mittente (*Path MTU Discovery*). ICMPv6 aggiunge anche funzioni per la gestione dei gruppi multicast.

Notazione degli indirizzi (128 bit, $\sim 3{,}4 \times 10^{38}$). Esadecimale, 8 gruppi da 4 cifre (es. `2a03:2880:f108:0083:face:b00c:0000:25de`). **Compattazione**: si omettono gli zeri iniziali di ogni gruppo (`0083`→`83`); i gruppi **consecutivi** di soli zeri si comprimono con `::`, usabile **una sola volta** per indirizzo (sul gruppo più lungo, se ce n'è più di uno, RFC 5952): l'esempio sopra diventa `2a03:2880:f108:83:face:b00c::25de`.

Prefissi/sottoreti: stessa logica del CIDR (`/m`); composizione tipica: prefisso di routing (48 bit) + subnet ID (16 bit) + ID di interfaccia (64 bit) = 128 bit.

Indirizzi speciali: non specificato `::/128`; loopback `::1/128`; IPv4-mapped `::ffff:0:0/96` (le ultime 32 bit sono l'IPv4 in esadecimale, es. `193.175.55.16` → `::ffff:c1af:3710`); multicast `ff00::/8`; link-local unicast `fe80::/10` (l'equivalente IPv6 di `169.254.0.0/16`, per il networking locale senza router/DHCP).

Transizione IPv4→IPv6: tunneling. Non tutti i router possono aggiornarsi simultaneamente. Il **tunneling** incapsula un datagramma IPv6 interamente come **payload** di un datagramma IPv4, per attraversare tratti di rete ancora solo-IPv4 tra due router IPv6 (che fanno rispettivamente da "entrata" e "uscita" del tunnel): il payload IPv4 contiene per intero header + payload IPv6 originali, che vengono **riestratti intatti** all'uscita del tunnel.

5.4 Riepilogo (Parte 1)

- **Router**: separazione tra *data plane* (forwarding, veloce, hardware) e *control plane* (routing, lento, software); architetture di commutazione (memoria/bus/matrice) e problematiche di accodamento (HOL blocking, scheduling).
- **IPv4**: formato del datagramma, frammentazione (ormai rara in pratica), indirizzamento gerarchico con CIDR e *longest prefix matching*.
- **Protocolli di supporto**: NAT (traduzione indirizzi/porte), ARP (IP→MAC sulla rete locale), ICMP (errori e diagnostica, alla base di ping/traceroute), DHCP (assegnazione automatica degli indirizzi).
- **IPv6**: header semplificato, spazio di indirizzi enorme, niente più frammentazione in rete, transizione tramite tunneling.

Prossimo argomento (Parte 2): come si **costruiscono** le tabelle di inoltro – algoritmi di routing *link state* (Dijkstra) e *distance vector* (Bellman-Ford), protocolli intra-AS (OSPF, RIP) e inter-AS (BGP).

5.5 Protocolli di instradamento (Parte 2: come si costruiscono le tabelle)

Finora abbiamo assunto che le tabelle di inoltro fossero già pronte. Ora vediamo **come si costruiscono**: è compito degli **algoritmi di routing**.

5.5.1 Il problema come grafo

Si modella la rete come un grafo $G = (N, E)$: N = insieme dei router, E = insieme dei collegamenti. Ogni link (x, x') ha un **costo** $c(x, x')$, che può rappresentare numero di salti, inverso della banda, congestione, ecc. – dipende dall'algoritmo usato. Il costo di un percorso è la somma dei costi dei link attraversati; l'obiettivo è trovare il percorso a **costo minimo** tra due nodi.

Due grandi famiglie di algoritmi:

- **Informazioni globali** (*link state*): ogni nodo conosce l'intera topologia e i costi di **tutti** i link (ottenuta facendo circolare in broadcast lo stato dei link).
- **Informazioni distribuite** (*distance vector*): ogni nodo conosce solo i propri vicini diretti e il costo verso di essi; il calcolo del percorso è **iterativo**, tramite scambio di informazioni con i soli vicini.

5.5.2 Algoritmo di Dijkstra (link state)

Notazione e pseudocodice

$c(x,y)$: costo del link x-y ($=\infty$ se non collegati). $D(v)$: costo corrente del percorso verso v. $p(v)$: predecessore di v lungo tale percorso. N': insieme dei nodi per cui il cammino minimo è già stato determinato.

```
1.  Inizializzazione: N' = {u}   // u = nodo sorgente
2.  for all nodes v:
3.      if v e' adiacente a u: D(v) = c(u,v), p(v) = u
4.      else: D(v) = infinito
5.  Loop:
6.      trova w non in N' tale che D(w) e' minimo
7.      aggiungi w a N'
8.      aggiorna D(v) per ogni v adiacente a w e non in N':
9.          if D(w) + c(w,v) < D(v):
10.             D(v) = D(w) + c(w,v);  p(v) = w
11. until (tutti i nodi sono in N')
```

Esempio completo

Grafo: nodi $\{u,v,w,x,y,z\}$, sorgente u. Costi: $c(u,w)$=3, $c(u,x)$=5, $c(u,v)$=7, $c(w,x)$=4, $c(w,v)$=3, $c(w,y)$=8, $c(x,y)$=7, $c(x,z)$=9, $c(v,y)$=4, $c(y,z)$=2.

Passo	N'	$D(v),p(v)$	$D(w),p(w)$	$D(x),p(x)$	$D(y),p(y)$	$D(z),p(z)$
0	u	7,u	**3,**u	5,u	∞	∞
1	uw	6,w	–	**5,**u	11,w	∞
2	uwx	**6,**w	–	–	11,w	14,x
3	$uwxv$	–	–	–	**10,**v	14,x
4	$uwxvy$	–	–	–	–	**12,**y
5	$uwxvyz$	–	–	–	–	–

Ad ogni passo si aggiunge a N' il nodo col D minimo non ancora incluso (in **grassetto**). **Tabella di inoltro risultante per** u (ricostruita seguendo i predecessori a ritroso): verso v → next-hop w (percorso u-w-v); verso w → next-hop w; verso x → next-hop x; verso y → next-hop w (percorso u-w-v-y); verso z → next-hop w (percorso u-w-v-y-z).

Complessità: con n nodi, $n(n+1)/2$ confronti → $O(n^2)$ nella versione base (esistono implementazioni $O(n \log n)$). **Attenzione**: se il costo di un link dipende dal traffico che ci passa (es. costo $\propto$ carico), l'algoritmo può **oscillare** – tutti i nodi ricalcolano

simultaneamente sulla base di costi che, nel frattempo, sono già cambiati a causa delle nuove scelte di routing altrui.

5.5.3 Sistemi Autonomi e OSPF

Internet è composta da moltissime sottoreti indipendenti. Un **Sistema Autonomo** (**AS**) è un insieme di router sotto lo stesso controllo amministrativo (es. un ISP), identificato da un numero assegnato dai registri regionali ICANN (RFC 1930). Questo porta alla distinzione tra **Intra-AS routing** (dentro un AS) e **Inter-AS routing** (tra AS diversi, si veda BGP più avanti).

OSPF (*Open Shortest Path First*) è il protocollo intra-AS a link state più diffuso: dissemina lo stato dei link assumendo che tutti i router conoscano l'intera topologia dell'AS, e calcola i percorsi con **Dijkstra**. Usa direttamente datagrammi IP (nessun livello di trasporto), inviando gli aggiornamenti all'indirizzo multicast `224.0.0.5`. Tre sotto-protocolli: **Hello** (verifica quali link/vicini sono attivi), **Exchange** (informa un vicino appena scoperto sulla topologia nota), **Flooding** (propaga i cambiamenti di stato dei link a tutti i router – un messaggio per link punto-punto, oppure uno solo per dominio di broadcast, per ridurre le repliche inutili).

OSPF gerarchico: su reti grandi, una gerarchia a due livelli – una **dorsale** (*backbone*, trattata essa stessa come un'"area") e diverse **reti di area**; i link state circolano **solo dentro la propria area**; i router di bordo di ciascuna area riassumono la distanza verso le reti dell'area agli altri router, limitando la dimensione dei messaggi flooded.

Il costo dei link non è "neutro"

Poiché in OSPF l'**unica** cosa che determina la scelta del percorso è il costo dei link, un operatore può **scegliere ad arte** i costi per ottenere un certo controllo sul traffico che attraversa la propria rete (*traffic engineering*) – di fatto invertendo la relazione causa-effetto "naturale" del protocollo: invece di lasciare che il costo determini il percorso, si parte da un obiettivo di instradamento desiderato e si sceglie il costo di conseguenza.

5.5.4 Algoritmo di Bellman-Ford (distance vector)

Pseudocodice (nodo x)

```
1.  Inizializzazione: per ogni destinazione y: D_x(y) = c(x,y)
      // c(x,y) = infinito se non vicini, 0 se x = y
2.  invia D_x a tutti i vicini in N_x
3.  Loop:
4.      attendi un cambiamento locale o la ricezione di D_w da un vicino w
5.      per ogni destinazione y:
6.          D_x(y) = min_v { c(x,v) + D_v(y) }      // v vicino di x
7.          next_hop_x(y) = argmin_v { c(x,v) + D_v(y) }
8.      if D_x e' cambiato per una qualsiasi destinazione:
9.          invia il nuovo D_x a tutti i vicini
```

Questa è l'equazione di **Bellman-Ford**: il costo minimo verso y è il minimo, su tutti i vicini v, di (costo verso v) + (costo che v dichiara verso y).

Esempio completo – tre nodi

Grafo triangolare: $c(X,Y)$=2, $c(X,Z)$=7, $c(Y,Z)$=1.
Stato iniziale (solo vicini diretti): X: $Y \to (Y,2)$, $Z \to (Z,7)$. Y: $X \to (X,2)$, $Z \to (Z,1)$. Z: $X \to (X,7)$, $Y \to (Y,1)$.
Y invia $D_Y = [(X,2),(Z,1)]$ ai suoi vicini. X **riceve**: $D_X(Y) + D_Y(Z) = 2+1 = 3 < D_X(Z) = 7 \Rightarrow$ aggiorna $Z \to (Y,3)$. Z **riceve**: $D_Z(Y) + D_Y(X) = 1+2 = 3 < D_Z(X) = 7 \Rightarrow$ aggiorna $X \to (Y,3)$.
Convergenza finale: X: $Y \to (Y,2)$, $Z \to (Y,3)$ (via Y, non più il link diretto costoso). Y: $X \to (X,2)$, $Z \to (Z,1)$ (invariato, già ottimo). Z: $X \to (Y,3)$, $Y \to (Y,1)$ (via Y). Il link diretto X-Z (costo 7) viene "scavalcato" dal percorso X-Y-Z (costo 3).

Un secondo esempio nelle slide (grafo a 5 nodi A, B, C, D, E con $c(A,B)$=7, $c(A,E)$=1, $c(B,C)$=1, $c(B,E)$=5, $c(C,D)$=2, $c(D,E)$=2) mostra la stessa dinamica su una topologia più ricca, convergendo dopo diversi cicli di scambio a percorsi che spesso passano per E (il nodo più centrale).

Il problema del "count-to-infinity"

Se un link si rompe, le informazioni **vecchie e ormai sbagliate** possono rimbalzare tra due nodi che si "credono" a vicenda: es. A raggiungeva una rete solo tramite X; se il link A-X si rompe, ma B crede ancora che A sappia raggiungere quella rete (informazione non aggiornata), B continuerà ad annunciare ad A un percorso che in realtà passa per A stesso! Il costo dichiarato **cresce lentamente** ad ogni scambio (4, 6, 8, ...) invece di essere riconosciuto subito come infinito – da cui il nome. **Le buone notizie (costi minori) si propagano in fretta; le cattive notizie (costi maggiori/link caduti) si propagano lentamente.**
Contromisure:

- **Numero massimo di hop** (es. 15 in RIP, con 16 = "infinito"): limita il tempo di convergenza nel caso peggiore.
- **Split horizon**: un nodo non include mai, negli annunci verso un vicino v, le rotte che ha imparato **da** v stesso.
- **Poisoned reverse**: variante più esplicita – se un nodo x raggiunge z passando per y, comunica comunque a y un costo di ∞ verso z (invece di ometterlo, glielo dichiara esplicitamente irraggiungibile tramite lui).

Limite importante: split horizon e poisoned reverse eliminano i loop tra **due** nodi, ma **non** garantiscono l'assenza di loop più lunghi (3 o più nodi): in tali casi il conteggio lento può comunque verificarsi per diversi cicli prima di convergere al valore corretto.

5.5.5 RIP – Routing Information Protocol

Protocollo intra-AS **distance vector**: semplice da implementare e gestire, ma con **convergenza lenta** e limiti sulla dimensione della rete gestibile. Incluso in UNIX BSD dal 1982. Il costo di un link è il **numero di hop** (massimo 15, con 16 = ∞). Ogni 30

secondi (o quando cambia la tabella locale), un router invia un "RIP advertisement" (fino a 25 sottoreti di destinazione + relativa distanza) usando **UDP porta 520**, indirizzo multicast `224.0.0.9`.

Se un router non riceve notizie da un vicino per **180 secondi**, considera il link/vicino guasto: aggiorna la propria tabella, propaga l'informazione (con poisoned reverse, distanza = 16, per evitare loop), e i vicini a loro volta propagano se le loro tabelle cambiano.

Nota implementativa: RIP gira come un normale **processo applicativo** (`routed`), che scambia messaggi su una socket UDP standard e poi scrive/legge la tabella di inoltro del sistema operativo – a differenza di OSPF, che opera direttamente sopra IP.

5.5.6 Link state vs. distance vector: confronto

	Link state	**Distance vector**
Complessità messaggi	$O(nE)$ messaggi totali (con n nodi, E link)	Solo verso i vicini; tempo di convergenza variabile
Velocità di convergenza	Algoritmo $O(n^2)$; possibili oscillazioni (costi mal definiti)	Dipende; possibili cicli e count-to-infinity
Robustezza a guasti/errori	Ogni nodo calcola la propria tabella in modo indipendente (un errore locale resta locale)	La tabella di ogni nodo è usata anche dagli altri: un errore **si propaga** nella rete

5.5.7 BGP – Border Gateway Protocol

Manca un ultimo ingrediente: far "parlare" tra loro i router di bordo di AS diversi. **BGP** (RFC 4271, v4 dal 2006) è di fatto l'**unico** protocollo standard per lo scambio di informazioni di raggiungibilità (prefissi) **tra** AS – oggi oltre 60.000 AS interconnessi nel mondo.

Relazioni tra AS. **Client-Provider** (il client paga il provider per la raggiungibilità); **Provider-Client** (simmetrico); **Peer** (due AS si scambiano liberamente le proprie informazioni di raggiungibilità, tipicamente senza pagamento). Le interconnessioni sono regolate da **contratti** commerciali, e ogni AS ha piena **libertà amministrativa**: può decidere se pubblicizzare le proprie reti, può ritirare in ogni momento la raggiungibilità di una propria rete, può decidere se offrire transito verso altri AS.

Meccanismo. Ogni AS ha router detti **BGP speaker**, che comunicano tramite connessioni **TCP**. BGP è un **Path Vector Protocol**: a differenza di un semplice distance vector, l'informazione condivisa include non solo il costo, ma l'intero **percorso** (lista di AS attraversati) per raggiungere un prefisso, es.: destinazione `14.8.0.0/16`, next hop AS15, percorso $\{15, 32, 571, 11\}$ – utile anche per rilevare ed evitare loop (un AS scarta una rotta che contiene già il proprio numero).

Il "best path" BGP non è il percorso più veloce

A differenza di OSPF/RIP (dove si preferisce il costo/percorso più breve), in BGP il concetto di percorso "preferito" è **vago** e dipende principalmente dalle **policy** di ciascun AS (es. un AS potrebbe preferire un percorso più lungo perché ha un accordo commerciale più vantaggioso, o perché lo ritiene più stabile). Ogni speaker BGP condivide con i vicini **solo** il proprio percorso preferito (*best path*): un percorso mai scelto come "migliore" da nessuno non verrà mai propagato oltre. **Il best path BGP è ottimo secondo le politiche commerciali e di *traffic engineering* degli operatori, non secondo metriche di performance di rete.**

I quattro messaggi BGP. **OPEN** (apre una connessione, negoziando un *hold time* – si usa il minore tra i due proposti – e scambiando il *BGP identifier*, seguito da un **KEEPALIVE** di conferma). **KEEPALIVE** (mantiene viva la connessione, inviato abbastanza spesso da non far scadere l'hold time; solo 19 Byte, nessun contenuto oltre l'header). **UPDATE** (veicola le informazioni di raggiungibilità: **additive**, nuovi percorsi; o tramite **WITHDRAW**, rimozione di percorsi non più validi – es. dopo la caduta di un link, l'informazione "non so più raggiungere questa rete" si propaga di AS in AS; un *implicit withdraw* evita di mandare un WITHDRAW seguito subito da un UPDATE, sostituendoli con un solo annuncio del nuovo percorso migliore). **NOTIFICATION** (segnala errori, con 6 codici e 20 sottocodici).

Le tre tabelle di BGP (RIB). **ADJ_RIB_IN** (rotte accettate in ingresso, usate per valutare percorsi alternativi) → **Routing Table** (contiene solo i *best path* attuali) → **ADJ_RIB_OUT** (rotte che hanno superato i filtri in uscita, da condividere). Un UPDATE in arrivo passa prima dai **filtri di ingresso**; se accettato, aggiorna ADJ_-RIB_IN; si ricalcola il best path; se questo cambia, si aggiornano Routing Table e ADJ_-RIB_OUT, e il cambiamento si inoltra ai vicini (dopo i **filtri di uscita**). I filtri possono essere generici ("scarta tutto ciò che contiene AS129") o molto specifici (fino al singolo campo di un pacchetto).

Caso di studio: il disservizio Facebook del 4 ottobre 2021

Un comando di routine per controllare lo stato della dorsale di trasporto dati interna di Facebook innescò, per un **bug**, un'ondata di messaggi **WITHDRAW** che ritirarono le rotte BGP di tutti i prefissi di Facebook (rilevata da Cloudflare come un picco di aggiornamenti BGP alle 15:40 UTC). I server DNS di Facebook, non più raggiungibili, smisero a loro volta di rispondere – rendendo Facebook, WhatsApp, Instagram e Messenger irraggiungibili per il resto del mondo. La ripresa fu complicata dal fatto che gli stessi strumenti diagnostici e l'accesso remoto ai sistemi dipendevano dalla rete ora isolata: non era possibile semplicemente "riavviare tutto". Il servizio tornò online alle 21:28 UTC. **Lezione**: un errore di configurazione/software a livello di un singolo protocollo (BGP) può isolare completamente un intero operatore da Internet, indipendentemente da quanto siano ridondanti i suoi server applicativi.

Aggregazione delle rotte: per risparmiare spazio nei messaggi, prefissi contigui possono essere aggregati in un annuncio più corto – a costo di una minore precisione

sul percorso esatto (es. `104.0.0.0/16` annunciato invece dei due /17 sottostanti, con l'informazione di AS-path che diventa una combinazione dei percorsi originali).

5.5.8 Network neutrality

Chiusura riflessiva del capitolo: quanto siamo davvero liberi quando navighiamo, e quanto siamo invece condizionati dalle decisioni (tecniche e soprattutto **commerciali**) prese da altri lungo il percorso? Un ISP potrebbe, in linea di principio, discriminare il traffico che non gli porta un guadagno diretto – il principio di **network neutrality** sostiene che ogni sito debba essere trattato allo stesso modo, senza corsie preferenziali basate su accordi economici. È un tema attivamente dibattuto e regolamentato in modo diverso da Paese a Paese.

5.6 Riepilogo completo del Capitolo 4-5 (livello di rete)

- **Piano dati**: architettura dei router (data plane hardware vs. control plane software), formato e frammentazione IPv4, indirizzamento CIDR con longest-prefix-matching, NAT, indirizzi speciali, ARP, ICMP, DHCP, IPv6.
- **Piano di controllo**: come si costruiscono le tabelle di inoltro tramite algoritmi di routing – **link state/Dijkstra** (informazione globale, usato da OSPF all'interno di un AS) e **distance vector/Bellman-Ford** (informazione locale/iterativa, usato da RIP), con le rispettive patologie (oscillazione, count-to-infinity) e contromisure.
- **Instradamento tra AS**: **BGP**, un *path vector protocol* **policy-driven** (non orientato alle prestazioni), che tiene insieme l'intera Internet come rete di reti amministrativamente indipendenti.

Prossimo argomento: dal "cuore" della rete (IP, routing) si torna verso il livello immediatamente sottostante – il **livello di collegamento** (*data link*): Ethernet, WiFi, switch, ARP visto dal basso (Capitolo 6).

Appendice A – Banca domande d'esame (con soluzioni ragionate)

Domande fornite dal docente in preparazione all'esame (a.a. 2020-2021). Le prime 15 erano a risposta singola, dalla 16 in poi a risposta multipla. Riporto la risposta corretta dalla chiave ufficiale, più il ragionamento e il rimando alla sezione di questi appunti.

A.1 – Concetti generali e prestazioni (Cap. 1)

1. **Quale di queste è una corretta definizione di throughput?**
   Risposta: pacchetti ricevuti correttamente *al secondo*.
   Ragionamento: il throughput è sempre una **quantità di lavoro per unità di tempo**. "Pacchetti ricevuti correttamente" senza il tempo è un conteggio, non un throughput. (§1.4)

2. **Perché la commutazione di pacchetto è "meglio" di quella di circuito?**
   Risposta: perché permette a più utenti di condividere efficientemente le stesse risorse (e perché non necessita la prenotazione delle risorse).
   Ragionamento: è la commutazione di *circuito* che riserva risorse; la commutazione di pacchetto **non evita affatto le congestioni** – anzi, le rende possibili. (§1.3.2)

3. **Cos'è un cavo Ethernet S/FTP?**
   Risposta: un cavo con una schermatura per ciascun doppino *e* una per il cavo intero.
   Ragionamento: nella notazione X/YTP, **X** è la schermatura dell'intero cavo (S = maglia metallica intrecciata) e **Y** quella del singolo doppino (F = foiled/shielded). (§1.2.3)

4. **Cos'è il prodotto banda-ritardo?**
   Risposta: il prodotto tra tasso di trasmissione [bit/s] e ritardo di propagazione [s]; se vengono trasmessi abbastanza bit, rappresenta il numero di bit contemporaneamente presenti sul collegamento.
   Ragionamento: entrambe le formulazioni sono la stessa grandezza vista da due angolazioni – è la "capacità del tubo" in bit. **Attenzione ai distrattori**: non c'entra il ritardo di accodamento né la velocità di propagazione in m/s. (§1.4.6)

5. **Detti L = lunghezza del pacchetto [bit], R = tasso di trasmissione [bit/s], d = lunghezza del link [m], c = velocità di propagazione [m/s]: in che condizioni i primi bit vengono ricevuti prima che la trasmissione sia completata?**
   Risposta: $L/R > d/c$.
   Ragionamento: L/R è il tempo di **trasmissione**, d/c quello di **propagazione**. Se il primo è maggiore del secondo, il primo bit ha già raggiunto la destinazione mentre il mittente sta ancora spingendo fuori gli ultimi. È esattamente l'analogia del casello autostradale a 1000 km/h. (§1.4.1)

6. **Quali vantaggi presenta un'architettura a livelli?**
   Risposte: (a) rende chiare le interazioni tra i componenti, perché i livelli sono interconnessi sequenzialmente; (b) facilita il progetto, perché si può sostituire l'implementazione di un livello senza cambiare gli altri.
   *Ragionamento su cosa **non** è vero*: la stratificazione non riduce la duplicazione di funzionalità – anzi, la crea (es. controllo d'errore sia a livello 2 sia a livello 4); e non garantisce affatto che ogni layer abbia tutte le informazioni che gli servono (è proprio il limite che motiva le ottimizzazioni cross-layer). (§1.5.1)

7. **Cosa si intende per incapsulamento?**
   Risposta: il processo per cui una data unità dati viene racchiusa tra un header (e opzionalmente un trailer) di un altro protocollo, *indipendentemente dal suo contenuto*.
   Ragionamento: l'"indipendentemente dal contenuto" è il punto: ogni layer tratta la PDU superiore come una **busta chiusa** che non apre. Il distrattore "ma solo se non contiene già header di altri protocolli" nega proprio il meccanismo dell'incapsulamento annidato. (§1.5.4)

A.2 – Livello applicazione (Cap. 2)

8. **Per quale ragione un utente non usa SMTP per *leggere* la propria posta?**
   Risposta: perché il server dell'utente dovrebbe essere sempre attivo per ricevere i messaggi inviati.
   Ragionamento: SMTP è un protocollo **push**: consegna al server del destinatario. Per *prelevare* serve un protocollo pull (POP3/IMAP/HTTP). Senza di essi, l'utente dovrebbe ospitare lui stesso un server SMTP sempre acceso. (§2.4.3)

9. **Che differenza c'è tra una query DNS ricorsiva e una iterativa?**
   Risposta: nella ricorsiva l'host delega il server DNS locale, che a catena delega tutti i server della gerarchia fino a quello autoritativo (equivalentemente: "la ricorsiva delega il recupero a un server DNS locale").
   Ragionamento: il distrattore pericoloso è quello che descrive l'host che interroga *uno per uno* tutti i server – quella è la **iterativa**, non la ricorsiva. (§2.5.1)

10. **Perché una cache web è utile?**
    Risposte: (a) perché alleggerisce ("scarica") il link di accesso a Internet della LAN fornendo localmente i contenuti statici; (b) perché controlla automaticamente se una pagina è cambiata e, se non lo è, fornisce la copia locale.
    Ragionamento: il punto (b) è il **GET condizionale** con `If-modified-since`. Falso invece che la cache ospiti *tutte* le componenti di una pagina, o che usi un link separato. (§2.2.6)

11. **Cos'è la codifica base64?**
    Risposte: (a) un algoritmo per rappresentare gruppi di 6 bit usando 8 bit, in modo che gli 8 bit risultanti abbiano il primo bit a 0; (b) ... in modo che rappresentino lettere maiuscole, minuscole, numeri e pochi caratteri speciali.
    Ragionamento: entrambe descrivono lo stesso fatto – il risultato sta nei 128 caratteri ASCII stampabili, quindi il bit più significativo è sempre 0. Non è compressione: al contrario, **espande** i dati di circa il 33%. (§2.4.2)

12. **Perché la distribuzione di file peer-to-peer può essere conveniente?**
    Risposte: (a) perché non richiede al server di inviare N copie, una per host; (b) perché più host collaborano all'upload; (c) perché risolve il collo di bottiglia sul link di *upload* del server.
    Ragionamento: falso che risolva il collo di bottiglia sul **download** dei client – il termine $F/\min_i(d_i)$ resta identico nelle due formule. Il P2P scala perché ogni nuovo peer porta anche nuova capacità di upload. (§2.6.1)

13. **Quale strategia di distribuzione dei server arriva più vicina all'utente finale?**
    Risposta: enter-deep (la chiave ufficiale indica anche *bring-home*).
    Ragionamento: **enter-deep** (Akamai) installa i server *dentro* le reti di accesso degli ISP; **bring-home** (Limelight) si ferma agli IXP, quindi un gradino più lontano. Se la domanda chiede la *più* vicina in senso stretto, la risposta è enter-deep. Segnalo che la chiave le dà entrambe: verifica con il docente. (§2.7.2)

14. **Cosa si intende per query flooding?**
    Risposta: il meccanismo con cui un peer inoltra una query a tutti i propri

vicini nella rete di copertura (overlay).
Ragionamento: è il modello Gnutella, completamente distribuito, con raggio limitato da un TTL per motivi di scalabilità. Non c'entra con BitTorrent (che usa un tracker) né con il parallelismo del browser. (§2.6.3)

A.3 – Livello di trasporto (Cap. 3)

15. **Quali funzioni sono implementate dal livello di trasporto?**
Risposte: trasferimento dati senza errori e in ordine su un collegamento end-to-end; trasferimento di dati segmentati in ordine; controllo di flusso; *e comunque dipende dal protocollo usato.*
Ragionamento: l'ultima opzione non è una scappatoia, è **la risposta più precisa**: UDP non offre nessuna di queste funzioni. Non sono di livello 4: il multicast, la risoluzione MAC↔IP (livello 2/3), la scoperta del percorso ottimo (livello 3). (§3.1)

16. **Cosa comportano il multiplexing e il demultiplexing al livello di trasporto?**
Risposte: tutte e quattro. Al trasmettitore: invio di segmenti da socket diverse sullo stesso link; al ricevitore: separazione e consegna alla socket giusta. Inserimento di PCI (le porte) per l'indirizzamento. In TCP: stessa socket per i segmenti con la stessa **quadrupla**. In UDP: stessa socket per i segmenti con la stessa **coppia** (IP dest., porta dest.).
Ragionamento: la differenza 4-tupla vs. 2-tupla è la cosa che viene chiesta più spesso. (§3.2)

17. **In UDP, il checksum...**
Risposte: (a) consente il controllo di correttezza sia dell'header sia del payload; (b) è calcolato in modo che, sommando tutte le parole di 16 bit *checksum incluso*, si ottenga una parola di soli 1.
Ragionamento: la proprietà "tutti 1" vale per costruzione, perché il checksum è il **complemento a 1** della somma. Nota: il checksum **IP** copre invece il solo header – non confonderli. (§3.3.1)

18. **Perché il pipelining migliora l'utilizzo del link?**
Risposte: (a) perché consente di inviare più pacchetti prima di riceverne gli ACK; (b) perché non si resta in silenzio ad attendere un ACK per ogni pacchetto.
Ragionamento: sono due formulazioni dello stesso fatto. Il pipelining non ha niente a che vedere con la saturazione dei buffer (quello è il **controllo di flusso**). Ricorda l'esercizio: Stop-and-Wait su 1 Gbit/s → efficienza dello 0,027%. (§3.4.2)

19. **Come fa un host A a chiudere *gentilmente* una connessione TCP con B?**
Risposta: A invia FIN; B risponde con FIN+ACK; B invia ad A gli eventuali dati rimasti; B invia FIN; A risponde con FIN+ACK. (La chiave della versione a risposta multipla accetta anche la variante senza dati residui e la chiusura brusca con RST.)
Ragionamento: la chiusura è **bidirezionale** – va chiusa in entrambe le direzioni. Il distrattore che si ferma a due messaggi descrive solo una connessione **semi-chiusa** (*half-closed*), non chiusa. (§3.5.5)

20. **Quali casi rappresentano una conseguenza della congestione?**
Risposte: (a) con buffer infinito, ritardo elevatissimo avvicinandosi al throughput massimo; (b) in situazioni tipiche, scarto di pacchetti per code finite, più lavoro per consegnare i segmenti, throughput ridotto; (c) con buffer finito, ritardi elevati che causano timeout prematuri e ritrasmissioni inutili di pacchetti già consegnati.
Ragionamento: sono esattamente gli scenari 1, 2 e 3 del libro. Falso che connessioni diverse sullo stesso link osservino *esattamente* lo stesso throughput: è l'obiettivo dell'AIMD, non una garanzia. (§3.6)

A.4 – Livello data link (Cap. 6)

21. **Che differenza c'è tra CRC e checksum?**
Risposta: il CRC richiede la divisione per un numero binario, il checksum si basa solo su somme binarie.
Ragionamento: nessuno dei due corregge gli errori, entrambi li *rilevano* soltanto. E le dimensioni delle parole (16/32 bit) non sono la differenza concettuale. (§6.2.3)

22. **Quale caratteristica ha un protocollo MAC CSMA p-persistente (con $p < 1$)?**
Risposte: (a) se trova il canale occupato, attende finché non si libera, poi con probabilità p trasmette e con probabilità $1-p$ va in backoff; (b) se trova il canale vuoto, trasmette immediatamente.
Ragionamento: attenzione a distinguerlo dal **1-persistente** (canale occupato → trasmette *subito* appena si libera) e dal **non-persistente** (canale occupato → attende un tempo lungo e rifà il sensing). (§6.3.4)

23. **Il backward learning è...**
Risposta: l'algoritmo con cui gli switch imparano su quale porta è raggiungibile un certo host / indirizzo di livello 2.
Ragionamento: il distrattore più insidioso descrive il **flooding** (inoltro su tutte le porte tranne quella di arrivo), che è la procedura usata *quando il backward learning non ha ancora appreso* – non il backward learning stesso. Né va confuso con lo **STP** (eliminazione dei link ridondanti). (§6.5.3)

24. **Nella tabella di uno switch, l'associazione MAC → porta è...**
Risposta: molti a uno (molti MAC, una porta).
Ragionamento: dietro una singola porta può esserci un hub, o un altro switch, e quindi **molti host**. Il viceversa è impossibile: lo stesso MAC non può stare su due porte contemporaneamente (se succede, è un loop). (§6.5.3)

25. **Perché organizzare una LAN fisica in più VLAN?**
Risposte: (a) per segmentare i domini di broadcast; (b) per proteggere host sensibili il cui traffico non dovrebbe essere inoltrato ad altre LAN; (c) per isolare aree sperimentali della rete.
Ragionamento: falso che servano a gestire automaticamente gli algoritmi di routing (sono di livello 2, il routing è di livello 3) o che dipendano da un limite di host di IP. (§6.5.5)

Nota di metodo: la chiave ufficiale dà solo le lettere. Il ragionamento e i rimandi qui sopra sono ricostruiti dal contenuto delle lezioni: se una spiegazione non ti torna, il punto da rileggere è la sezione indicata, non la risposta.

Appendice B – Esercizi d'esame svolti

Leggi prima questo

Dalle testimonianze degli orali, i tre tipi di esercizio pratico che vengono chiesti sono, in ordine di frequenza: **(1) Dijkstra** (∼42% dei casi), **(2) subnetting** (∼29%), **(3) timeline TCP con perdite** (∼25%). Praticamente nient'altro. Questa appendice li copre tutti e tre con soluzioni complete.

B.1 – Subnetting: il metodo

Procedura in 5 passi

1. **Da host a bit**: per ospitare H host servono $H+2$ indirizzi (rete + broadcast non assegnabili), quindi b bit di HostID con $2^b \geq H+2$.
2. **Maschera**: prefisso $= 32 - b$.
3. **"Numero magico"**: la dimensione del blocco è 2^b. I net-ID delle sottoreti successive si ottengono **sommando** 2^b all'ottetto giusto.
4. **Ordina dalla più grande alla più piccola** se le sottoreti hanno dimensioni diverse (VLSM), così eviti buchi.
5. **Per ogni sottorete indica sempre**: net-ID, subnet mask, range di indirizzi assegnabili, numero di host. Sono le quattro cose che il testo chiede esplicitamente.

Errore classico: per 8 host non basta una `/29` (8 indirizzi totali, ma solo **6** assegnabili). Serve una `/28`.

Esercizio B.1 (chiesto all'orale)

Dato il pool pubblico `131.25.197.192/26`, quanti host si possono indirizzare? Suddividerlo a maschera fissa in 4 LAN che devono ospitare rispettivamente 14, 14, 8 e 8 host.

Soluzione.

Capienza del pool: `/26` $\Rightarrow 32 - 26 = 6$ bit di HostID $\Rightarrow 2^6 = 64$ indirizzi, di cui **62 assegnabili**. Range: `131.25.197.192` – `131.25.197.255`.

Dimensionamento: la LAN più grande chiede 14 host $\Rightarrow$ servono $14 + 2 = 16$ indirizzi $\Rightarrow 2^4$, cioè **4 bit di HostID**. A maschera fissa, tutte e quattro le LAN sono `/28` (prendo 2 bit in più rispetto a `/26`: $2^2 = 4$ sottoreti, esattamente quante me ne servono). Numero magico: **16**.

LAN	Net-ID	Subnet mask	Range assegnabile	Host
LAN1	131.25.197.192/28	255.255.255.240	.193 – .206	14
LAN2	131.25.197.208/28	255.255.255.240	.209 – .222	14
LAN3	131.25.197.224/28	255.255.255.240	.225 – .238	14
LAN4	131.25.197.240/28	255.255.255.240	.241 – .254	14

Osservazione da dire all'esame: qui la maschera fissa **non spreca nulla**, perché anche con VLSM le due LAN da 8 host richiederebbero comunque una `/28` (una `/29` darebbe solo 6 indirizzi assegnabili < 8). Farlo notare dimostra che hai capito il vincolo, non che hai applicato una ricetta.

B.2 – Longest prefix matching

Esercizio B.2 – Tabella di inoltro

Data la tabella, determinare l'interfaccia di uscita per cinque destinazioni.

Rete	Subnet mask	Interfaccia
128.96.39.0	255.255.255.128 (/25)	Interface 0
128.96.39.128	255.255.255.128 (/25)	Interface 1
128.96.40.0	255.255.255.128 (/25)	Interface 2
192.4.153.0	255.255.255.192 (/26)	Interface 3
0.0.0.0	0.0.0.0 (/0)	Interface 4 (default)

Metodo: per ogni riga, calcolare `destinazione & maschera` e confrontare col net-ID. Vince il **prefisso più lungo** tra quelli che corrispondono.

Destinazione	AND con la maschera	Risultato	Uscita
128.96.39.10	& 255.255.255.128	128.96.39.0	**Interface 0**
128.96.40.12	& 255.255.255.128	128.96.40.0	**Interface 2**
128.96.40.151	nessun match	0.0.0.0	**Interface 4**
192.4.153.17	& 255.255.255.192	192.4.153.0	**Interface 3**
192.4.153.90	nessun match	0.0.0.0	**Interface 4**

I due casi che fregano: `128.96.40.151` ha l'ottavo bit dell'ultimo ottetto a 1 ($151 \geq 128$), quindi cade nella seconda metà del `/25` – che **nessuno ha annunciato** $\rightarrow$ default. Stesso ragionamento per `192.4.153.90`: con maschera `/26` (blocchi da 64) cade in `192.4.153.64`, non in `.0`.
Nota sull'implementazione reale: un router non prova tutte le righe in parallelo – parte dalla **maschera più lunga** e scende; al primo match inoltra. Arrivando alla `/0` trova sempre il default gateway.

B.3 – Configurazione di rete completa + Dijkstra

Esercizio B.3 – "Company" (traccia d'esame Lo Cigno)

Azienda con due sedi S1 ed S2, ciascuna con una LAN pubblica e una WLAN privata, connesse a router R1 ed R2. L'ISP ha `172.172.0.0/16`; l'azienda ha `130.192.64.0/23` per le due reti pubbliche; le reti private sono tutte `/21` con indirizzi `192.168.x.x`. I router usano OSPF. Costi dei link: R1–R3 = 1, R2–R5 = 1, R3–R4 = 5, R3–R9 = 2, R9–R7 = 1, R9–R8 = 5, R7–R8 = 2, R4–R8 = 2, R4–R5 = 1, R5–R6 = 1, R6–R8 = 1.

Punto 1 – LAN pubbliche. Il pool `/23` (512 indirizzi) va diviso in **due parti uguali** (stesso numero di postazioni nelle due sedi), quindi due `/24`:

	Net-ID	Subnet mask	Range / host	
LAN1	130.192.64.0/24	255.255.255.0	.1 – .254	(254 host)
LAN2	130.192.65.0/24	255.255.255.0	.1 – .254	(254 host)

Punto 2 – WLAN private. Vincolo dato: `/21` $\Rightarrow 2^{11} = 2048$ indirizzi, **2046 utenti** per sottorete. Con `192.168.x.x`, il numero magico su `/21` è 8 nel terzo ottetto:

	Net-ID	Subnet mask	Range / utenti	
WLAN1	192.168.0.0/21	255.255.248.0	192.168.0.1 – 192.168.7.254	(2046)
WLAN2	192.168.8.0/21	255.255.248.0	192.168.8.1 – 192.168.15.254	(2046)

Punto 3 – Interfacce e tabella di R1. Il link R1–R3 è una `/30`: dato R3 eth0 = `172.172.0.2`, la sottorete è `172.172.0.0/30` (indirizzi: `.0` rete, `.1`, `.2`, `.3` broadcast), quindi **R1 eth0 = 172.172.0.1**.

Interfaccia	Indirizzo	Rete
eth0 (verso R3)	172.172.0.1/30	172.172.0.0/30
eth1 (LAN1)	130.192.64.1/24	130.192.64.0/24
eth2 (WLAN1)	192.168.0.1/21	192.168.0.0/21

Destinazione	Maschera	Interfaccia	Next hop
130.192.64.0	/24	eth1	diretto
192.168.0.0	/21	eth2	diretto
172.172.0.0	/30	eth0	diretto
0.0.0.0	/0	eth0	172.172.0.2 (R3)

Punto 4 – Serve il NAT? Dipende dalle sottoreti. Per far comunicare **tra loro** le quattro sottoreti aziendali **no**: basta che i router annuncino le rotte, perché un indirizzo privato non è instradabile su Internet ma **dentro una intranet sì**. Serve invece il NAT (su R1 e R2) perché gli host delle **WLAN private** raggiungano **Internet**: i loro indirizzi `192.168.x.x` vengono bloccati dai router pubblici. Le LAN pubbliche non ne hanno bisogno.

Punto 5 – Dijkstra da R1.

Passo	N'	R3	R9	R7	R4	R8	R5	R6	R2
0	R1	**1,R1**	∞	∞	∞	∞	∞	∞	∞
1	R1,R3	–	**3,R3**	∞	6,R3	∞	∞	∞	∞
2	+R9	–	–	**4,R9**	6,R3	8,R9	∞	∞	∞

B.4 – Timeline TCP con perdite

Regola d'oro per questo esercizio

Esistono **più algoritmi** TCP per gestire le perdite. **All'esame dichiara sempre quale stai usando** (tipicamente Fast Recovery, la macchina a stati del Kurose), a meno che il testo non lo specifichi. La soluzione ufficiale del docente lo scrive esplicitamente.

Esercizio B.4 (esame 16 giugno 2015, soluzione ufficiale)

Trasferimento HTTP di un file da 16300 Byte più 300 Byte di comando PUT, su link a 4,8 Mbit/s con $RTT = 2$ ms e $RTO = 120$ ms. MTU Ethernet.
(a) MSS. $1500 - 20$ (IP) $- 20$ (TCP) $=$ **1460** Byte.
(b) Porte. Destinazione: **80** (HTTP). Sorgente: una porta effimera > 1023. Segue il three-way handshake: SYN $\rightarrow$ SYN+ACK $\rightarrow$ ACK.
(c) Numero di segmenti. Da inviare: $16300 + 300 = 16600$ Byte.

$$\left\lfloor \frac{16600}{1460} \right\rfloor = 11 \text{ segmenti da 1460 Byte} \quad + \quad 1 \text{ segmento da } 16600 - 1460 \cdot 11 = \mathbf{540} \text{ Byte}$$

Totale: **12 segmenti**.
(d) Durata senza perdite. Tempi di trasmissione: $1500\,\text{B}/4{,}8\,\text{Mbit/s} = 2{,}5$ ms per i frame pieni; $580\,\text{B}/4{,}8\,\text{Mbit/s} \approx 1$ ms per l'ultimo. ACK e FIN si considerano di dimensione trascurabile.

1 RTT apertura connessione	2 ms
1 RTT attesa ACK del primo segmento	2 ms
11 frame da 1500 B + 1 da 580 B	28,5 ms
1 RTT attesa ACK ultimo segmento	2 ms
1,5 RTT chiusura	3 ms
Totale	**37,5 ms**

(e) Con perdita dei segmenti 4 e 9. Questo è il cuore dell'esercizio – segui l'evoluzione di CWND:

1. Slow start: CWND $= 1, 2, 3, 4$. Quando CWND $= 4$ il client invia i segmenti **4 (perso)**, 5, 6, 7.
2. I segmenti 5, 6, 7 arrivano fuori sequenza $\rightarrow$ il server genera **3 ACK duplicati** (tutti "A3").
3. Al 3º ACK duplicato scatta **Fast Retransmit/Recovery**: SSTHRESH = CWND/2 = 2; CWND = SSTHRESH +3 = 5; il segmento 4 viene **ritrasmesso subito** (e la finestra consente di inviare anche l'8).
4. Arriva l'ACK cumulativo A7 (il server conferma tutto fino al 7): **fine del recovery**, CWND = SSTHRESH = 2, si entra in **congestion avoidance**.
5. Il client invia 9 (**perso**) e 10. Il 10 genera **un solo** ACK duplicato (A8): **non bastano 3**, quindi niente fast retransmit.
6. La perdita viene rilevata solo allo **scadere dell'RTO**: SSTHRESH = 1, CWND = 1, si riparte da **slow start**.
7. Ritrasmesso il 9, arriva A10, CWND = 2, si inviano gli ultimi due segmenti.

1 RTT apertura + 1 RTT primo ACK	4 ms
13 frame da 1500 B (2 ritrasmissioni) + 1 da 580 B	33,5 ms
1 RTT per la perdita del seg. 4 (attesa 3º dupACK)	2 ms
1 RTO per la perdita del seg. 9	**120 ms**
1 RTT attesa ACK dopo ritrasmissione del 9	2 ms

B.5 – Progetto di un protocollo (traccia aperta)

Appare negli scritti come seconda domanda. Non si chiede un progetto reale, ma di **dimostrare che sai cos'è un protocollo**. Schema di risposta, su un esempio (protocollo per sensori remoti, su UDP porta 5555, pacchetti IP $\leq$ 128 Byte):

- **Calcola subito il budget**: $128 - 20$ (IP) $- 8$ (UDP) $=$ **100** Byte per il messaggio applicativo.
- **Giustifica la scelta di formato**: con un budget così stretto servono **campi a dimensione fissa**, non un protocollo testuale tipo HTTP (inefficiente e a rischio di non stare in un solo datagramma).
- **Definisci i campi obbligatori**: `Protocol Version`; `Transaction Id` (identifica la richiesta ed è ricopiato nella risposta, per non confondersi con risposte ritardate o pacchetti persi); `Message Type` (include se è richiesta o risposta, es. nel bit più significativo); `CRC` (il checksum UDP è **opzionale**, quindi un controllo applicativo è utile); `Payload Length`; `Payload`.
- **Definisci un sottoinsieme di comandi**, non tutti: richiesta/risposta per `reboot`, `reset` di un sensore, elenco sensori, ultima misura, ultime n misure, più i codici d'errore.
- **Segnala i casi limite**: cosa succede se una risposta (es. n misure) non sta in un pacchetto? Si definisce un messaggio di continuazione, oppure si delega all'applicazione.

Appendice C – Cosa viene chiesto davvero all'orale

Frequenze ricavate da 24 testimonianze di studenti. L'orale dura 10–15 minuti: tipicamente **2 domande di teoria + 1 esercizio**.

C.1 – Esercizio pratico

Tipo	Frequenza	Dove ripassarlo
Dijkstra (+ albero di copertura)	∼42%	§B.3, §5
Subnetting / suddivisione pool IP	∼29%	§B.1, §B.3
TCP: slow start, fast recovery, congestione	∼25%	§B.4, §3.7
Distance vector (Bellman-Ford)	∼8%	§5

Dettaglio utile: più d'uno riporta che il docente lo ha **fermato dopo due righe** di Dijkstra. Non serve finire l'esercizio, serve impostarlo correttamente. E almeno uno è stato messo in difficoltà da un pool IP **volutamente troppo piccolo** per gli host richiesti: se i conti non tornano, dillo – è il punto della domanda.

C.2 – Teoria, per frequenza

Molto frequenti **CSMA** in tutte le sue varianti (1-persistente, p-persistente, CSMA/CD vs CSMA/CA, CSMA di 802.11, terminale nascosto e come risolverlo) – §6.3.4, §6.6.5; **cookie** e **cache web** – §2.2.5–6.

Frequenti **ping / traceroute / ICMP** – §4 (ICMP); **frammentazione** e header IP con TTL – §4; **incapsulamento** – §1.5.4; **switch**: hub vs switch, backward learning, STP – §6.5; **controllo di flusso vs controllo di congestione** (la differenza, non le definizioni separate) – §3.5.7 e §3.6.

Ricorrenti **NAT** – §4; **link state vs distance vector** – §5; **ARQ** (stop-and-wait vs pipelining, Go-Back-N vs Selective Repeat) – §3.4; **ARP** – §4; **DNS** – §2.5; **CIDR** classful vs classless + **cosa succede se il router non sa dove inoltrare** (risposta: `0.0.0.0/0`) – §4; **IPv4 vs IPv6** e **tunneling** – §4; **AIMD** – §3.7.1; **VLAN** – §6.5.5; **BGP** – §5; **SMTP** – §2.4; **stratificazione**: che senso ha dividere in livelli e come comunicano – §1.5.

Due pattern da conoscere

1. Le domande sono collegate tra loro. Esempi reali: "CSMA/CD *e poi* cos'è l'incapsulamento *e poi* come funziona la transizione IPv4→IPv6"; "CIDR *e poi* cosa succede se il router non sa dove inviare un pacchetto". Prepara le **connessioni**, non le definizioni isolate.

2. Il docente riprende gli errori dello scritto. Almeno due testimonianze riportano di essersi visti chiedere all'orale esattamente l'argomento sbagliato allo scritto (cookie, CSMA p-persistente). **Rivedi il tuo scritto prima dell'orale**: è lì che sai già quali saranno le domande.

6 Appendice D: Esempio di Prova d'Esame Completa con Soluzioni (AA 2018/19)

Prova d'esame completa con soluzioni ufficiali (fonte: *Esame-con-tracce-di-soluzione.pdf*). Utile per capire il livello di dettaglio atteso nelle risposte scritte.

Esercizio 1 (Configurazione di una rete locale) — AA 2018/19

Un'università deve configurare la rete di un nuovo edificio contenente aule didattiche, laboratori, sala server e uffici. La rete permette ai computer presenti nell'edificio di comunicare tra di loro e con Internet. L'Università ha acquistato un link cablato a 10 Gbps più un link di backup wireless a 1 Gbps per garantire la connettività ad Internet anche in caso di guasto del link cablato. Il link wireless ha un costo di utilizzo molto alto, quindi non deve essere utilizzato quando il link cablato è attivo.

1. Supponendo che entrambi i router R1 ed R2 notifichino agli altri router la loro capacità di raggiungere Internet, come posso assicurare che il link wireless venga usato solo in caso di necessità?

2. Configurare le sottoreti STUDENTI, LABORATORI e DIPENDENTI utilizzando pool di IP privati a scelta, sapendo che l'edificio può ospitare circa 1000 studenti, 120 computer per i laboratori e 40 dipendenti. Assegnare gli indirizzi IP alle interfacce dei router.

3. Configurare due sottoreti per la sala SERVER: una per i 20 server con servizi pubblici (web, mail, storage) e una per 40 macchine con IP privato usate come cluster di calcolo. Pool pubblico disponibile: `193.205.112.128/25`. È possibile collegare la singola interfaccia fisica `e3` di R4 a entrambe le sottoreti server? Perché?

4. Configurare le reti punto-a-punto fra i router, assegnando gli indirizzi IP alle interfacce (le interfacce di R1 ed R2 verso Internet sono già configurate dal provider).

5. Per far comunicare i dispositivi nelle sottoreti private (STUDENTI, LABORATORI, DIPENDENTI, cluster di calcolo) con i server ad IP pubblico, serve un meccanismo particolare?

Soluzione (ufficiale):

Punto 1. È sufficiente che R2 annunci un costo del link wireless tale per cui, indipendentemente dalla sottorete in cui si trova l'host, il costo per raggiungere Internet attraverso R2 sia sempre superiore a quello attraverso R1. Con una metrica hop-count (come OSPF o RIP), basta assegnare costo 4 al link wireless e costo 1 a tutti gli altri link: in questo modo anche un pacchetto generato direttamente da R2 preferisce transitare da R1 per uscire su Internet, finché il link cablato è disponibile. **Idea chiave**: il routing a stato di collegamento sceglie sempre il percorso di costo minimo, quindi "penalizzare" pesantemente il link di backup nella metrica è sufficiente a renderlo inutilizzato a meno che non sia l'unico cammino disponibile (link cablato guasto).

Punto 2. Pool privato scelto: `10.10.0.0/16`.

- **STUDENTI** (serve una sottorete da almeno 1024, meglio 2048 indirizzi): `10.10.128.0/21`
- **LABORATORI**: `10.10.192.0/24`
- **DIPENDENTI**: `10.10.160.0/24`

Le due sottoreti `/24` sono volutamente sovradimensionate rispetto al minimo stretto: con indirizzi privati, che sono una risorsa "rinnovabile" (non scarsa come i pubblici), è prassi non assegnare sottoreti più piccole del necessario se non per ragioni specifiche.

Indirizzi delle interfacce dei router verso queste sottoreti:

- R3 eth3: `10.10.192.1` (verso LABORATORI)
- R3 eth4: `10.10.128.1` (verso STUDENTI)
- R2 eth2: `10.10.160.1` (verso DIPENDENTI)

Punto 3. Bastano 32 indirizzi pubblici e 64 privati:

- **SERVER PUBBLICI**: `193.205.112.160/27`
- **SERVER PRIVATI** (cluster di calcolo): `10.10.161.0/26`

Anche qui si sarebbe potuto assegnare un `/24` al cluster privato, ma trattandosi di macchine quasi certamente a indirizzo statico in un sistema "chiuso", è altrettanto valida la scelta di assegnare il minimo indispensabile.

Sì, **è possibile** assegnare due indirizzi IP (uno per sottorete) alla stessa interfaccia fisica `e3`: il vincolo IP è che un indirizzo unicast appartenga ad una e una sola interfaccia, non il viceversa — un'interfaccia può avere più indirizzi. Non serve nessun accorgimento particolare a livello IP, anche se spesso si preferisce definire interfacce virtuali distinte sulla stessa interfaccia fisica a livello di sistema operativo.

Punto 4. Si assegnano sottoreti `/30` alle interconnessioni punto-a-punto, ad esempio:

Link	**Subnet**	**Indirizzi assegnati**
R1e0 – R3e0	`10.10.0.4/30`	R1e0: `10.10.0.5`, R3e0: `10.10.0.6`
R3e1 – R2e1	`10.10.0.8/30`	R3e1: `10.10.0.9`, R2e1: `10.10.0.10`
R3e2 – R4e2	`10.10.0.12/30`	R3e2: `10.10.0.13`, R4e2: `10.10.0.14`
R4e0 – R2e0	`10.10.0.16/30`	R4e0: `10.10.0.17`, R2e0: `10.10.0.18`

In alternativa, se le interconnessioni fra router sono realizzate su una rete broadcast (es. Ethernet commutata, plausibile in una rete di campus), si può assegnare un'unica sottorete a tutte le interfacce di interconnessione: in tal caso serve un `/28` per avere gli 8 indirizzi unicast necessari (un `/29` ne darebbe solo 6), ad esempio `10.10.0.0/28`.

Punto 5. No, non serve alcun meccanismo particolare: basta che i router annuncino correttamente le sottoreti via routing interno. Un indirizzo IP privato non può essere instradato su Internet, ma **all'interno di un'Intranet sì** — e qui tutta la comunicazione (sottoreti private verso i server, anche quelli con IP pubblico) avviene internamente alla rete dell'università, non attraversa mai Internet.

Esercizio 2 (Progetto di un Protocollo) — AA 2018/19

Si deve progettare un protocollo di livello applicativo per l'interrogazione e la configurazione di sensori remoti. Il protocollo si appoggia su UDP, con il server sulla porta 5555. Vincoli: i pacchetti IP non devono superare i 128 byte (header inclusi), e comandi/risposte devono sempre stare in un singolo datagramma UDP. Progettare un protocollo che supporti la lettura dei valori dei sensori e operazioni semplici (reset di una periferica, riavvio, ecc.).

Soluzione (ufficiale):

Dato il vincolo di dimensione, conviene un protocollo a **campi di dimensione fissa** piuttosto che testuale (stile HTTP): un protocollo basato su tag/testo sarebbe troppo inefficiente e rischierebbe di non stare in un solo datagramma.

Calcolo dello spazio disponibile: header IP (20 byte) + header UDP (8 byte) = 28 byte, quindi il payload applicativo massimo è $M = 128 - 28 = 100$ byte.

Architettura: protocollo **client/server transazionale** — il server gira sul sensore, il client (es. un PC) interroga in qualsiasi momento. Non serve preoccuparsi di indirizzi IP o porte client (effimere, assegnate dal SO).

Formato del messaggio (100 byte totali):

Campo	**Dim.**	**Scopo**
Protocol Version	4 bit	versione del protocollo
Transaction Id	12 bit	numero casuale identificativo della richiesta (evita confusione con pacchetti persi/risposte ritardate); copiato nella risposta; `0x000` e `0xFFF` riservati
Message Type	16 bit	tipo di operazione (richiesta/risposta codificata nel bit più significativo)
CRC	16 bit	ridondanza ciclica (stessa idea di TCP) per verificare l'integrità; utile perché il checksum UDP sul payload è opzionale
Payload Length	8 bit	basterebbero 7 bit, ma disallineare i byte sarebbe controproducente
Payload	0–93 byte	dati applicativi
Padding	0–93 byte	completa il messaggio a dimensione fissa (valore fisso `0xCC`)

Nota: si è assunta dimensione del pacchetto **fissa** (non solo "al massimo" 128 byte); è una scelta arbitraria ma permette di motivare il campo di padding. Un pacchetto a dimensione variabile senza padding sarebbe altrettanto corretto (e renderebbe superfluo anche Payload Length, dato che il payload terminerebbe alla fine del segmento UDP).

Codici di comando (Message Type, 2 byte meno significativi, i 2 più significativi a 0 = comandi standard):

`0000 0000`	riservato
`1111 1111`	riservato
`0xxx xxxx`	richieste
`1xxx xxxx`	risposte (stesso Transaction Id della richiesta)
`0000 0001 / 1000 0001 / 1000 0010`	reboot: richiesta / ack inizio reboot / reboot completato (richiede una EPROM per persistere il Tr. Id attraverso il riavvio)
`0000 0011 / 1000 0011`	reset di un sensore / esito del reset
`0001 0000 / 1001 0000`	richiesta elenco sensori / risposta con l'elenco
`0001 0001 / 1001 0001`	richiesta ultima misura di un sensore / risposta
`0001 0010 / 1001 0010`	richiesta ultime n misure / risposta (può non stare in un solo pacchetto se n è grande: da gestire con frammentazione applicativa)
`1111 1110`	comando non supportato
`1111 1101`	errore di sistema

7 Appendice E: Esercizio "Company" — Configurazione di Rete (traccia d'esame, soluzione proposta)

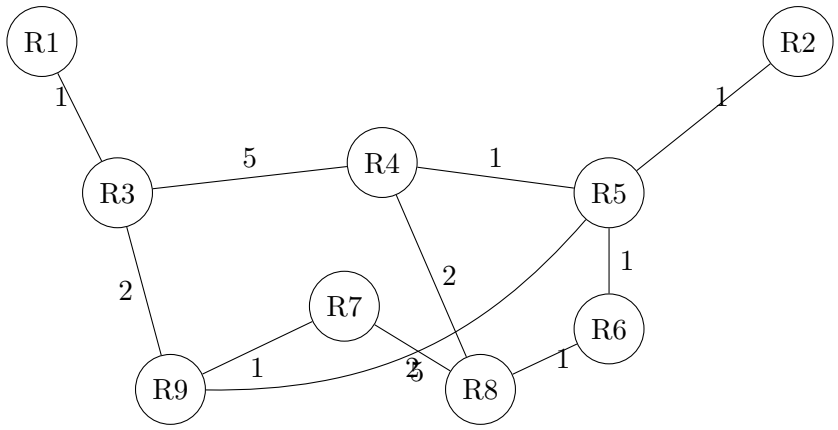
[Guessing/Likely — sezione senza soluzione ufficiale nel materiale] Questo esercizio (fonte: *08_Esercizi-20-12.pdf*) non include una soluzione nel materiale fornito. Quella che segue è una soluzione che ho costruito io applicando le stesse regole viste a lezione (subnetting, routing, Dijkstra, MST): utile per esercitarsi, ma non trattarla come una chiave ufficiale.

Esercizio: Configurazione di una Rete (Company)

Un'azienda "Company" ha due sedi S1 ed S2 in città diverse. In ciascuna sede la rete interna è divisa in due sottoreti connesse a un router (R1 in S1, R2 in S2): una pubblica (LAN1/LAN2, postazioni fisse) e una privata (WLAN1/WLAN2, wireless). L'azienda accede a Internet tramite un ISP con pool pubblico `172.172.0.0/16`. L'azienda dispone del pool pubblico `130.192.64.0/23` per le due reti pubbliche. Tutte le reti private sono `/21`, con indirizzi `192.168.x.x`. Tutti i router usano OSPF.

1. Assegnare gli indirizzi IP a LAN1 e LAN2 massimizzando il numero di postazioni fisse, con lo stesso numero in entrambe le sedi (net-id, mask, range, # host).
2. Assegnare gli indirizzi IP a WLAN1 e WLAN2 analogamente (net-id, mask, range, # utenti max).
3. Assegnare gli indirizzi alle interfacce di R1 e definirne la tabella di routing. L'interfaccia eth0 di R3 è `172.172.0.2/30`.
4. Serve NAT per far comunicare gli host delle quattro sottoreti tra loro?
5. Calcolare con Dijkstra i cammini minimi da R1 verso tutti gli altri router, mostrando l'evoluzione in tabella.
6. Disegnare il minimum spanning tree con radice in R1.

Topologia (dalla figura): R1 e R2 sono i router di accesso delle due sedi; la rete dell'ISP/backbone aziendale collega R3–R9.



Soluzione proposta:

Punto 1 — LAN1 e LAN2. Il pool `130.192.64.0/23` copre 512 indirizzi (`130.192.64.0`–`130.192.65.255`). L'unico modo di dividerlo in **due parti uguali** è in due blocchi `/24`:

- **LAN1**: net-id `130.192.64.0/24`, range utilizzabile `130.192.64.1 - 130.192.64.254`, 254 postazioni fisse.
- **LAN2**: net-id `130.192.65.0/24`, range utilizzabile `130.192.65.1 - 130.192.65.254`, 254 postazioni fisse.

Punto 2 — WLAN1 e WLAN2. Dimensione fissata dal testo a `/21` ($2^{11} = 2048$ indirizzi, 2046 utilizzabili):

- **WLAN1**: net-id `192.168.0.0/21`, range `192.168.0.1 - 192.168.7.254`, max 2046 utenti simultanei.
- **WLAN2**: net-id `192.168.8.0/21`, range `192.168.8.1 - 192.168.15.254`, max 2046 utenti simultanei.

Punto 3 — Interfacce e routing table di R1. R1 ha tre interfacce: eth0 verso R3 (backbone), eth1 verso LAN1, eth2 verso WLAN1.

Il testo fissa l'eth0 di R3 a `172.172.0.2/30`: la sottorete `/30` che la contiene è `172.172.0.0/30` (network `.0`, host utilizzabili `.1` e `.2`, broadcast `.3`). Quindi:

- R1 eth0: `172.172.0.1/30`
- R1 eth1: `130.192.64.1/24` (primo indirizzo utile di LAN1, gateway)
- R1 eth2: `192.168.0.1/21` (primo indirizzo utile di WLAN1, gateway)

Destinazione	**Next hop**	**Interfaccia**
`130.192.64.0/24` (LAN1)	diretta	eth1
`192.168.0.0/21` (WLAN1)	diretta	eth2
`172.172.0.0/30`	diretta	eth0
`0.0.0.0/0` (default: LAN2, WLAN2, Internet)	`172.172.0.2` (R3)	eth0

Nota: R1 è un router di accesso (foglia): una default route verso R3 è sufficiente, perché ogni destinazione non locale (compresa LAN2) passa comunque dal backbone. OSPF tra i router del backbone potrebbe in alternativa propagare una rotta specifica per `130.192.65.0/24`, ma non cambia la sostanza della risposta.

Punto 4 — Serve NAT? Dipende dalle sottoreti (opzione corretta tra le tre proposte):

- **WLAN1 ↔ LAN1** (stessa sede): **no NAT**. È solo instradamento locale tra due interfacce dello stesso router R1; un indirizzo privato è perfettamente valido in un dominio di routing locale.
- **LAN1 ↔ LAN2** (sedi diverse, entrambe pubbliche): **no NAT**. Sono indirizzi pubblici, nativamente instradabili sul backbone OSPF e sulla rete dell'ISP.
- **Qualsiasi comunicazione che coinvolge WLAN1 o WLAN2 verso l'altra sede** (WLAN1↔LAN2, WLAN1↔WLAN2, ecc.): **serve NAT**. Il traffico deve attraversare il backbone pubblico (R3–R9, instradato via OSPF solo su prefissi pubblici): un indirizzo sorgente privato non avrebbe un percorso di ritorno instradabile su quella rete, quindi va tradotto in un indirizzo pubblico prima di uscire da R1/R2.

Punto 5 — Dijkstra da R1. Grafo completo (R1–R9) con i pesi del disegno. Evoluzione dell'algoritmo (notazione $D(v)$ = distanza corrente, $p(v)$ = predecessore; N' = insieme dei nodi confermati):

Passo	N'	D(R3),p	D(R9),p	D(R7),p	D(R4),p
0	R1	1,R1	∞	∞	∞
1	+R3	–	3,R3	∞	6,R3
2	+R9	–	–	4,R9	6,R3
3	+R7	–	–	–	6,R3
4	+R4	–	–	–	–

Passo	N'	D(R8),p	D(R6),p	D(R5),p	D(R2),p
0	R1	∞	∞	∞	∞
1	+R3	∞	∞	∞	∞
2	+R9	∞	∞	8,R9	∞
3	+R7	6,R7	∞	8,R9	∞
4	+R4	6,R7	∞	7,R4	∞
5	+R8	–	7,R8	7,R4	∞
6	+R5	–	7,R8	–	8,R5
7	+R6	–	–	–	8,R5
8	+R2	–	–	–	–

Risultato finale (cammini minimi da R1):

- R3: 1 (R1–R3)
- R9: 3 (R1–R3–R9)
- R7: 4 (R1–R3–R9–R7)
- R4: 6 (R1–R3–R4) – *non* via R7–R8–R6–R5–R4, che costerebbe 9
- R8: 6 (R1–R3–R9–R7–R8)
- R5: 7 (R1–R3–R4–R5)
- R6: 7 (R1–R3–R9–R7–R8–R6)
- R2: 8 (R1–R3–R4–R5–R2)

Punto 6 — Minimum Spanning Tree radice R1. Attenzione: non è lo stesso albero del punto 5! Con Kruskal (ordino gli archi per peso crescente e aggiungo solo se non creano cicli):

Archi di peso 1 (tutti non creano cicli): R1–R3, R9–R7, R4–R5, R8–R6, R6–R5, R2–R5 $\Rightarrow$ si formano tre componenti: $\{R1, R3\}$, $\{R9, R7\}$, $\{R2, R4, R5, R8, R6\}$.

Archi di peso 2: R3–R9 (unisce le prime due componenti), R7–R8 (unisce tutto in un'unica componente con 9 nodi e 8 archi $\Rightarrow$ MST completo). L'arco R4–R8 (peso 2) non serve più: creerebbe un ciclo.

Arco MST	**Peso**
R1–R3	1
R3–R9	2
R9–R7	1
R7–R8	2
R8–R6	1
R6–R5	1
R5–R4	1
R5–R2	1
Totale	**10**

Gli archi `R3-R4` (peso 5) e `R9-R5` (peso 5) **non** entrano nell'MST: sono gli archi "costosi" che Dijkstra invece usa parzialmente (R3–R4) perché minimizza la distanza dalla sorgente, non il peso totale dell'albero. Il MST pesa 10, l'albero dei cammini minimi del punto 5 pesa 14 (somma dei suoi archi) ma garantisce invece le distanze minime da R1 verso ciascun nodo, cosa che il MST **non** garantisce (es. nel MST la distanza R1→R4 sarebbe $1 + 2 + 1 + 2 + 1 + 1 + 1 = 9$, contro i 6 ottenuti da Dijkstra).

